beamer named overlay specifications with beanoves

Jérôme Laurens

v1.0 2025/06/03

Abstract

This package allows the management of multiple named overlay specifications in beamer documents. Named overlay specifications are very handy both during edition and to manage complex and variable beamer overlay specifications. In particular, they allow to replace raw numbers in beamer <...> overlay specifications by logical identifiers. Demonstration files are available for download as part of the development repository. As a side effect, this is another solution to this latex.org forum query.

Contents

1 Implementation

1.1 Package declarations

```
\NeedsTeXFormat{LaTeX2e}[2025/01/01]
\ProvidesExplPackage
  {beanoves}
  {2025/06/03}
  {1.0}
  {Named overlay specifications for beamer}
```

1.2 Overview

Reserved namespace: identifiers containing the case insensitive string `beanoves` or containing the case insensitive string `bnvs` delimited by two non characters.

There are mainly two steps: parsing and resolution. Parsing occurs while executing the `\Beanoves` command to build a data model whereas the resolution translates queries into beamer specifications based on this data model.

The development is made easier due to a facility layer over expl.

1.3 Facility layer: definitions and naming

In order to make the code shorter and easier to read during development, we add a layer over LaTeX3. The `c` and `v` argument specifiers take a slightly different meaning when used in a function which name contains `bnvs` or `BNVS`. Where LaTeX3 would transform `l__bnvs_ref_tl` into `\l__bnvs_ref_tl`, bnvs will directly transform `ref` into `\l__bnvs_ref_tl`. The type of the local variable used depends on the context and may be `seq` or `int` for example. There are however a pair of exceptions mentionned below. For a better reading experience, "`ref`" will generally stand for `\l__bnvs_ref_tl`, whereas "`path` sequence" will generally stand for `\l__bnvs_path_seq`. Other similar shortcuts are used as well.

Functions with `BNVS` in their names are management functions. They belong to a deeper layer and do not really contain any logic specific to the beanoves package.

`\BNVS:c` `\BNVS:c {⟨cs core name⟩}`
`\BNVS_l:cn` `\BNVS_l:cn {⟨local variable core name⟩} {⟨type⟩}`
`\BNVS_g:cn` `\BNVS_g:cn {⟨global variable core name⟩} {⟨type⟩}`

These are naming functions internally used to focus on the discrimining part of variable or function names.

```
\cs_new:Npn \BNVS:c    #1    {  __bnvs_#1    }
\cs_new:Npn \BNVS_l:cn #1 #2 { l__bnvs_#1_#2 }
\cs_new:Npn \BNVS_g:cn #1 #2 { g__bnvs_#1_#2 }
```

`\BNVS_use_Raw:N` `\BNVS_use_Raw:N ⟨cs⟩`
`\BNVS_use_Raw:c` `\BNVS_use_Raw:c {⟨cs name⟩}`
`\BNVS_use_Raw:Nc` `\BNVS_use_Raw:Nc ⟨function⟩ {⟨cs name⟩}`
`\BNVS_use_Raw:nc` `\BNVS_use_Raw:nc {⟨tokens⟩} {⟨cs name⟩}`
`\BNVS_use:c` `\BNVS_use:c {⟨cs core⟩}`
`\BNVS_use:Nc` `\BNVS_use:Nc ⟨function⟩ {⟨cs core⟩}`
`\BNVS_use:nc` `\BNVS_use:nc {⟨tokens⟩} {⟨cs core⟩}`

`\BNVS_use_Raw:c` is a convenient wrapper over `\use:c`. possibly prepended with some code, for debugging and testing. It needs 3 expansion steps just like `\BNVS_use:c`. The other are used to expand `\use:c` enough before usage by ⟨*function*⟩ or ⟨*tokens*⟩. The first argument of ⟨*function*⟩ has type `N`. The next token after ⟨*tokens*⟩ will have type `N` too. ⟨*cs name*⟩ is a full cs name whereas ⟨*cs core*⟩ will be prepended with the appropriate prefix specific to the beanoves package.

```
\cs_new:Npn \BNVS_use_Raw:N #1 { #1 }
\cs_new:Npn \BNVS_use_Raw:c #1 {
  \exp_last_unbraced:No
  \BNVS_use_Raw:N { \cs:w #1 \cs_end: }
}

\cs_new:Npn \BNVS_use:c #1 {
  \BNVS_use_Raw:c { \BNVS:c { #1 } }
}

\cs_new:Npn \BNVS_use_Raw:NN #1 #2 {
  #1 #2
}
```

```
\cs_new:Npn \BNVS_use_Raw:nN #1 #2 {
  #1 #2
}

\cs_new:Npn \BNVS_use_Raw:Nc #1 #2 {
  \exp_args:NNc \BNVS_use_Raw:NN #1 { #2 }
}

\cs_new:Npn \BNVS_use_Raw:nc #1 #2 {
  \exp_last_unbraced:Nno
  \BNVS_use_Raw:nN { #1 } { \cs:w #2 \cs_end: }
}

\cs_new:Npn \BNVS_use:Nc #1 #2 {
  \BNVS_use_Raw:Nc #1 { \BNVS:c { #2 } }
}

\cs_new:Npn \BNVS_use:nc #1 #2 {
  \BNVS_use_Raw:nc { #1 } { \BNVS:c { #2 } }
}

\cs_new:Npn \BNVS_tl_use_unbraced:nv #1 #2 {
  \exp_last_unbraced:Nnf \use:n { #1 } { \BNVS_tl_use:nv {\use:n} { #2 } }
}
\cs_new:Npn \BNVS_tl_use:nvv #1 #2 {
  \BNVS_tl_use:nv { \BNVS_tl_use:nv { #1 } { #2 } }
}
\cs_new:Npn \BNVS_tl_use:nvvv #1 #2 {
  \BNVS_tl_use:nvv { \BNVS_tl_use:nv { #1 } { #2 } }
}
\cs_new:Npn \BNVS_tl_use:nvvvv #1 #2 {
  \BNVS_tl_use:nvvv { \BNVS_tl_use:nv { #1 } { #2 } }
}

\cs_new:Npn \BNVS_log:n #1 { }
\cs_generate_variant:Nn \BNVS_log:n { x }
```

1.3.1 Debug

\BNVS_DEBUG_ON:	\BNVS_DEBUG_ON:
\BNVS_DEBUG_OFF:	\BNVS_DEBUG_OFF:
\BNVS_DEBUG_push:nn	\BNVS_DEBUG_push:nn {⟨*types on*⟩} {⟨*types off*⟩}
\BNVS_DEBUG_pop:	\BNVS_DEBUG_pop:

These functions activate or deactivate the debug mode. They are only active in the beanoves-debug package. Manage debug messaging for one given ⟨*type*⟩ or ⟨*types*⟩. The implementation is not publicly exposed. The ⟨*type*⟩ is a single letter or **.

1.3.2 Facility layer: Functions

\BNVS_new:cpn
\BNVS_set:cpn

\BNVS_new:cpn is like \cs_new:cpn except that the name argument is tagged for beanoves package. Similarly for \BNVS_set:cpn.

```
\cs_new:Npn \BNVS_new:cpn #1 {
  \cs_new:cpn { \BNVS:c { #1 } }
}
\cs_new:Npn \BNVS_set:cpn #1 {
  \cs_set:cpn { \BNVS:c { #1 } }
}
\cs_generate_variant:Nn \cs_generate_variant:Nn { c }
\cs_new:Npn \BNVS_generate_variant:cn #1 {
  \cs_generate_variant:cn { \BNVS:c { #1 } }
}
```

1.3.3 logging

\BNVS_warning:n
\BNVS_warning:x
\BNVS_error:n
\BNVS_error:x
\BNVS_fatal:n
\BNVS_fatal:x

\BNVS_warning:n {⟨message⟩}
\BNVS_error:n {⟨message⟩}
\BNVS_fatal:n {⟨message⟩}

Very rudimentary. No long message available for error recovery.

```
\msg_new:nnn { beanoves } { :n } { #1 }
\msg_new:nnn { beanoves } { :nn } { #1~(#2) }
\cs_new:Npn \BNVS_warning:n {
  \msg_warning:nnn { beanoves } { :n }
}
\cs_new:Npn \BNVS_warning:x {
  \msg_warning:nnx { beanoves } { :n }
}
\cs_new:Npn \BNVS_error:n {
  \msg_error:nnn { beanoves } { :n }
}
\cs_generate_variant:Nn \BNVS_error:n { x }
\cs_new:Npn \BNVS_fatal:n {
  \msg_fatal:nnn { beanoves } { :n }
}
\cs_new:Npn \BNVS_fatal:x {
  \msg_fatal:nnx { beanoves } { :n }
}
\cs_new:Npn \BNVS_fatal_unreachable: {
  \BNVS_fatal:n { Unreachable }
}
\cs_new:Npn \BNVS_fatal_unreachable: { }
```

1.3.4 Facility layer: Variables

\BNVS_N_new:c \BNVS_N_new:c {⟨*type*⟩}
\BNVS_v_new:c \BNVS_v_new:c {⟨*type*⟩}

Creates a collection of typed utility functions, see usage below. These functions are undefined when no longer used. ⟨*type*⟩ is one of `tl`, `seq`...

```
\cs_new:Npn \BNVS_N_new:c #1 {
  \cs_new:cpn { BNVS_#1:c } ##1 {
    l \BNVS:c{ ##1 } \tl_if_empty:nF { ##1 } { _ } #1
  }
  \cs_new:cpn { BNVS_#1_new:c } ##1 {
    \use:c { #1_new:c } { \use:c { BNVS_#1:c } { ##1 } }
  }
  \cs_new:cpn { BNVS_#1_use:c } ##1 {
    \use:c { \cs:w BNVS_#1:c \cs_end: { ##1 } }
  }
  \cs_new:cpn { BNVS_#1_use:Nc } ##1 ##2 {
    \BNVS_use_Raw:Nc
      ##1 { \cs:w BNVS_#1:c \cs_end: { ##2 } }
  }
  \cs_new:cpn { BNVS_#1_use:nc } ##1 ##2 {
    \BNVS_use_Raw:nc
      { ##1 } { \cs:w BNVS_#1:c \cs_end: { ##2 } }
  }
}

\cs_new:Npn \BNVS_v_new:c #1 {
  \cs_new:cpn { BNVS_#1_use:Nv } ##1 ##2 {
    \exp_args:Nv ##1
      { \BNVS_use_Raw:c { BNVS_#1:c } { ##2 } }
  }
  \cs_new:cpn { BNVS_#1_use:cv } ##1 ##2 {
    \exp_args:Nnv \BNVS_use:c { ##1 }
      { \BNVS_use_Raw:c { BNVS_#1:c } { ##2 } }
  }
  \cs_new:cpn { BNVS_#1_use:nv } ##1 ##2 {
    \exp_args:Nnv \use:n { ##1 }
      { \BNVS_use_Raw:c { BNVS_#1:c } { ##2 } }
  }
}

\BNVS_N_new:c { bool }
\BNVS_N_new:c { int }
\BNVS_v_new:c { int }
\BNVS_N_new:c { tl }
\BNVS_v_new:c { tl }
\cs_new:Npn \BNVS_tl_use:Nvv #1 #2 {
  \BNVS_tl_use:nv { \BNVS_tl_use:Nv #1 { #2 } }
}
```

```
\cs_new:Npn \BNVS_tl_use:Nvvv #1 #2 {
  \BNVS_tl_use:nvv { \BNVS_tl_use:Nv #1 { #2 } }
}

\cs_new:Npn \BNVS_tl_use:Nvvvv #1 #2 {
  \BNVS_tl_use:nvvv { \BNVS_tl_use:Nv #1 { #2 } }
}

\BNVS_N_new:c { str }
\BNVS_v_new:c { str }
\BNVS_N_new:c { seq }
\BNVS_v_new:c { seq }
\cs_undefine:N \BNVS_N_new:c
```

\BNVS_use:Ncn \BNVS_use:Ncn ⟨*function*⟩ {⟨*core name*⟩} {⟨*type*⟩}

```
\cs_new:Npn \BNVS_use:Ncn #1 #2 #3 {
  \BNVS_use_Raw:c { BNVS_#3_use:Nc }   #1   { #2 }
}

\cs_new:Npn \BNVS_use:ncn #1 #2 #3 {
  \BNVS_use_Raw:c { BNVS_#3_use:nc } { #1 } { #2 }
}

\cs_new:Npn \BNVS_use:Nvn #1 #2 #3 {
  \BNVS_use_Raw:c { BNVS_#3_use:Nv }   #1   { #2 }
}

\cs_new:Npn \BNVS_use:nvn #1 #2 #3 {
  \BNVS_use_Raw:c { BNVS_#3_use:nv } { #1 } { #2 }
}

\cs_new:Npn \BNVS_use:Ncncn #1 #2 #3 {
  \BNVS_use:ncn {
    \BNVS_use:Ncn   #1   { #2 } { #3 }
  }
}

\cs_new:Npn \BNVS_use:ncncn #1 #2 #3 {
  \BNVS_use:ncn {
    \BNVS_use:ncn { #1 } { #2 } { #3 }
  }
}

\cs_new:Npn \BNVS_use:Nvncn #1 #2 #3 {
  \BNVS_use:ncn {
    \BNVS_use:Nvn   #1   { #2 } { #3 }
  }
}

\cs_new:Npn \BNVS_use:nvncn #1 #2 #3 {
  \BNVS_use:ncn {
    \BNVS_use:nvn { #1 } { #2 } { #3 }
  }
}
```

```
\cs_new:Npn \BNVS_use:Ncncncn #1 #2 #3 #4 #5 {
  \BNVS_use:ncn {
    \BNVS_use:Ncncn   #1   { #2 } { #3 } { #4 } { #5 }
  }
}

\cs_new:Npn \BNVS_use:ncncncn #1 #2 #3 #4 #5 {
  \BNVS_use:ncn {
    \BNVS_use:ncncn { #1 } { #2 } { #3 } { #4 } { #5 }
  }
}
```

\BNVS_new_c:cn \BNVS_new_c:nc {⟨type⟩} {⟨core name⟩}

```
\cs_new:Npn \BNVS_new_c:nc #1 #2 {
  \BNVS_new:cpn { #1_#2:c } {
    \BNVS_use_Raw:c { BNVS_#1_use:nc } { \BNVS_use_Raw:c { #1_#2:N } }
  }
}

\cs_new:Npn \BNVS_new_cn:nc #1 #2 {
  \BNVS_new:cpn { #1_#2:cn } ##1 {
    \BNVS_use:ncn { \BNVS_use_Raw:c { #1_#2:Nn } } { ##1 } { #1 }
  }
}

\cs_new:Npn \BNVS_new_cnn:ncN #1 #2 #3 {
  \BNVS_new:cpn { #2:cnn } ##1 {
    \BNVS_use:Ncn { #3 } { ##1 } { #1 }
  }
}

\cs_new:Npn \BNVS_new_cnn:nc #1 #2 {
  \BNVS_use_Raw:nc {
    \BNVS_new_cnn:ncN { #1 } { #1_#2 }
  } { #1_#2:Nnn }
}

\cs_new:Npn \BNVS_new_cnv:ncN #1 #2 #3 {
  \BNVS_new:cpn { #2:cnv } ##1 ##2 {
    \BNVS_tl_use:nv {
      \BNVS_use:Ncn #3 { ##1 } { #1 } {  ##2 }
    }
  }
}

\cs_new:Npn \BNVS_new_cnv:nc #1 #2 {
  \BNVS_use_Raw:nc {
    \BNVS_new_cnv:ncN { #1 } { #1_#2 }
  } { #1_#2:Nnn }
}
```

```
\cs_new:Npn \BNVS_new_cnx:ncN #1 #2 #3 {
  \BNVS_new:cpn { #2:cnx } ##1 ##2 {
    \exp_args:Nnx \use:n {
      \BNVS_use:Ncn #3 { ##1 } { #1 } {  ##2 }
    }
  }
}

\cs_new:Npn \BNVS_new_cnx:nc #1 #2 {
  \BNVS_use_Raw:nc {
    \BNVS_new_cnx:ncN { #1 } { #1_#2 }
  } { #1_#2:Nnn }
}

\cs_new:Npn \BNVS_new_cc:ncNn #1 #2 #3 #4 {
  \BNVS_new:cpn { #2:cc } ##1 ##2 {
    \BNVS_use:Ncncn #3 { ##1 } { #1 } { ##2 } { #4 }
  }
}

\cs_new:Npn \BNVS_new_cc:ncn #1 #2 {
  \BNVS_use_Raw:nc {
    \BNVS_new_cc:ncNn { #1 } { #1_#2 }
  } { #1_#2:NN }
}

\cs_new:Npn \BNVS_new_cc:nc #1 #2 {
  \BNVS_new_cc:ncn { #1 } { #2 } { #1 }
}

\cs_new:Npn \BNVS_new_cn:ncNn #1 #2 #3 #4 {
  \BNVS_new:cpn { #2:cn } ##1 {
    \BNVS_use:Ncn #3 { ##1 } { #1 }
  }
}

\cs_new:Npn \BNVS_new_cn:ncn #1 #2 {
  \BNVS_use_Raw:nc {
    \BNVS_new_cn:ncNn { #1 } { #1_#2 }
  } { #1_#2:Nn }
}

\cs_new:Npn \BNVS_new_cv:ncNn #1 #2 #3 #4 {
  \BNVS_new:cpn { #2:cv } ##1 ##2 {
    \BNVS_use:nvn {
      \BNVS_use:Ncn #3 { ##1 } { #1 }
    } { ##2 } { #4 }
  }
}

\cs_new:Npn \BNVS_new_cv:ncn #1 #2 {
  \BNVS_use_Raw:nc {
    \BNVS_new_cv:ncNn { #1 } { #1_#2 }
  } { #1_#2:Nn }
}
```

```
\cs_new:Npn \BNVS_new_cv:nc #1 #2 {
  \BNVS_new_cv:ncn { #1 } { #2 } { #1 }
}

\cs_new:Npn \BNVS_l_use:Ncn #1 #2 #3 {
  \BNVS_use_Raw:Nc   #1   { \BNVS_l:cn { #2 } { #3 } }
}

\cs_new:Npn \BNVS_l_use:ncn #1 #2 #3 {
  \BNVS_use_Raw:nc { #1 } { \BNVS_l:cn { #2 } { #3 } }
}

\cs_new:Npn \BNVS_g_use:Ncn #1 #2 #3 {
  \BNVS_use_Raw:Nc   #1   { \BNVS_g:cn { #2 } { #3 } }
}

\cs_new:Npn \BNVS_g_use:ncn #1 #2 #3 {
  \BNVS_use_Raw:nc { #1 } { \BNVS_g:cn { #2 } { #3 } }
}
```

\BNVS_new_conditional:cpnn `\BNVS_new_conditional:cpnn {⟨core⟩} ⟨parameter⟩ {⟨conditions⟩} {⟨code⟩}`

```
\cs_generate_variant:Nn \prg_new_conditional:Npnn { c }

\cs_new:Npn \BNVS_new_conditional:cpnn #1 {
  \prg_new_conditional:cpnn { \BNVS:c { #1 } }
}
\cs_generate_variant:Nn \prg_generate_conditional_variant:Nnn { c }
\cs_new:Npn \BNVS_generate_conditional_variant:cnn #1 {
  \prg_generate_conditional_variant:cnn { \BNVS:c { #1 } }
}

\cs_new:Npn \BNVS_new_conditional_vn:cNnn #1 #2 #3 #4 {
  \BNVS_new_conditional:cpnn { #1:vn } ##1 ##2 { #4 } {
    \BNVS_use:Nvn #2 { ##1 } { #3 } { ##2 } {
      \prg_return_true:
    } {
      \prg_return_false:
    }
  }
}

\cs_new:Npn \BNVS_new_conditional_vn:cnn #1 #2 {
  \BNVS_use:nc {
    \BNVS_new_conditional_vn:cNnn { #1 }
  } { #1:nn TF } { #2 }
}

\cs_new:Npn \BNVS_new_conditional_vc:cNnn #1 #2 #3 #4 {
  \BNVS_new_conditional:cpnn { #1:vc } ##1 ##2 { #4 } {
    \BNVS_use:Nvn #2 { ##1 } { #3 } { ##2 } {
      \prg_return_true:
    } {
      \prg_return_false:
    }
  }
}
```

```
\cs_new:Npn \BNVS_new_conditional_vc:cnn #1 {
  \BNVS_use:nc {
    \BNVS_new_conditional_vc:cNnn { #1 }
  } { #1:ncTF }
}

\cs_new:Npn \BNVS_new_conditional_vvc:cNnnn #1 #2 #3 #4 #5 {
  \BNVS_new_conditional:cpnn { #1:vvc } ##1 ##2 ##3 { #5 } {
    \BNVS_use:nvn {
      \BNVS_use:Nvn #2 { ##1 } { #3 }
    } { ##2 } { #4 } { ##3 } {
      \prg_return_true:
    } {
      \prg_return_false:
    }
  }
}

\cs_new:Npn \BNVS_new_conditional_vvc:cnnn #1 {
  \BNVS_use:nc {
    \BNVS_new_conditional_vvc:cNnnn { #1 }
  } { #1:nncTF }
}

\cs_new:Npn \BNVS_new_conditional_vc:cNn #1 #2 #3 {
  \BNVS_new_conditional:cpnn { #1:vc } ##1 ##2 { #3 } {
    \BNVS_tl_use:Nv #2 { ##1 } { ##2 } {
      \prg_return_true:
    } {
      \prg_return_false:
    }
  }
}

\cs_new:Npn \BNVS_new_conditional_vc:cn #1 {
  \BNVS_use:nc {
    \BNVS_new_conditional_vc:cNn { #1 }
  } { #1:ncTF }
}

\cs_new:Npn \BNVS_new_conditional_vvc:cNn #1 #2 #3 {
  \BNVS_new_conditional:cpnn { #1:vvc } ##1 ##2 ##3 { #3 } {
    \BNVS_tl_use:nv {
      \BNVS_tl_use:Nv #2 { ##1 }
    } { ##2 } { ##3 } {
      \prg_return_true:
    } {
      \prg_return_false:
    }
  }
}
```

```
\cs_new:Npn \BNVS_new_conditional_vvc:cn #1 {
  \BNVS_use:nc {
    \BNVS_new_conditional_vvc:cNn { #1 }
  } { #1:nncTF }
}
```

1.3.5 Regex

```
\cs_new:Npn \BNVS_regex_use:Nc #1 #2 {
  \BNVS_use_Raw:Nc #1 { c \BNVS:c { #2 } _regex }
}
```

1.3.6 Token lists

__bnvs_tl_clear:c	__bnvs_tl_clear:c {⟨*core key tl*⟩}
__bnvs_tl_clear_new:c	__bnvs_tl_use:c {⟨*core*⟩}
__bnvs_tl_new	__bnvs_tl_count:c {⟨*core*⟩}
__bnvs_tl_set_eq:cc	__bnvs_tl_set_eq:cc {⟨*lhs core name*⟩} {⟨*rhs core name*⟩}
__bnvs_tl_set:cn	__bnvs_tl_set:cn {⟨*core*⟩} {⟨*tl*⟩}
__bnvs_tl_set:(cv|cx|ce|co)	__bnvs_tl_set:cv {⟨*core*⟩} {⟨*value core name*⟩}
__bnvs_tl_put_left:cn	__bnvs_tl_put_left:cn {⟨*core*⟩} {⟨*tl*⟩}
__bnvs_tl_put_right:cn	__bnvs_tl_put_right:cn {⟨*core*⟩} {⟨*tl*⟩}
__bnvs_tl_put_right:(cx|ce|cv|co)	__bnvs_tl_put_right:cv {⟨*core*⟩} {⟨*value core name*⟩}
	__bnvs_tl_put_right:co {⟨*core*⟩} {⟨*tl*⟩}

These are shortcuts to

- \tl_clear:c {l__bnvs_⟨*core*⟩_tl}
- \tl_clear_new:c {l__bnvs_⟨*core*⟩_tl}
- \tl_use:c {l__bnvs_⟨*core*⟩_tl}
- \tl_set_eq:cc {l__bnvs_⟨*lhs core*⟩_tl} {l__bnvs_⟨*rhs core*⟩_tl}
- \tl_set:cv {l__bnvs_⟨*core*⟩_tl} {l__bnvs_⟨*value core*⟩_tl}
- \tl_set:cx {l__bnvs_⟨*core*⟩_tl} {⟨*tl*⟩}
- \tl_set:co {l__bnvs_⟨*core*⟩_tl} {⟨*tl*⟩}
- \tl_put_left:cn {l__bnvs_⟨*core*⟩_tl} {⟨*tl*⟩}
- \tl_put_right:cn {l__bnvs_⟨*core*⟩_tl} {⟨*tl*⟩}
- \tl_put_right:cv {l__bnvs_⟨*core*⟩_tl} {l__bnvs_⟨*value core*⟩_tl}
- \tl_put_right:co {l__bnvs_⟨*core*⟩_tl} {⟨*tl*⟩}

__bnvs_tl_clear_all:n __bnvs_tl_clear_all:n {⟨*cores*⟩}

Run __bnvs_tl_clear:c for each individual item of ⟨*cores*⟩.

```
\BNVS_new:cpn { tl_clear_all:n } #1 {
  \tl_map_inline:nn { #1 } {
    \__bnvs_tl_clear:c { ##1 }
  }
}
```

\BNVS_new_conditional_vnc:cn `\BNVS_new_conditional_vnc:cn {⟨core⟩} {⟨conditions⟩}`

Creates ⟨*core*⟩:vncTF from ⟨*core*⟩:nncTF.

```
\cs_new:Npn \BNVS_new_conditional_vnc:cNn #1 #2 #3 {
  \BNVS_new_conditional:cpnn { #1:vnc } ##1 ##2 ##3 { #3 } {
    \BNVS_tl_use:Nv #2 { ##1 } { ##2 } { ##3 } {
      \prg_return_true:
    } {
      \prg_return_false:
    }
  }
}

\cs_new:Npn \BNVS_new_conditional_vnc:cn #1 {
  \BNVS_use:nc {
    \BNVS_new_conditional_vnc:cNn { #1 }
  } { #1:nncTF }
}
```

\BNVS_new_conditional_vvnc:cn `\BNVS_new_conditional_vvnc:cn {⟨core⟩} {⟨conditions⟩}`

Creates ⟨*core*⟩:vvncTF from ⟨*core*⟩:nnncTF.

```
\cs_new:Npn \BNVS_new_conditional_vvnc:cNn #1 #2 #3 {
  \BNVS_new_conditional:cpnn { #1:vvnc } ##1 ##2 ##3 ##4 { #3 } {
    \BNVS_tl_use:nv {
      \BNVS_tl_use:Nv #2 { ##1 }
    } { ##2 } { ##3 } { ##4 } {
      \prg_return_true:
    } {
      \prg_return_false:
    }
  }
}

\cs_new:Npn \BNVS_new_conditional_vvnc:cn #1 {
  \BNVS_use:nc {
    \BNVS_new_conditional_vvnc:cNn { #1 }
  } { #1:nnncTF }
}
```

```
\cs_new:Npn \BNVS_new_conditional_vvvc:cNn #1 #2 #3 {
  \BNVS_new_conditional:cpnn { #1:vvvc } ##1 ##2 ##3 ##4 { #3 } {
    \BNVS_tl_use:nvv {
      \BNVS_tl_use:Nv #2 { ##1 }
    } { ##2 } { ##3 } { ##4 } {
      \prg_return_true:
    } {
      \prg_return_false:
    }
  }
}
```

\BNVS_new_conditional_vvvc:cn \BNVS_new_conditional_vvvc:cn {⟨*core*⟩} {⟨*conditions*⟩}

Creates ⟨*core*⟩:vvvcTF from ⟨*core*⟩:nnncTF.

```
\cs_new:Npn \BNVS_new_conditional_vvvc:cn #1 {
  \BNVS_use:nc {
    \BNVS_new_conditional_vvvc:cNn { #1 }
  } { #1:nnncTF }
}

\cs_new:Npn \BNVS_new_tl_c:c {
  \BNVS_new_c:nc { tl }
}
\BNVS_new_tl_c:c { clear }
\BNVS_new_tl_c:c { clear_new }
\BNVS_new_tl_c:c { use }
\BNVS_new_tl_c:c { count }

\BNVS_new:cpn { tl_set_eq:cc } #1 #2 {
  \BNVS_use:ncncn { \tl_set_eq:NN } { #1 } { tl } { #2 } { tl }
}

\cs_new:Npn \BNVS_new_tl_cn:c {
  \BNVS_new_cn:nc { tl }
}

\cs_new:Npn \BNVS_new_tl_cv:c #1 {
  \BNVS_new_cv:ncn { tl } { #1 } { tl }
}
\BNVS_new_tl_cn:c { set }
\BNVS_new_tl_cv:c { set }

\BNVS_new:cpn { tl_set:cx } {
  \exp_args:Nnx \__bnvs_tl_set:cn
}

\BNVS_new:cpn { tl_set:ce } {
  \exp_args:Nne \__bnvs_tl_set:cn
}

\BNVS_new:cpn { tl_set:co } {
  \exp_args:Nno \__bnvs_tl_set:cn
}
```

```
\BNVS_new_tl_cn:c { put_right }
\BNVS_new_tl_cv:c { put_right }
% \BNVS_generate_variant:cn { tl_put_right:cn } { cx }

\BNVS_new:cpn { tl_put_right:cx } {
  \exp_args:Nnnx \BNVS_use:c { tl_put_right:cn }
}

\BNVS_new:cpn { tl_put_right:ce } {
  \exp_args:Nnne \BNVS_use:c { tl_put_right:cn }
}
\BNVS_new:cpn { tl_put_right:co } {
  \exp_args:Nnno \BNVS_use:c { tl_put_right:cn }
}
\BNVS_new_tl_cn:c { put_left }
\BNVS_new_tl_cv:c { put_left }
% \BNVS_generate_variant:cn { tl_put_left:cn } { cx }
% \begin{KWN.test}{bnvs:c={ tl_put_right:co }, eringo}
% \__bnvs_tl_set:cn { TEST_A } { JES }
% \__bnvs_tl_set:co { TEST_B } { { \l__bnvs_TEST_A_tl } }
% \assert_equal_tl:vnn { TEST_B } { { JES } } A
% \end{KWN.test}

\BNVS_new:cpn { tl_put_left:cx } {
  \exp_args:Nnnx \BNVS_use:c { tl_put_left:cn }
}
```

`\__bnvs_tl_if_exist:c`*TF* `\__bnvs_tl_if_empty:cTF` {⟨*core*⟩} {⟨*yes:*⟩} {⟨*no:*⟩}
`\__bnvs_tl_if_empty:c`*TF* `\__bnvs_tl_if_exist:cTF` {⟨*core*⟩} {⟨*yes:*⟩} {⟨*no:*⟩}
`\__bnvs_tl_if_blank:v`*TF* `\__bnvs_tl_if_blank:vTF` {⟨*core*⟩} {⟨*yes:*⟩} {⟨*no:*⟩}
`\__bnvs_tl_if_eq:cn`*TF* `\__bnvs_tl_if_eq:cnTF` {⟨*core*⟩} {⟨*tl*⟩} {⟨*yes:*⟩} {⟨*no:*⟩}

These are shortcuts to

- `\tl_if_exist:cTF {l__bnvs_`⟨*core*⟩`_tl}` {⟨*yes:*⟩} {⟨*no:*⟩}
- `\tl_if_empty:cTF {l__bnvs_`⟨*core*⟩`_tl}` {⟨*yes:*⟩} {⟨*no:*⟩}
- `\tl_if_eq:cnTF {l__bnvs_`⟨*core*⟩`_tl}`{⟨*tl*⟩} {⟨*yes:*⟩} {⟨*no:*⟩}

```
\cs_new:Npn \BNVS_new_conditional_c:ncNn #1 #2 #3 #4 {
  \BNVS_new_conditional:cpnn { #2 } ##1 { #4 } {
    \BNVS_use:Ncn #3 { ##1 } { #1 } {
      \prg_return_true:
    } {
      \prg_return_false:
    }
  }
}
```

```
\cs_new:Npn \BNVS_new_conditional_c:ncn #1 #2 {
  \BNVS_use_Raw:nc {
    \BNVS_new_conditional_c:ncNn { #1 } { #1_#2:c }
  } { #1_#2:NTF }
}
\BNVS_new_conditional_c:ncn { tl } { if_empty } { p, T, F, TF }
\BNVS_new_conditional_c:ncn { tl } { if_exist } { p, T, F, TF }

\BNVS_new_conditional:cpnn { tl_if_blank:v } #1 { T, F, TF } {
  \BNVS_tl_use:Nv \tl_if_blank:nTF { #1 } {
    \prg_return_true:
  } {
    \prg_return_false:
  }
}

\cs_new:Npn \BNVS_new_conditional_cn:ncNn #1 #2 #3 #4 {
  \BNVS_new_conditional:cpnn { #2:cn } ##1 ##2 { #4 } {
    \BNVS_use:Ncn #3 { ##1 } { #1 } { ##2 } {
      \prg_return_true:
    } {
      \prg_return_false:
    }
  }
}

\cs_new:Npn \BNVS_new_conditional_cn:ncn #1 #2 {
  \BNVS_use_Raw:nc {
    \BNVS_new_conditional_cn:ncNn { #1 } { #1_#2 }
  } { #1_#2:NnTF }
}
\BNVS_new_conditional_cn:ncn { tl } { if_eq } { T, F, TF }

\cs_new:Npn \BNVS_new_conditional_cv:ncNn #1 #2 #3 #4 {
  \BNVS_new_conditional:cpnn { #2:cv } ##1 ##2 { #4 } {
    \BNVS_use:nvn {
      \BNVS_use:Ncn #3 { ##1 } { #1 }
          } { ##2 } { #1 } {
      \prg_return_true:
    } {
      \prg_return_false:
    }
  }
}

\cs_new:Npn \BNVS_new_conditional_cv:ncn #1 #2 {
  \BNVS_use_Raw:nc {
    \BNVS_new_conditional_cv:ncNn { #1 } { #1_#2 }
  } { #1_#2:NnTF }
}
\BNVS_new_conditional_cv:ncn { tl } { if_eq } { T, F, TF }

\BNVS_new:cpn { gput_right:cccn } #1 #2 #3 {
  \tl_gput_right:cn { \__bnvs_g_IPK:w { #1 } { #2 } { #3 } }
}
```

1.3.7 Strings

__bnvs_str_if_eq:vvTF __bnvs_str_if_eq:vvTF {⟨core⟩} {⟨tl⟩} {⟨yes:⟩} {⟨no:⟩}

These are shortcuts to

- \str_if_eq:ccTF l__bnvs_⟨core⟩_tl l__bnvs_⟨core⟩_tl {⟨yes:⟩} {⟨no:⟩}

```
\cs_new:Npn \BNVS_new_conditional_vv:cNn #1 #2 #3 {
  \BNVS_new_conditional:cpnn { #1:vv } ##1 ##2 { #3 } {
    \BNVS_tl_use:nv {
      \BNVS_tl_use:Nv #2 { ##1 }
    } { ##2 } {
      \prg_return_true:
    } {
      \prg_return_false:
    }
  }
}

\cs_new:Npn \BNVS_new_conditional_vv:cn #1 {
  \BNVS_use:nc {
    \BNVS_new_conditional_vvnc:cNn { #1 }
  } { #1:nnTF }
}

\cs_new:Npn \BNVS_new_conditional_vn:ncNn #1 #2 #3 #4 {
  \BNVS_new_conditional:cpnn { #2:vn } ##1 ##2 { #4 } {
    \BNVS_use:Nvn #3 { ##1 } { #1 } { ##2 } {
      \prg_return_true:
    } {
      \prg_return_false:
    }
  }
}

\cs_new:Npn \BNVS_new_conditional_vn:ncn #1 #2 {
  \BNVS_use_Raw:nc {
    \BNVS_new_conditional_vn:ncNn { #1 } { #1_#2 }
  } { #1_#2:nnTF }
}
\BNVS_new_conditional_vn:ncn { str } { if_eq } { T, F, TF }

\cs_new:Npn \BNVS_new_conditional_vv:ncNn #1 #2 #3 #4 {
  \BNVS_new_conditional:cpnn { #2:vv } ##1 ##2 { #4 } {
    \BNVS_use:nvn {
      \BNVS_use:Nvn #3 { ##1 } { #1 }
    }  { ##2 } { #1 } {
      \prg_return_true:
    } {
      \prg_return_false:
    }
  }
}
```

```
\cs_new:Npn \BNVS_new_conditional_vv:ncn #1 #2 {
  \BNVS_use_Raw:nc {
    \BNVS_new_conditional_vv:ncNn { #1 } { #1_#2 }
  } { #1_#2:nnTF }
}
\BNVS_new_conditional_vv:ncn { str } { if_eq } { T, F, TF }
```

1.3.8 Sequences

__bnvs_seq_count:c	__bnvs_seq_new:c {⟨*core*⟩}
__bnvs_seq_clear:c	__bnvs_seq_count:c {⟨*core*⟩}
__bnvs_seq_set_eq:cc	__bnvs_seq_clear:c {⟨*core*⟩}
__bnvs_seq_gset_eq:cc	__bnvs_seq_set_eq:cc {⟨$core_1$⟩} {⟨$core_2$⟩}
__bnvs_seq_use:cn	__bnvs_seq_use:cn {⟨*core*⟩} {⟨*separator*⟩}
__bnvs_seq_item:cn	__bnvs_seq_item:cn {⟨*core*⟩} {⟨*integer expression*⟩}
__bnvs_seq_remove_all:cn	__bnvs_seq_remove_all:cn {⟨*core*⟩} {⟨*tl*⟩}
__bnvs_seq_put_left:cn	__bnvs_seq_put_right:cn {⟨*seq core*⟩} {⟨*tl*⟩}
__bnvs_seq_put_left:cv	__bnvs_seq_put_right:cv {⟨*seq core*⟩} {⟨*tl core*⟩}
__bnvs_seq_put_right:cn	__bnvs_seq_set_split:cnn {⟨*seq core*⟩} {⟨*tl*⟩} {⟨*separator*⟩}
__bnvs_seq_put_right:cv	__bnvs_seq_pop_left:cc {⟨$core_1$⟩} {⟨$core_2$⟩}
__bnvs_seq_set_split:cnn	
__bnvs_seq_set_split:(cnv|cnx)	
__bnvs_seq_pop_left:cc	

These are shortcuts to

- \seq_set_eq:cc {l__bnvs_⟨*core*$_1$⟩_seq} {l__bnvs_⟨*core*$_2$⟩_seq}
- \seq_count:c {l__bnvs_⟨*core*⟩_seq}
- \seq_use:cn {l__bnvs_⟨*core*⟩_seq} {⟨*separator*⟩}
- \seq_item:cn {l__bnvs_⟨*core*⟩_seq} {⟨*integer expression*⟩}
- \seq_remove_all:cn {l__bnvs_⟨*core*⟩_seq} {⟨*tl*⟩}
- \seq_clear:c {l__bnvs_⟨*core*⟩_seq}
- \seq_put_right:cv {l__bnvs_⟨*core*⟩_seq} {l__bnvs_⟨*core*⟩_tl}
- \seq_set_split:cnn {l__bnvs_⟨*core*⟩_seq} {l__bnvs_⟨*core*⟩_tl}\KWNmeta{tl}

```
\BNVS_new_c:nc   { seq } { count }
\BNVS_new_c:nc   { seq } { clear }
\BNVS_new_cn:nc  { seq } { use }
\BNVS_new_cn:nc  { seq } { item }
\BNVS_new_cn:nc  { seq } { remove_all }
\BNVS_new_cn:nc  { seq } { map_inline }
\BNVS_new_cc:nc  { seq } { set_eq }
\BNVS_new_cc:nc  { seq } { gset_eq }
\BNVS_new_cn:ncn { seq } { put_left  } { tl }
\BNVS_new_cv:ncn { seq } { put_left  } { tl }
```

```
\BNVS_new_cn:ncn { seq } { put_right } { tl }
\BNVS_new_cv:ncn { seq } { put_right } { tl }
\BNVS_new_cnn:nc { seq } { set_split }
\BNVS_new_cnv:nc { seq } { set_split }
\BNVS_new_cnx:nc { seq } { set_split }
\BNVS_new_cc:ncn { seq } { pop_left  } { tl }
\BNVS_new_cc:ncn { seq } { pop_right } { tl }
```

__bnvs_seq_if_empty:c*TF*
__bnvs_seq_get_right:cc*TF*
__bnvs_seq_pop_left:cc*TF*
__bnvs_seq_pop_right:cc*TF*

__bnvs_seq_if_empty:cTF {⟨*seq core*⟩} {⟨*yes:*⟩} {⟨*no:*⟩}
__bnvs_seq_get_right:ccTF {⟨*seq core*⟩} {⟨*tl core*⟩} {⟨*yes:*⟩} {⟨*no:*⟩}

```
\cs_new:Npn \BNVS_new_conditional_cc:ncnn #1 #2 #3 #4 {
  \BNVS_new_conditional:cpnn { #1_#2:cc } ##1 ##2 { #4 } {
    \BNVS_use:ncncn {
      \BNVS_use_Raw:c { #1_#2:NNTF }
    } { ##1 } { #1 } { ##2 } { #3 } {
      \prg_return_true:
    } {
      \prg_return_false:
    }
  }
}
\BNVS_new_conditional_c:ncn { seq } { if_empty } { T, F, TF }
\BNVS_new_conditional_cc:ncnn
  { seq } { get_right } { tl } { T, F, TF }
\BNVS_new_conditional_cc:ncnn
  { seq } { pop_left  } { tl } { T, F, TF }
\BNVS_new_conditional_cc:ncnn
  { seq } { pop_right } { tl } { T, F, TF }
```

1.3.9 Integers

__bnvs_int_new:c
__bnvs_int_use:c
__bnvs_int_zero:c
__bnvs_int_inc:c
__bnvs_int_decr:c
__bnvs_int_set:cn
__bnvs_int_set:cv

__bnvs_int_new:c {⟨*core*⟩}
__bnvs_int_use:c {⟨*core*⟩}
__bnvs_int_incr:c {⟨*core*⟩}
__bnvs_int_decr:c {⟨*core*⟩}
__bnvs_int_set:cn {⟨*core*⟩} {⟨*value*⟩}

These are shortcuts to

- \int_new:c {l__bnvs_⟨*core*⟩_int}
- \int_use:c {l__bnvs_⟨*core*⟩_int}
- \int_incr:c {l__bnvs_⟨*core*⟩_int}
- \int_decr:c {l__bnvs_⟨*core*⟩_int}
- \int_set:cn {l__bnvs_⟨*core*⟩_int} {⟨*value*⟩}

```
\BNVS_new_c:nc   { int } { new  }
\BNVS_new_c:nc   { int } { use  }
\BNVS_new_c:nc   { int } { zero }
\BNVS_new_c:nc   { int } { incr }
\BNVS_new_c:nc   { int } { decr }
\BNVS_new_cn:nc  { int } { set  }
\BNVS_new_cv:ncn { int } { set  } { int }
```

1.4 Debug facilities

Typesetting file `beanoves.dtx` creates both beanoves and beanoves-debug style files. The former is intended for everyday use whereas the latter contains supplemental debugging and testing facilities which are intentionally left undocumented. For example, we have aliases for `\group_begin:` and `\group_end:` to allow the display of supplemental informations while debugging. We also have some techniques for coverage as well as inline testing.

1.5 Local variables

We make heavy use of local variables and function scopes. Many functions are executed within a TEX group, which ensures no name collision with the caller stack. The number of variables used has not been optimized, nor the TEX groups used. Optimization often goes against readability. Next local variables are used by different functions. These are one word letter variables.

\l___bnvs__A__tl: ,

\l___bnvs__Z__tl: ,

\l___bnvs__L__tl: Start, end and lentgh of a range

```
    \tl_new:N \l__bnvs_A_tl
    \tl_new:N \l__bnvs_Z_tl
    \tl_new:N \l__bnvs_L_tl
```

\l___bnvs__C__tl: A function name in a call, as regex catched group

\l___bnvs__I__tl: A frame identifier

\l___bnvs__J__tl: The current frame identifier

\l___bnvs__S__tl: A short name, a regex catched group.

\l___bnvs__R__tl: A relative, a regex catched group.

\l___bnvs__P__tl: A path, a regex catched group.

\l___bnvs__M__tl: An assignment modifier, a regex catched group.

```
    \tl_new:N \l__bnvs_I_tl
    \tl_new:N \l__bnvs_J_tl
    \tl_new:N \l__bnvs_K_tl
```

```
      \tl_new:N \l__bnvs_S_tl
      \tl_new:N \l__bnvs_R_tl
      \tl_new:N \l__bnvs_P_tl
      \tl_new:N \l__bnvs_M_tl
```

\l___bnvs__scanned__tl: To store the scanned material.

```
      \tl_new:N \l__bnvs_xprt_tl
      \tl_new:N \l__bnvs_xprted_Raw_tl
      \tl_new:N \l__bnvs_xprted_tl
```

\l___bnvs__N__tl: The representation of an unsigned decimal integer.

```
      \tl_new:N \l__bnvs_N_tl
```

\l___bnvs__V__tl: A number value as opposit of a range value.

```
      \tl_new:N \l__bnvs_V_tl
```

\l___bnvs__H__tl: The left hand side of an assignment.

```
      \tl_new:N \l__bnvs_H_tl
```

```
\tl_new:N \l__bnvs_B_tl
\tl_new:N \l__bnvs_T_tl
\tl_new:N \l__bnvs_U_tl
\tl_new:N \l__bnvs_F_tl
\tl_new:N \l__bnvs_Q_tl
\tl_new:N \l__bnvs_a_tl
\tl_new:N \l__bnvs_z_tl
\tl_new:N \l__bnvs_l_tl
\tl_new:N \l__bnvs_r_tl
\tl_new:N \l__bnvs_n_tl
\tl_new:N \l__bnvs_ref_tl
\tl_new:N \l__bnvs_v_tl
\tl_new:N \l__bnvs_b_tl
\tl_new:N \l__bnvs_c_tl
\tl_new:N \l__bnvs_ans_tl
\tl_new:N \l__bnvs_base_tl
\tl_new:N \l__bnvs_group_tl
\tl_new:N \l__bnvs_scaN_tl
\tl_new:N \l__bnvs_query_tl
\tl_new:N \l__bnvs_n_incr_tl
\tl_new:N \l__bnvs_incr_tl
\tl_new:N \l__bnvs_plus_tl
\tl_new:N \l__bnvs_rhs_tl
\tl_new:N \l__bnvs_post_tl
\tl_new:N \l__bnvs_suffix_tl
\tl_new:N \l__bnvs_index_tl
\int_new:N \g__bnvs_call_int
\int_new:N \l__bnvs_i_int
\seq_new:N \g__bnvs_def_seq
\seq_new:N \l__bnvs_a_seq
```

```
\seq_new:N \l__bnvs_b_seq
\seq_new:N \l__bnvs_ans_seq
\seq_new:N \l__bnvs_Q_seq
\seq_new:N \l__bnvs_R_seq
\seq_new:N \l__bnvs_match_seq
\seq_new:N \l__bnvs_split_seq
\seq_new:N \l__bnvs_P_seq
\bool_new:N \l__bnvs_parse_bool
\bool_set_false:N \l__bnvs_parse_bool
\bool_new:N \l__bnvs_deep_bool
\bool_set_false:N \l__bnvs_deep_bool
\bool_new:N \l__bnvs_no_range_bool
\bool_set_false:N \l__bnvs_no_range_bool

\BNVS_new:cpn { tl_clear_ans: } {
  \__bnvs_tl_clear:c { ans }
}

\cs_new:Npn \BNVS_error_ans:x {
  \__bnvs_tl_put_right:cn { ans } 0
  \BNVS_error:x
}
```

In order to implement the provide feature, we add getters and setters

```
\BNVS_new:cpn { set_true:c } #1 {
  \exp_args:Nc \bool_set_true:N { \BNVS_l:cn { #1 } { bool } }
}

\BNVS_new:cpn { set_false:c } #1 {
  \exp_args:Nc \bool_set_false:N { \BNVS_l:cn { #1 } { bool } }
}
```

1.6 Infinite loop management

Unending recursivity is managed here.

\g__bnvs_call_int

Some functions calls, as well as some loop bodies, decrement this counter. When this counter reaches 0, an error is raised or a computation is aborted.

(*End of definition for* `\g__bnvs_call_int`.)

```
\int_const:Nn \c__bnvs_max_call_int { 8192 }
```

__bnvs_greset_call:

`\__bnvs_greset_call:`

Reset globally the call stack counter to its maximum value.

```
\BNVS_new:cpn  { greset_call: } {
  \int_gset:Nn \g__bnvs_call_int { \c__bnvs_max_call_int }
}
```

__bnvs_if_call:TF __bnvs_call_do:TF {⟨yes:⟩} {⟨no:⟩}

Decrement the \g__bnvs_call_int counter globally and execute ⟨yes:⟩ if we have not reached 0, ⟨no:⟩ otherwise.

```
\BNVS_new_conditional:cpnn { if_call: } { T, F, TF } {
  \int_gdecr:N \g__bnvs_call_int
  \int_compare:nNnTF \g__bnvs_call_int > 0 {
    \prg_return_true:
  } {
    \prg_return_false:
  }
}
```

1.7 Overlay specification

1.7.1 Registration

We keep track of the ⟨id⟩ ! ⟨path⟩ combinations and provide looping mechanisms. Hereafter, ⟨id⟩, ⟨path⟩ and ⟨key⟩ allways represent pure strings.

__bnvs_g_IPK:w __bnvs_g_IPK:w {⟨id⟩} {⟨path⟩} {⟨key⟩}
__bnvs_g_IP:w __bnvs_g_IP:w {⟨id⟩} {⟨path⟩}
__bnvs_g_I:w __bnvs_g_I:w {⟨id⟩}

Create a unique global name from the arguments.

```
\BNVS_new:cpn { g_IPK:w } #1 #2 #3 { g__bnvs@#1!#2/#3_tl }
\BNVS_new:cpn { g_IP:w } #1 #2 { g__bnvs@#1!#2_keys_seq }
\BNVS_new:cpn { g_I:w } #1 { g__bnvs@#1_paths_seq }
```

\g__bnvs_I_seq Global list of globally registered identifiers.

(*End of definition for* \g__bnvs_I_seq.)

```
\seq_new:N \g__bnvs_I_seq
```

1.7.2 Data model

The whole data model is somehow similar to a hierarchical file system: the ⟨id⟩ corresponds to the drive, the ⟨path⟩ points to a directory, each directory may contain individual files. File named "=" contains initial data whereas file named "@" contains the static resolved data. The latter is initially undefined and becomes empty as soon as the initial data is defined or set. In order to allow index override, files respectively named "⟨N⟩" and "@⟨N⟩" are used similarly, ⟨N⟩ denoting a normalized integer (no leading 0, no leading + sign). The dot (.) is used to separate components of ⟨path⟩.

1.7.3 Low level IPK model functions

These are `gset` and `gunset` functions. Each `gset` function must be balanced by a `gunset` function to prevent "memory leaks".

`\__bnvs_gset:nnnn` `\__bnvs_gset:nnnn {⟨id⟩} {⟨path⟩} {⟨key⟩} {⟨def⟩}`

`\__bnvs_gset:(nnnv|nnne|vnnn|nvnn|nvnv|vvnn|vvnv)`

Manage the storage. Keep track of all the ⟨*path*⟩'s for a given ⟨*id*⟩ and all the ⟨*key*⟩'s for a given ⟨*id*⟩`!`⟨*path*⟩ combination.

Implementation details, next macros are defined lazily at the global level:

- `\__bnvs_gset@`⟨*id*⟩`!`⟨*path*⟩`:nn {`⟨*key*⟩`} {`⟨*def*⟩`}`,
- `\__bnvs_gset@`⟨*id*⟩`:nnn {`⟨*path*⟩`} {`⟨*key*⟩`} {`⟨*def*⟩`}`,
- `\g__bnvs@`⟨*id*⟩`!_paths_tl`, records all ⟨*path*⟩ for ⟨*id*⟩,
- `\g__bnvs@`⟨*id*⟩`!`⟨*path*⟩`_keys_tl`, records all ⟨*key*⟩ for ⟨*id*⟩`!`⟨*path*⟩,
- `\g__bnvs@`⟨*id*⟩`!`⟨*path*⟩`/`⟨*key*⟩`_tl`, record the ⟨*def*⟩ for the ⟨*id*⟩`!`⟨*path*⟩`/`⟨*key*⟩ combination.

```
\BNVS_new:cpn { gset:Nn } #1 {
  \tl_gclear_new:N #1
  \tl_gset:Nn #1
}
\BNVS_new:cpn { gset_IPKV:NNw } #1 #2 #3 #4 {
  \seq_gclear_new:N #1
  \cs_new:Npn #2 ##1 ##2 {
    \seq_if_in:NnF #1 { ##1 } {
      \seq_gput_right:Nn #1 { ##1 }
    }
    \exp_args:Nc \__bnvs_gset:Nn {
      \__bnvs_g_IPK:w { #3 } { #4 } { ##1 }
    } { ##2 }
  }
  #2
}
\BNVS_new:cpn { gset:NNnnnn } #1 #2 #3 #4 {
  \cs_if_exist_use:NF #1 {
    \seq_gput_right:Nn \g__bnvs_I_seq { #3 }
    \seq_gclear_new:N #2
    \cs_new:Npn #1 ##1 {
      \seq_if_in:NnF #2 { ##1 } {
        \seq_gput_right:Nn #2 { ##1 }
      }
      \exp_args:Ncc \__bnvs_gset_IPKV:NNw
      { \__bnvs_g_IP:w { #3 } { ##1 } }
      { __bnvs_gset@#3!##1:nn } { #3 } { ##1 }
    }
    #1
  }
```

```
  { #4 }
}
\BNVS_new:cpn { gset:nnnn } #1 #2 #3 #4 {
  \cs_if_exist_use:cF { __bnvs_gset@#1!#2:nn } {
    \exp_args:Ncc \__bnvs_gset:NNnnnn
      { __bnvs_gset@#1:nnn } { \__bnvs_g_I:w { #1 } } { #1 } { #2 }
  } { #3 } { #4 }
}
\BNVS_new:cpn { gset:vnnn } {
  \BNVS_tl_use:cv { gset:nnnn }
}

\BNVS_new:cpn { gset:nvnn } #1 {
  \BNVS_tl_use:nv { \__bnvs_gset:nnnn { #1 } }
}

\BNVS_new:cpn { gset:vvnn } {
  \BNVS_tl_use:Nvv \__bnvs_gset:nnnn
}

\BNVS_new:cpn { gset:nnnv } #1 #2 #3 {
  \BNVS_tl_use:nv {
    \__bnvs_gset:nnnn { #1 } { #2 } { #3 }
  }
}

\BNVS_new:cpn { gset:vvnv } #1 #2 #3 {
  \BNVS_tl_use:nv {
    \__bnvs_gset:vvnn { #1 } { #2 } { #3 }
  }
}

\BNVS_new:cpn { gset:nvnv } {
  \BNVS_tl_use:Nv \__bnvs_gset:nnnv
}

\cs_generate_variant:Nn \__bnvs_gset:nnnn { nnne }
```

`\__bnvs_gset:nnv` `\__bnvs_gset:nnv {⟨id⟩} {⟨path⟩} {⟨key⟩}`

Syntactic sugar for `\__bnvs_gset:nnnv {⟨id⟩} {⟨path⟩} {⟨key⟩} {⟨key⟩}`.

```
\BNVS_new:cpn { gset:nnv } #1 #2 #3 {
  \__bnvs_gset:nnnv { #1 } { #2 } { #3 } { #3 }
}
```

`\__bnvs_gunset:nnn`
`\__bnvs_gunset:(nvn|vvn)`

`\__bnvs_gunset:nnv {⟨id⟩} {⟨path⟩} {⟨key⟩}`

Removes the specifications for the ⟨*id*⟩ ! ⟨*path*⟩ / ⟨*key*⟩ combination.

```
\seq_new:N \l__bnvs_I_seq
\BNVS_new:cpn { seq_gremove_once:Nn } #1 #2 {
  \seq_clear:N \l__bnvs_I_seq
  \cs_set:Npn \BNVS_seq_gremove_Nn:n ##1 {
    \tl_if_eq:nnTF { ##1 } { #2 } {
      \cs_set:Npn \BNVS_seq_gremove_Nn:n ####1 {
        \seq_put_right:Nn \l__bnvs_I_seq { ####1 }
      }
    } {
      \seq_put_right:Nn \l__bnvs_I_seq { ##1 }
    }
  }
  \seq_map_function:NN #1 \BNVS_seq_gremove_Nn:n
  \seq_gset_eq:NN #1 \l__bnvs_I_seq
}

\BNVS_new:cpn { gunset_IP:Nw } #1 #2 #3 {
  \__bnvs_seq_gremove_once:Nn #1 { #3 }
  \seq_if_empty:NT #1 {
    \__bnvs_seq_gremove_once:Nn \g__bnvs_I_seq { #2 }
    \cs_undefine:c { __bnvs_gset@#2:nnn}
  }
}

\BNVS_new:cpn { gunset:NNnnn } #1 #2 #3 #4 #5 {
  \tl_if_exist:NT #1 {
    \cs_undefine:N #1
    \__bnvs_seq_gremove_once:Nn #2 { #5 }
    \seq_if_empty:NT #2 {
      \exp_args:Nc \__bnvs_gunset_IP:Nw { \__bnvs_g_I:w { #3 } } { #3 } { #4 }
      \cs_undefine:c { __bnvs_gset@#3!#4:nn}
    }
  }
}

\BNVS_new:cpn { gunset:nnn } #1 #2 #3 {
  \exp_args:Ncc
  \__bnvs_gunset:NNnnn
    { \__bnvs_g_IPK:w { #1 } { #2 } { #3 } }
    { \__bnvs_g_IP:w  { #1 } { #2 } }
    { #1 } { #2 } { #3 }
}

\BNVS_new:cpn { gunset:nvn } #1 {
  \BNVS_tl_use:nv { \__bnvs_gunset:nnn { #1 } }
}

\BNVS_new:cpn { gunset:vvn } {
  \BNVS_tl_use:Nvv \__bnvs_gunset:nnn
}
```

__bnvs_is_gset:nnn*TF* __bnvs_is_gset:nnnTF {⟨*id*⟩} {⟨*path*⟩} {⟨*key*⟩} {⟨*yes:*⟩} {⟨*no:*⟩}
__bnvs_is_gset:(nvn|vvn|vxn)*TF*

Test for the existence of some data for that ⟨*id*⟩ ! ⟨*path*⟩ / ⟨*key*⟩ combination.

```
\BNVS_new_conditional:cpnn { is_gset:nnn } #1 #2 #3 { T, F, TF } {
  \tl_if_exist:cTF { \__bnvs_g_IPK:w { #1 } { #2 } { #3 } }
    \prg_return_true: \prg_return_false:
}

\BNVS_new_conditional:cpnn { is_gset:vxn } #1 #2 #3 { T, F, TF } {
  \exp_args:NNnx \BNVS_tl_use:Nv
  \__bnvs_is_gset:nnnTF { #1 } { #2 } { #3 }
    \prg_return_true: \prg_return_false:
}

\BNVS_new_conditional:cpnn { is_gset:nvn } #1 #2 #3 { T, F, TF } {
  \BNVS_tl_use:nv { \__bnvs_is_gset:nnnTF { #1 } } { #2 } { #3 }
    \prg_return_true: \prg_return_false:
}

\BNVS_new_conditional:cpnn { is_gset:vvn } #1 #2 #3 { T, F, TF } {
  \BNVS_tl_use:Nvv \__bnvs_is_gset:nnnTF { #1 } { #2 } { #3 }
    \prg_return_true: \prg_return_false:
}
```

1.7.4 Loops

__bnvs_foreach_I:N
__bnvs_foreach_I:n
__bnvs_foreach_I_break:
__bnvs_foreach_I_break:n

__bnvs_foreach_I:N ⟨*function:n*⟩
__bnvs_foreach_I:n {⟨*:n code*⟩}
__bnvs_foreach_I_break:
__bnvs_foreach_I_break:n {⟨*code*⟩}

Execute the ⟨*function:n*⟩ or the ⟨*:n code*⟩ for each declared identifier. The break commands work like \seq_map_break: and

```
\BNVS_new:cpn { foreach_I:N } {
  \seq_map_function:NN \g__bnvs_I_seq
}

\BNVS_new:cpn { foreach_I:n } {
  \seq_map_inline:Nn \g__bnvs_I_seq
}

\cs_set_eq:NN \__bnvs_foreach_I_break:  \seq_map_break:
\cs_set_eq:NN \__bnvs_foreach_I_break:n \seq_map_break:n
```

__bnvs_foreach_P:nN*TF*
__bnvs_foreach_P:nn*TF*

__bnvs_foreach_P:nNTF {⟨*id*⟩} ⟨*function:nn*⟩ {⟨*yes:*⟩} {⟨*no:*⟩}
__bnvs_foreach_P:nnTF {⟨*id*⟩} {⟨*code:nn*⟩} {⟨*yes:*⟩} {⟨*no:*⟩}

If ⟨*id*⟩ is a declared identifier, execute ⟨*function:nn*⟩ or ⟨*code:nn*⟩ for each combination of ⟨*id*⟩ and its associate ⟨*path*⟩s. Both are the arguments passed to ⟨*function:nn*⟩ or ⟨*:nn code*⟩ body (through ##1 and ##2).

```
\BNVS_new_conditional:cpnn { foreach_P:nN } #1 #2 { T, F, TF } {
  \seq_if_exist:cTF { \__bnvs_g_I:w { #1 } } {
    \seq_map_inline:cn {
      \__bnvs_g_I:w { #1 }
    } {
```

```
      #2 { #1 } { ##1 }
    }
    \prg_return_true:
  } {
    \prg_return_false:
  }
}

\BNVS_new_conditional:cpnn { foreach_P:nn } #1 #2 { T, F, TF } {
  \seq_if_exist:cTF { \__bnvs_g_I:w { #1 } } {
    \cs_set:Npn \BNVS_foreach_P_nn:nn ##1 ##2 { #2 }
    \seq_map_inline:cn { \__bnvs_g_I:w { #1 } } {
      \BNVS_foreach_P_nn:nn { #1 } { ##1 }
    }
    \prg_return_true:
  } {
    \prg_return_false:
  }
}
```

`\__bnvs_foreach_K:nnN` `\__bnvs_foreach_K:nnN {⟨id⟩} {⟨path⟩} ⟨function:nnn⟩`
`\__bnvs_foreach_K:nnn` `\__bnvs_foreach_K:nnn {⟨id⟩} {⟨path⟩} {⟨code:nnn⟩}`

Execute the ⟨*function:nnn*⟩ or the ⟨*code:nnn*⟩ body for each of ⟨*id*⟩ ! ⟨*tag*⟩ / ⟨*key*⟩ combination. These are the arguments to the function or code body.

```
\BNVS_new:cpn { foreach_K:NnnN } #1 #2 #3 #4 {
  \seq_if_exist:NT #1 {
    \seq_map_inline:Nn #1 {
      #4 { #2 } { #3 } { ##1 }
    }
  }
}

\BNVS_new:cpn { foreach_K:nnN } #1 #2 #3 {
  \exp_args:Nc \__bnvs_foreach_K:NnnN
    { \__bnvs_g_IP:w { #1 } { #2 } }
    { #1 } { #2 } #3
}

\BNVS_new:cpn { foreach_K:nnn } #1 #2 #3 {
  \cs_set:Npn \BNVS_foreach_K_nnn:w ##1 ##2 ##3 { #3 }
  \__bnvs_foreach_K:nnN { #1 } { #2 } \BNVS_foreach_K_nnn:w
}
```

`\__bnvs_foreach_R:nnTF` `\__bnvs_foreach_R:nnNTF {⟨id⟩} {⟨tag⟩} ⟨function:nn⟩ {⟨yes:⟩} {⟨no:⟩}`
`\__bnvs_foreach_R:nnnTF` `\__bnvs_foreach_R:nnnTF {⟨id⟩} {⟨tag⟩} {⟨code:nn⟩} {⟨yes:⟩} {⟨no:⟩}`

If the ⟨*id*⟩ is known, execute the ⟨*function:nn*⟩ or the ⟨*code:nn*⟩ body for each of ⟨*id*⟩ ! ⟨*tag*⟩ . ⟨*subtag*⟩ combination. The ⟨*id*⟩ and ⟨*tag*⟩ . ⟨*subtag*⟩ are the arguments of the function and the function body. After that the ⟨*yes:*⟩ code is executed if some combination is found, otherwise it executes ⟨*no:*⟩ code.

```
\BNVS_new_conditional:cpnn { foreach_R:nnn } #1 #2 #3 { T, F, TF } {
  \cs_set:Npn \BNVS_foreach_R_nnnTF:nn ##1 ##2 {
```

```
    #3
  }
  \__bnvs_foreach_R:nnNTF { #1 } { #2  } \BNVS_foreach_R_nnnTF:nn
    \prg_return_true: \prg_return_false:
}

\BNVS_new_conditional:cpnn { foreach_R:nnN } #1 #2 #3 { T, F, TF } {
  \seq_if_exist:cTF { \__bnvs_g_I:w { #1 } } {
```

This is not reentrant.

```
    \seq_map_inline:cn { \__bnvs_g_I:w { #1 } } {
      \cs_set:Npn \BNVS_foreach_R_nnnTF_return: { \prg_return_true: }
      \str_if_in:nnTF { ~ ##1 . } { ~ #2 . } {
        #3 { #1 } { ##1 }
      } {
      }
    }
    \BNVS_foreach_R_nnnTF_return:
  } {
    \prg_return_false:
  }
}
```

`\__bnvs_foreach_IP:N` `\__bnvs_foreach_IP:N` ⟨*function:nn*⟩
`\__bnvs_foreach_IP:n` `\__bnvs_foreach_IP:n` {⟨*code:nn*⟩}

Execute the ⟨*function:nn*⟩ or the ⟨*code:nn*⟩ for each ⟨*id*⟩ ! ⟨*path*⟩ combination.

```
\BNVS_new:cpn { foreach_IP:N } #1 {
  \__bnvs_foreach_I:n {
    \__bnvs_foreach_P:nNT { ##1 } #1 { }
  }
}

\BNVS_new:cpn { foreach_IP:NT } #1 #2 {
  \__bnvs_foreach_I:n {
    \__bnvs_foreach_P:nNT { ##1 } #1 { #2 }
  }
}

\BNVS_new:cpn { foreach_IP:n } #1 {
  \cs_set:Npn \BNVS_foreach_IP_n:nn ##1 ##2 { #1 }
  \__bnvs_foreach_I:n {
    \__bnvs_foreach_P:nNT { ##1 } \BNVS_foreach_IP_n:nn { }
  }
}
```

1.7.5 Path function

__bnvs_walk_path:nnnn `\__bnvs_walk_path:nnnn {⟨path⟩} {⟨relative⟩} {⟨first:n⟩} {⟨other:n⟩}`

UNUSED ⟨*path*⟩ is a dot separated list of components.

```
\BNVS_seq_new:c { walk }
\BNVS_set:cpn { walk_path:nnnn } #1 #2 #3 #4 {
  \BNVS_set:cpn { walk_path_ii:n }   ##1 { #2 }
  \BNVS_set:cpn { walk_path_iii:n }  ##1 { #3 }
  \BNVS_set:cpn { walk_path_iiii:n } ##1 { #4 }
  \BNVS_seq_use:Nc \seq_set_split:Nnn { walk } . { #1 }
  \BNVS_set:cpn { walk_path_next: } { }
  \cs_set:Npn \KWN:n ##1 {
    \BNVS_set:cpn { walk_path_next: } {
      \__bnvs_walk_path_ii:n { ##1 }
    }
    \cs_set:Npn \KWN:n ####1 {
      \__bnvs_walk_path_iii:n { ##1 }
      \BNVS_set:cpn { walk_path_next: } { }
      \cs_set:Npn \KWN:n ########1 {
        \__bnvs_walk_path_iiii:n { ########1 }
      }
      \__bnvs_walk_path_iiii:n { ####1 }
    }
  }
  \BNVS_seq_use:Nc \seq_map_function:NN { walk } \KWN:n
  \__bnvs_walk_path_next:
}
```

__bnvs_foreach_relative:nn `\__bnvs_foreach_relative:nn {⟨relative⟩} {⟨code:n⟩}`

⟨*relative*⟩ is a dot separated list of components starting with a dot.

```
\BNVS_set:cpn { foreach_relative:nn } #1 #2 {
  \BNVS_set:cpn { foreach_relative:n } ##1 { #2 }
  \BNVS_seq_use:Nc \seq_set_split:Nnn { walk } . { #1 }
  \cs_set:Npn \KWN:n ##1 {
    \cs_set:Npn \KWN:n { \__bnvs_foreach_relative:n }
  }
  \BNVS_seq_use:Nc \seq_map_function:NN { walk } \KWN:n
}
```

1.7.6 Low level IP model functions

1.7.7 Low level I model functions

1.7.8 Getting

__bnvs_if_get:nnnTF
__bnvs_if_spec:nnnTF

__bnvs_if_get:nnnTF {⟨id⟩} {⟨path⟩} {⟨key⟩} {⟨yes:n⟩} {⟨no:⟩}
__bnvs_if_spec:nnnTF {⟨id⟩} {⟨path⟩} {⟨key⟩} {⟨yes:n⟩} {⟨no:⟩}

The __bnvs_if_get:nnnTF variant calls ⟨yes:n⟩ with what was stored for ⟨id⟩ ! ⟨path⟩ / ⟨key⟩. Otherwise executes ⟨no:⟩ without changing the contents of the ⟨ans⟩ tl variable.

The _spec: variant is similar except that it uses ⟨id⟩ ! ⟨path⟩ / ⟨key⟩ combinations when available or ! ⟨path⟩ / ⟨key⟩ otherwise.

```
\BNVS_new:cpn { if_get:nnnTF } #1 #2 #3 #4 #5 {
  \__bnvs_is_gset:nnnTF { #1 } { #2 } { #3 } {
    \cs_set:Npn \KWN:n ##1 { #4 }
    \exp_args:Nv \KWN:n { \__bnvs_g_IPK:w { #1 } { #2 } { #3 } }
  } {
    #5
  }
}

\BNVS_new:cpn { if_spec:nnnTF } #1 #2 #3 #4 #5 {
  \__bnvs_is_gset:nnnTF { #1 } { #2 } { #3 } {
    \cs_set:Npn \KWN:n ##1 { #4 }
    \exp_args:Nv \KWN:n { \__bnvs_g_IPK:w { #1 } { #2 } { #3 } }
  } {
    \tl_if_empty:nTF { #1 } {
      #5
    } {
      \__bnvs_is_gset:nnnTF {} { #2 } { #3 } {
        \cs_set:Npn \KWN:n ##1 { #4 }
        \exp_args:Nv \KWN:n { \__bnvs_g_IPK:w {} { #2 } { #3 } }
      } {
        #5
      }
    }
  }
}
```

__bnvs_if_get:nnncTF
__bnvs_if_spec:nnncTF
__bnvs_if_get:nncTF
__bnvs_if_get:vvcTF

__bnvs_if_get:nnncTF {⟨id⟩} {⟨path⟩} {⟨key⟩} {⟨ans⟩} {⟨yes:⟩} {⟨no:⟩}
__bnvs_if_spec:nnncTF {⟨id⟩} {⟨path⟩} {⟨key⟩} {⟨ans⟩} {⟨yes:⟩} {⟨no:⟩}
__bnvs_if_get:nncTF {⟨id⟩} {⟨path⟩} {⟨ans⟩} {⟨yes:⟩} {⟨no:⟩}

The __bnvs_if_get:nnnc... variant puts what was stored for ⟨id⟩ ! ⟨path⟩ / ⟨key⟩ into the ⟨ans⟩ variable, if any, then executes ⟨yes:⟩. Otherwise executes ⟨no:⟩ without changing the contents of the ⟨ans⟩ tl variable.

__bnvs_if_get:nncTF is a convenient shortcut for __bnvs_if_get:nnncTF when ⟨key⟩ and ⟨ans⟩ happen to be the same.

The _spec: variant is similar except that it uses ⟨id⟩ ! ⟨path⟩ / ⟨key⟩ combinations when available or ! ⟨path⟩ / ⟨key⟩ otherwise.

```
\BNVS_new_conditional:cpnn { if_get:nnnc } #1 #2 #3 #4 { T, F, TF } {
```

```
  \__bnvs_is_gset:nnnTF { #1 } { #2 } { #3 } {
    \BNVS_tl_use:Nc \cs_set_eq:Nc
      { #4 } { \__bnvs_g_IPK:w { #1 } { #2 } { #3 } }
    \prg_return_true:
  } {
    \prg_return_false:
  }
}
\BNVS_undefine:c { if_get:nnnc }

\BNVS_new_conditional:cpnn { if_spec:nnnc } #1 #2 #3 #4 { T, F, TF } {
  \__bnvs_is_gset:nnnTF { #1 } { #2 } { #3 } {
    \tl_set_eq:cc { \BNVS_l:cn { #4 } { tl } } {
      \__bnvs_g_IPK:w { #1 } { #2 } { #3 }
    }
    \prg_return_true:
  } {
    \tl_if_empty:nTF { #1 } {
      \prg_return_false:
    } {
      \__bnvs_if_spec:nnncTF {} { #2 } { #3 } { #4 }
        \prg_return_true: \prg_return_false:
    }
  }
}
\BNVS_undefine:c { if_spec:nnnc }

\BNVS_new_conditional:cpnn { if_get:nnc } #1 #2 #3 { T, F, TF } {
  \__bnvs_if_get:nnncTF { #1 } { #2 } { #3 } { #3 }
    \prg_return_true: \prg_return_false:
}

\BNVS_new_conditional:cpnn { if_get:vvc } #1 #2 #3 { T, F, TF } {
  \BNVS_tl_use:Nvv \__bnvs_if_get:nncTF { #1 } { #2 } { #3 }
    \prg_return_true: \prg_return_false:
}
```

`\__bnvs_if_resolved:nnc`*TF* `\__bnvs_if_resolved:nnncTF {`⟨*id*⟩`} {`⟨*path*⟩`} {`⟨*ans*⟩`} {`⟨*yes:*⟩`} {`⟨*no:*⟩`}`

Execute ⟨*yes:*⟩ if resolution succeeded for either ⟨*id*⟩`!`⟨*path*⟩`/r` or the fallback `!`⟨*path*⟩`/r`, otherwise execute ⟨*no:*⟩.

```
\BNVS_new_conditional:cpnn { if_resolved:nnc } #1 #2 #3 { T, F, TF } {
  \__bnvs_if_get:nnncTF { #1 } { #2 } r { #3 } {
    \prg_return_true:
    \prg_return_false:
  } {
    \tl_if_empty:nTF { #1 } {
      \prg_return_false:
    } {
      \__bnvs_if_resolved:nnncTF { } { #2 } r { #3 }
        \prg_return_true:
        \prg_return_false:
    }
  }
```

```
}
\BNVS_undefine:c { if_resolved:nnc }
```

`\__bnvs_if_resolved:nnncTF` `\__bnvs_if_resolved:nnncTF {⟨id⟩} {⟨path⟩} {⟨key⟩} {⟨ans⟩} {⟨yes:⟩} {⟨no:⟩}`

Execute ⟨*yes:*⟩ if resolution succeeded for either ⟨*id*⟩ `!` ⟨*path*⟩ `/` ⟨*key*⟩ or the fallback `!` ⟨*path*⟩ `/` ⟨*key*⟩, otherwise execute ⟨*no:*⟩.

```
\BNVS_new_conditional:cpnn { if_resolved:nnnc } #1 #2 #3 #4 { T, F, TF } {
  \__bnvs_if_get:nnncTF { #1 } { #2 } { #3 } { #4 } {
    \__bnvs_is_will_gset:cTF { #3 }
    \prg_return_true: \prg_return_false:
  } {
    \tl_if_empty:nTF { #1 } {
      \prg_return_false:
    } {
      \__bnvs_if_resolved:nnncTF { #1 } { #2 } { #3 } { #4 }
        \prg_return_true: \prg_return_false:
    }
  }
}
```

`\__bnvs_if_resolved_IP:wTF` `\__bnvs_if_resolved_IP:wTF {⟨id⟩} {⟨path⟩} {⟨yes:n⟩} {⟨no:⟩}`

If ⟨*id*⟩ `!` ⟨*path*⟩ `/r` has been set, execute ⟨*yes:n*⟩ code with that value as argument, otherwise execute ⟨*no:*⟩ code.

```
\BNVS_tl_new:c { if_resolved_IP_wTF }
\BNVS_new:cpn { if_resolved_IP:wTF } #1 #2 {
  \__bnvs_if_get:nnncTF { #1 } { #2 } r { if_resolved_IP_wTF } {
    \BNVS_tl_use:Nv \BNVS_apply_i:nnn { if_resolved_IP_wTF }
  } {
    \use_ii:nn
  }
}
```

`\__bnvs_require:nnnc` `\__bnvs_require:nnnc {⟨id⟩} {⟨path⟩} {⟨key⟩} {⟨ans⟩}`
`\__bnvs_require:nnc` `\__bnvs_require:nnc {⟨id⟩} {⟨path⟩} {⟨ans⟩}`
`\__bnvs_require:vvc`

Calls `\__bnvs_if_get:nnncTF`, does nothing on success and, on failure, does nothing or raise when in debug mode. `\__bnvs_require:nnc` is a shortcut for the more qualified function `\__bnvs_require:nnnc` when ⟨*key*⟩ and ⟨*ans*⟩ happen to be identical.

```
\BNVS_new:cpn { require:nnnc } #1 #2 #3 #4 {
  \__bnvs_if_get:nnncF { #1 } { #2 } { #3 } { #4 } {
    \BNVS_fatal_unreachable:
  }
}
```

`\__bnvs_gunset:nn` `\__bnvs_gunset:nn {⟨id⟩} {⟨path⟩}`
`\__bnvs_gunset:(nv|vn|vv)`

Removes the specifications for the ⟨*id*⟩ `!` ⟨*path*⟩ and all possible keys combination.

```
\BNVS_new:cpn { gunset:NNNnn } #1 #2 #3 #4 #5 {
  \seq_map_inline:Nn #1 {
    \__bnvs_gunset:nnn { #4 } { #5 } { ##1 }
  }
  \cs_undefine:N #1
  \__bnvs_seq_gremove_once:Nn #2 { #5 }
  \seq_if_empty:NT #2 {
    \cs_undefine:N #2
    \__bnvs_seq_gremove_once:Nn #3 { #4 }
    \cs_undefine:c { __bnvs_gset@ #4 :nnn }
  }
}
\BNVS_new:cpn { gunset:Nnn } #1 #2 #3 {
  \seq_map_inline:Nn #1 {
    \tl_if_in:nnT { \q_bnvs ##1 . } { \q_bnvs #3 . } {
      \exp_args:Nc \__bnvs_gunset:NNNnn
        { \__bnvs_g_IP:w  { #2 } { ##1 } }
        #1 \g__bnvs_I_seq { #2 } { ##1 }
      \cs_undefine:c { __bnvs_gset@{ #2 } { ##1 }:nn }
    }
  }
}
\BNVS_new:cpn { gunset:nn } #1 #2 {
  \exp_args:Nc
  \__bnvs_gunset:Nnn { \__bnvs_g_I:w { #1 } } { #1 } { #2 }
}

\BNVS_new:cpn { gunset:nv } #1 {
  \BNVS_tl_use:nv { \__bnvs_gunset:nn { #1 } }
}

\BNVS_new:cpn { gunset:vn } {
  \BNVS_tl_use:cv { gunset:nn }
}

\BNVS_new:cpn { gunset:vv } {
  \BNVS_tl_use:Nvv \__bnvs_gunset:nn
}
```

__bnvs_gunset_deep:nn
__bnvs_gunset_deep:(nv|vv)

__bnvs_gunset_deep:nn {⟨*id*⟩} {⟨*path*⟩}

Removes the specifications for each existing ⟨*id*⟩ ! ⟨*path*⟩ . ⟨*subpath*⟩ combination.

```
\BNVS_new:cpn { gunset_deep:Nnn } #1 #2 #3 {
  \seq_if_exist:NT #1 {
    \seq_map_inline:Nn #1 {
```

`#3` is exactly `##1` or start with `##1.`, which is equivalent to `#3.` starts with `##1.`:

```
      \tl_if_in:nnT { \q_bnvs ##1 . } { \q_bnvs #3 . } {
        \__bnvs_gunset:nn { #2 } { ##1 }
      }
    }
  }
}
```

```
\BNVS_new:cpn { gunset_deep:nn } #1 #2 {
  \exp_args:Nc \__bnvs_gunset_deep:Nnn
    { \__bnvs_g_I:w { #1 } }
    { #1 } { #2 }
}

\BNVS_new:cpn { gunset_deep:nv } #1 {
  \BNVS_tl_use:nv { \__bnvs_gunset_deep:nn { #1 } }
}

\BNVS_new:cpn { gunset_deep:vv } {
  \BNVS_tl_use:Nvv \__bnvs_gunset_deep:nn
}
```

__bnvs_gunset:n __bnvs_gunset:n {⟨id⟩}
__bnvs_gunset: __bnvs_gunset:

Removes the specifications for the ⟨id⟩ and all possible tag combination. The second does the same for all possible ⟨id⟩.

```
\BNVS_new:cpn { gunset:NNNn } #1 #2 #3 #4 {
  \cs_if_exist:NT #1 {
    \cs_undefine:N #1
    \seq_map_inline:Nn #2 {
      \__bnvs_gunset:nn { #4 } { ##1 }
    }
    \cs_undefine:N #2
    \__bnvs_seq_gremove_once:Nn #3 { #4 }
  }
}
\BNVS_new:cpn { gunset:n } #1 {
  \exp_args:Ncc \__bnvs_gunset:NNNn
    { __bnvs_gset@#1:nnn } { \__bnvs_g_I:w { #1 } }
    \g__bnvs_I_seq { #1 }
}

\BNVS_new:cpn { gunset: } {
  \seq_map_inline:Nn \g__bnvs_I_seq {
    \__bnvs_gunset:n { ##1 }
  }
}
```

__bnvs_VS_gunset: __bnvs_VS_gunset:

Removes any value for the "V" and "S" keys and any ⟨id⟩ ! ⟨path⟩ combination.

```
\BNVS_new:cpn { VS_gunset: } {
  \__bnvs_foreach_IP:n {
    \__bnvs_gunset:nnn { ##1 } { ##2 } V
    \__bnvs_gunset:nnn { ##1 } { ##2 } S
  }
}
```

1.7.9 Circular dependencies

__bnvs_will_gset:nnn `\__bnvs_will_gset:nnn {⟨id⟩} {⟨path⟩} {⟨key⟩}`

To manage circular dependencies. After this commmand, `\__bnvs_is_will_gset:nnnTF` is true for the same arguments.

```
\BNVS_new:cpn { will_gset:nnn } #1 #2 #3 {
  \__bnvs_gset:nnnn { #1 } { #2 } { #3 } { \q_no_value }
}
```

__bnvs_is_will_gset:c*TF* `\__bnvs_is_will_gset:cTF {⟨in⟩} {⟨yes:⟩} {⟨no:⟩}`

To manage circular dependencies. See `\__bnvs_will_gset:nnn` above.

```
\__bnvs_will_gset:nnn {⟨id⟩} {⟨path⟩} {⟨key⟩}
...
\__bnvs_if_get:nnncT {⟨id⟩} {⟨path⟩} {⟨key⟩} {⟨in⟩} {
  ...
  \__bnvs_is_will_gset:cTF
    {⟨in⟩}
    {⟨yes:⟩}
    {⟨no:⟩}
  ...
}
```

```
\BNVS_new_conditional:cpnn { is_will_gset:c } #1 { T, F, TF } {
  \BNVS_tl_use:Nv \quark_if_no_value:nTF { #1 } {
    \prg_return_true:
  } {
    \prg_return_false:
  }
}
```

1.7.10 Spring cleaning

__bnvs_gclear:nn `\__bnvs_gclear:nn {⟨id⟩} {⟨path⟩}`
__bnvs_gclear:nv `\__bnvs_gclear:n {⟨id⟩}`
__bnvs_gclear: `\__bnvs_gclear:`
__bnvs_greset:nn `\__bnvs_greset:nn {⟨id⟩} {⟨path⟩}`
__bnvs_greset:nv `\__bnvs_greset:n {⟨id⟩}`
__bnvs_greset: `\__bnvs_greset:`

The first ones clear out every data stored for ⟨*id*⟩ ! ⟨*path*⟩, including deeper and registration data. When ⟨*path*⟩ is omitted, any known path is considered. When ⟨*id*⟩ is omitted, any known id is considered. The second one only clears out the data created on the fly after the initialization step. The initial data (for key =) is left untouched.

```
\BNVS_new:cpn { greset:nn } #1 #2 {
```

```
  \__bnvs_foreach_P:nnT { #1 } {
    \tl_if_in:nnT { \q_bnvs ##2 . } { \q_bnvs #2 . } {
      \__bnvs_foreach_K:nnn { #1 } { ##2 } {
        \tl_if_in:nnF { \q_bnvs ####3 } { \q_bnvs = } {
          \__bnvs_gunset:nnn { #1 } { ##2 } { ####3 }
        }
      }
    }
  } { }
}

\BNVS_new:cpn { greset:nv } #1 {
  \BNVS_tl_use:nv { \__bnvs_greset:nn { #1 } }
}
```

Clears out every data associated to ⟨*id*⟩ ! ⟨*path*⟩, and any ⟨*id*⟩ ! ⟨*path*⟩ . ⟨*subpath*⟩.

```
\BNVS_new:cpn { gclear:nn } #1 #2 {
  \__bnvs_foreach_P:nnT { #1 } {
    \tl_if_in:nnT { \q_bnvs ##2 . } { \q_bnvs #2 . } {
      \__bnvs_foreach_K:nnn { #1 } { ##2 } {
        \tl_if_in:nnTF { \q_bnvs ####3 } { \q_bnvs = } {
          \__bnvs_gset:nnnn  { #1 } { ##2 } { ####3 } { 0 }
        } {
          \__bnvs_gunset:nnn { #1 } { ##2 } { ####3 }
        }
      }
    }
  } { }
}

\BNVS_new:cpn { gclear:n } #1 {
  \__bnvs_foreach_P:nNT { #1 } \__bnvs_gclear:nn {}
}

\BNVS_new:cpn { gclear: } {
  \__bnvs_foreach_IP:N \__bnvs_gclear:nn
}

\BNVS_new:cpn { greset:n } #1 {
  \__bnvs_foreach_P:nNT { #1 } \__bnvs_greset:nn {}
}

\BNVS_new:cpn { greset: } {
  \__bnvs_foreach_IP:N \__bnvs_greset:nn
}
```

__bnvs_gclear_resolved:nn
__bnvs_gclear_resolved:nv

__bnvs_gclear_resolved:nn {⟨*id*⟩} {⟨*path*⟩}

Clear the resolved data, aka any key that does not start with "=".

```
\quark_new:N \q__bnvs_X
```

```
\BNVS_new:cpn { gclear_resolved:nn } #1 #2 {
  \__bnvs_foreach_IP:nnn { #1 } { #2 } {
    \tl_if_in:nnF { \q__bnvs_X ##1 } { \q__bnvs_X = } {
      \__bnvs_gunset:nnn { #1 } { #2 } { ##1 }
    }
  }
}

\BNVS_new:cpn { gclear_resolved:nv } #1 {
  \BNVS_tl_use:nv {
    \__bnvs_gclear_resolved:nn { #1 }
  }
}
```

1.7.11 The provide mode

`\l__bnvs_provide_bool` Manage the provide mode.

```
\bool_new:N \l__bnvs_provide_bool
```

(*End of definition for* `\l__bnvs_provide_bool`.)

```
\BNVS_new:cpn { provide_ON: } {
  \__bnvs_set_true:c { provide }
}

\BNVS_new:cpn { provide_OFF: } {
  \__bnvs_set_false:c { provide }
}
\__bnvs_provide_OFF:
```

`\__bnvs_is_gset_and_provide:nnn`*TF* `\__bnvs_is_gset_and_provide:nnnTF {⟨id⟩} {⟨path⟩} {⟨key⟩} {⟨yes:⟩}`
`\__bnvs_is_gset_and_provide:(nvn|vvn)`*TF* `{⟨no:⟩}`
`\__bnvs_is_gset_and_provide:nn`*TF* `\__bnvs_is_gset_and_provide:nnTF {⟨id⟩} {⟨path⟩} {⟨yes:⟩} {⟨no:⟩}`

Execute ⟨*yes:*⟩ when in provide mode and ⟨*id*⟩ `!` ⟨*path*⟩ `/` ⟨*key*⟩ is known, ⟨*no:*⟩ otherwise. When not specified, ⟨*key*⟩ is "=".

```
\BNVS_new_conditional:cpnn { is_gset_and_provide:nnn }
        #1 #2 #3 { T, F, TF } {
  \__bnvs_if:cTF { provide } {
    \if_cs_exist:w \__bnvs_g_IPK:w { #1 } { #2 } { #3 } \cs_end:
      \prg_return_true:
    \else:
      \prg_return_false:
    \fi:
  } {
    \prg_return_false:
  }
}
```

```
\BNVS_new_conditional:cpnn { is_gset_and_provide:nvn }
                           #1 #2 #3 { T, F, TF } {
  \BNVS_tl_use:nv {
    \__bnvs_is_gset_and_provide:nnnTF { #1 }
  } { #2 } { #3 }
    \prg_return_true: \prg_return_false:
}

\BNVS_new_conditional:cpnn { is_gset_and_provide:vvn }
        #1 #2 #3 { T, F, TF } {
  \BNVS_tl_use:Nvv \__bnvs_is_gset_and_provide:nnnTF { #1 } { #2 } { #3 }
    \prg_return_true: \prg_return_false:
}
```

`\__bnvs_gprovide:TnnnnF` `\__bnvs_gprovide:TvnnnF` `\__bnvs_gprovide:TvvnnF`

`\__bnvs_gprovide:TnnnnF {⟨pre code⟩} {⟨id⟩} {⟨path⟩} {⟨key⟩} {⟨value⟩} {⟨no:⟩}`

Execute ⟨*no:*⟩ exclusively when not in provide mode. Does nothing when something was set for the ⟨*id*⟩ ! ⟨*path*⟩ / ⟨*key*⟩ combination. Execute ⟨*pre code*⟩ before setting.

```
\BNVS_new:cpn { gprovide:TnnnnF } #1 #2 #3 #4 #5 {
  \__bnvs_if:cTF { provide } {
    \__bnvs_is_gset:nnnF { #2 } { #3 } { #4 } {
      #1
      \__bnvs_gset:nnnn { #2 } { #3 } { #4 } { #5 }
    }
  }
}

\BNVS_new:cpn { gprovide:TvnnnF } #1 {
  \BNVS_tl_use:nv { \__bnvs_gprovide:TnnnnF { #1 } }
}
\BNVS_new:cpn { gprovide:TvvnnF } #1 {
  \BNVS_tl_use:nvv { \__bnvs_gprovide:TnnnnF { #1 } }
}
```

`\__bnvs_is_provided:nnn`*TF*

`\__bnvs_is_provided:nnnTF {⟨id⟩} {⟨path⟩} {⟨key⟩} {⟨yes:⟩} {⟨no:⟩}`

If ⟨*id*⟩ ! ⟨*path*⟩ / ⟨*key*⟩ has been successfully resolved and not unset since then, execute ⟨*yes:*⟩. Otherwise execute ⟨*no:*⟩.

```
\tl_new:N \l__bnvs_is_provided_tl
\BNVS_new_conditional:cpnn { is_provided:nnn } #1 #2 #3 { T, F, TF } {
  \__bnvs_if_get:nnncTF { #1 } { #2 } { #3 }  { is_provided } {
    \__bnvs_is_will_gset:cTF { is_provided } {
      \prg_return_false:
    } {
      \prg_return_true:
    }
  } {
    \prg_return_false:
  }
}
```

1.7.12 Initialize

__bnvs_ginit:nnnn `\__bnvs_ginit:nnnn {⟨id⟩} {⟨short⟩} {⟨relative⟩} {⟨definition⟩}`

Called by __bnvs_gdef:nnnn. This function supports provide mode by calling __bnvs_is_gset_and_provide:nnnTF. Somehow a shortcut to __bnvs_gset:nnnn with key "=". ⟨*relative*⟩ is a list of components, if it is not empty, each intermediate path is modified or defined appropriately by calling __bnvs_greset:nn among other things.

```
\BNVS_new:cpn { ginit:nnnn } #1 #2 #3 #4 {
  \tl_if_empty:nTF { #3 } {
    \__bnvs_is_gset_and_provide:nnnF { #1 } { #2 } = {
      \__bnvs_gunset_deep:nn { #1 } { #2 }
      \__bnvs_gset:nnnn { #1 } { #2 } = { #4 }
    }
  } {
    \BNVS_begin:
    \__bnvs_greset:nn { #1 } { #2 }
    \__bnvs_is_gset:nnnF { #1 } { #2 } = {
      \__bnvs_gset:nnnn { #1 } { #2 } = {}
    }
    \__bnvs_tl_set:cn P { #2 }
    \__bnvs_foreach_relative:nn { #3 } {
      \__bnvs_tl_put_right:cn P { . ##1 }
      \__bnvs_greset:nv { #1 } P
      \__bnvs_is_gset:nvnF { #1 } P = {
```

Each intermediate step is just defined empty when not already defined. The last step setting is overriden just after the loop.

```
        \__bnvs_gset:nvnn { #1 } P = {}
      }
    }
    \__bnvs_is_gset_and_provide:nvnF { #1 } P = {
      \__bnvs_gunset_deep:nv { #1 } P
      \__bnvs_gset:nvnn { #1 } P = { #4 }
    }
    \BNVS_end:
  }
}
```

__bnvs_ginit:nnn
__bnvs_ginit:(nnv|vvn)

`\__bnvs_ginit:nnn {⟨id⟩} {⟨path⟩} {⟨simple definition⟩}`

Called by __bnvs_gdef:nnn. Somehow a shortcut to __bnvs_gset:nnnn with key "=". If ⟨*path*⟩ has more than one component, each intermediate path is modified or defined appropriately. This function supports provide mode by calling __bnvs_is_gset_and_provide:nnnTF.

```
\BNVS_new:cpn { ginit:nnn } #1 #2 {
  \__bnvs_ginit:nnnn { #1 } { #2 } {}
}

\BNVS_new:cpn { ginit:nnv } #1 #2 {
  \BNVS_tl_use:nv { \__bnvs_ginit:nnn { #1 } { #2 } }
}
```

```
\BNVS_new:cpn { ginit:vvn } {
  \BNVS_tl_use:Nvv \__bnvs_ginit:nnn
}
```

`\__bnvs_if_should_ginit:nnTF` `\__bnvs_if_should_ginit:nnTF {⟨id⟩} {⟨path⟩} {⟨yes:⟩} {⟨no:⟩}`

Execute ⟨*yes:*⟩ when in provide mode and ⟨*id*⟩ ! ⟨*tag*⟩ combination is not initialized by `\__bnvs_ginit:nnn` . Execute ⟨*no:*⟩ otherwise.

```
\BNVS_new_conditional:cpnn { if_should_ginit:nn } #1 #2 { T, F, TF } {
  \__bnvs_if:cTF { provide } {
      \__bnvs_is_ginit:nnTF { #1 } { #2 } {
      \prg_return_false:
    } {
      \prg_return_true:
    }
  } {
    \prg_return_true:
  }
}
\BNVS_undefine:c { if_should_ginit:nn }
```

`\__bnvs_is_ginit:nnTF` `\__bnvs_is_ginit:nnTF {⟨id⟩} {⟨path⟩} {⟨yes:⟩} {⟨no:⟩}`

Test for the existence of some data for that ⟨*id*⟩ ! ⟨*path*⟩ combination.

```
\BNVS_new_conditional:cpnn { is_ginit:nn } #1 #2 { T, F, TF } {
  \__bnvs_is_gset:nnnTF { #1 } { #2 } = \prg_return_true: \prg_return_false:
}
\BNVS_undefine:c { is_ginit:nn }
```

`\__bnvs_if_ginit:nncTF` `\__bnvs_if_ginit:nncTF {⟨id⟩} {⟨tag⟩} {⟨var⟩} {⟨yes:⟩} {⟨no:⟩}`

Test for the existence of some initialization data for that ⟨*id*⟩ ! ⟨*path*⟩ combination. On success, ⟨*var*⟩ is set to this data.

```
\BNVS_new_conditional:cpnn { if_ginit:nnc } #1 #2 #3 { T, F, TF } {
  \__bnvs_if_get:nnncTF { #1 } { #2 } = { #3 } \prg_return_true: \prg_return_false:
}
\BNVS_undefine:c { if_ginit:nnc }
```

1.8 Regular expressions

`\c__bnvs_A_reserved_Z_regex` Reserved words.

(*End of definition for* `\c__bnvs_A_reserved_Z_regex`.)

```
\regex_const:Nn \c__bnvs_A_reserved_Z_regex {
  \A (?: _+ \d | _* [a-z] ) [_a-z0-9]* \Z
}
```

`\c__bnvs_A_VAZLLZZL_Z_regex` Used to parse named overlay specifications. V, A:Z, A::L on one side, :Z, :Z::L and ::L:Z on the other sides. Next are the capture groups. The first one is for the whole match.

(*End of definition for* `\c__bnvs_A_VAZLLZZL_Z_regex`.)

```
\regex_const:Nn \c__bnvs_A_VAZLLZZL_Z_regex {
  \A \s* (?:
```

- $2 \to \mathrm{V}$

```
    ( [^:]+? )
```

- $3, 4, 5 \to \mathrm{A} : \mathrm{Z}?$ or $\mathrm{A} :: \mathrm{L}?$

```
    | (?: ( [^:]+? ) \s* : (?: \s* ( [^:]*? ) | : \s* ( [^:]*? ) ) )
```

- $6, 7 \to ::(\mathrm{L}:\mathrm{Z})?$

```
    | (?: :: \s* (?: ( [^:]+? ) \s* : \s* ( [^:]+? ) )? )
```

- $8, 9 \to :(\mathrm{Z}::\mathrm{L})?$

```
    | (?: : \s* (?: ( [^:]+? ) \s* :: \s* ( [^:]*? ) )? )
  )
  \s* \Z
}
```

1.9 End group setters

```
\cs_new:Npn \BNVS_did_end_tl_put_right:cv #1 {
  \BNVS_did_end:n {
    \__bnvs_tl_put_right:cn { #1 }
  }
  \BNVS_tl_use:Nv \BNVS_did_end_braced:n
}

\cs_new:Npn \BNVS_end_tl_put_right:cv #1 #2 {
  \BNVS_did_end_tl_put_right:cv { #1 } { #2 }
  \BNVS_end:
}

\cs_new:Npn \BNVS_did_end_tl_gset:nnnv #1 #2 #3 {
  \BNVS_did_end:n {
    \__bnvs_gset:nnnn { #1 } { #2 } { #3 }
  }
  \BNVS_tl_use:Nv \BNVS_did_end_braced:n
}

\cs_new:Npn \BNVS_end_tl_gset:nnnv #1 #2 #3 #4 {
  \BNVS_did_end_tl_gset:nnnv { #1 } { #2 } { #3 } { #4 }
  \BNVS_end:
}

\cs_new:Npn \BNVS_did_end_tl_set:cv #1 {
  \BNVS_did_end:n {
    \__bnvs_tl_set:cn { #1 }
  }
  \BNVS_tl_use:Nv \BNVS_did_end_braced:n
}
```

```
\cs_new:Npn \BNVS_end_tl_set:cv {
  \BNVS_tl_set:ncv { \BNVS_end: }
}

\cs_new:Npn \BNVS_tl_set:ncv #1 #2 {
  \BNVS_tl_use:nv {
    #1
    \__bnvs_tl_set:cn { #2 }
  }
}

\cs_new:Npn \BNVS_tl_set:ncvcv #1 #2 #3 #4 {
  \BNVS_tl_set:ncv {
    \BNVS_tl_set:ncv { #1 } { #2 } { #3 }
  } { #4 }
}

\cs_new:Npn \BNVS_tl_set:ncvcvcv #1 #2 #3 #4 #5 #6 {
  \BNVS_tl_set:ncv {
    \BNVS_tl_set:ncv {
      \BNVS_tl_set:ncv { #1 } { #2 } { #3 }
    } { #4 } { #5 }
  } { #6 }
}

\cs_new:Npn \BNVS_did_end_int_set:cv #1 {
  \BNVS_did_end:n {
    \__bnvs_int_set:cn { #1 }
  }
  \BNVS_tl_use:Nv \BNVS_did_end_braced:n
}

\cs_new:Npn \BNVS_end_int_set:cv #1 #2 {
  \BNVS_did_end_int_set:cv { #1 } { #2 }
  \BNVS_end:
}
```

1.10 **beamer.cls** interface

Work in progress.

```
\RequirePackage{keyval}

\define@key{beamerframe}{beanoves~id}[]{
  \tl_set:Nx \l__bnvs_J_tl { #1 }
}

\bool_new:N \l__bnvs_in_frame_bool
\bool_set_false:N \l__bnvs_in_frame_bool
\AddToHook{env/beamer@frameslide/before}{
  \__bnvs_greset_call:
  \__bnvs_VS_gunset:
  \__bnvs_set_true:c { in_frame }
  \__bnvs_tl_clear:c I
  \__bnvs_tl_clear:c { S() }
}
```

```
\AddToHook{env/beamer@frameslide/after}{
  \__bnvs_set_false:c { in_frame }
}
```

1.11 Utilities

1.11.1 Split utilities

__bnvs_if_regex_split:cncTF
__bnvs_if_regex_split:cnTF

```
\__bnvs_if_regex_split:cncTF {⟨regex core⟩} {⟨expression⟩} ⟨seq core⟩ {⟨yes:⟩}
  {⟨no:⟩}
\__bnvs_if_regex_split:cnTF  {⟨regex core⟩} {⟨expression⟩} {⟨yes:⟩} {⟨no:⟩}
```

These are shortcuts to `\regex_split:NnNTF` with the `split` sequence as last N argument.

```
\BNVS_new_conditional:cpnn { if_regex_split:cnc } #1 #2 #3 { T, F, TF } {
  \BNVS_seq_use:nc {
    \BNVS_regex_use:Nc \regex_split:NnNTF { #1 } { #2 }
  } { #3 } \prg_return_true: \prg_return_false:
}

\BNVS_new_conditional:cpnn { if_regex_split:cn } #1 #2 { T, F, TF } {
  \BNVS_seq_use:nc {
    \BNVS_regex_use:Nc \regex_split:NnNTF { #1 } { #2 }
  } { split } \prg_return_true: \prg_return_false:
}
```

__bnvs_split_pop:
__bnvs_split_pop:c
__bnvs_split_pop:cc
__bnvs_split_pop:ccc
__bnvs_split_pop:cccc
__bnvs_split_pop:ccccc

```
\__bnvs_split_pop:
\__bnvs_split_pop:c     {⟨var_1⟩}
\__bnvs_split_pop:cc    {⟨var_1⟩} {⟨var_2⟩}
\__bnvs_split_pop:ccc   {⟨var_1⟩} {⟨var_2⟩} {⟨var_3⟩}
\__bnvs_split_pop:cccc  {⟨var_1⟩} {⟨var_2⟩} {⟨var_3⟩} {⟨var_4⟩}
\__bnvs_split_pop:ccccc {⟨var_1⟩} {⟨var_2⟩} {⟨var_3⟩} {⟨var_4⟩} {⟨var_5⟩}
```

```
\tl_new:N \l__bnvs_split_pop_tl
\BNVS_new:cpn { split_pop: } {
  \seq_pop_left:NN \l__bnvs_split_seq \l__bnvs_split_pop_tl
}

\BNVS_new:cpn { split_pop:c } #1 {
  \BNVS_tl_use:nc {
    \seq_pop_left:NN \l__bnvs_split_seq
  } { #1 }
}

\BNVS_new:cpn { split_pop:cc } #1 {
  \__bnvs_split_pop:c { #1 }
  \__bnvs_split_pop:c
}

\BNVS_new:cpn { split_pop:ccc } #1 {
  \__bnvs_split_pop:c { #1 }
  \__bnvs_split_pop:cc
}
```

```
\BNVS_new:cpn { split_pop:cccc } #1 {
  \__bnvs_split_pop:c { #1 }
  \__bnvs_split_pop:ccc
}

\BNVS_new:cpn { split_pop:ccccc } #1 {
  \__bnvs_split_pop:c { #1 }
  \__bnvs_split_pop:cccc
}

\BNVS_new_conditional:cpnn { split_if_pop:c } #1 { T, F, TF } {
  \__bnvs_seq_pop_left:ccTF { split } { #1 } {
    \prg_return_true:
  } {
    \prg_return_false:
  }
}
```

1.11.2 Match utilities

__bnvs_match_if_once:NnTF
__bnvs_match_if_once:NvTF
__bnvs_match_if_once:nnTF

__bnvs_match_if_once:NnTF ⟨*regex variable*⟩ {⟨*expression*⟩} {⟨*yes:*⟩} {⟨*no:*⟩}
__bnvs_match_if_once:nnTF {⟨*regex*⟩} {⟨*expression*⟩} {⟨*yes:*⟩} {⟨*no:*⟩}

These are shortcuts to

- `\regex_match_if_once:NnNTF` with the `match` sequence as N argument
- `\regex_match_if_once:nnNTF` with the `match` sequence as N argument
- `\regex_split:NnNTF` with the `split` sequence as last N argument

```
\BNVS_new_conditional:cpnn { if_extract_once:Ncn } #1 #2 #3 { T, F, TF } {
  \BNVS_use:ncn {
    \regex_extract_once:NnNTF #1 { #3 }
  } { #2 } { seq } \prg_return_true: \prg_return_false:
}
```

\l__bnvs_match_seq

Local storage for the match result.

(*End of definition for* `\l__bnvs_match_seq`.)

```
\BNVS_new_conditional:cpnn { match_if_once:Nn } #1 #2 { T, F, TF } {
  \BNVS_use:ncn {
    \regex_extract_once:NnNTF #1 { #2 }
  } { match } { seq } \prg_return_true: \prg_return_false:
}

\BNVS_new_conditional:cpnn { if_extract_once:Ncv } #1 #2 #3 { T, F, TF } {
  \BNVS_seq_use:nc {
    \BNVS_tl_use:nv {
      \regex_extract_once:NnNTF #1
    } { #3 }
  } { #2 } \prg_return_true: \prg_return_false:
}
```

```
\BNVS_new_conditional:cpnn { match_if_once:Nv } #1 #2 { T, F, TF } {
  \BNVS_seq_use:nc {
    \BNVS_tl_use:nv {
      \regex_extract_once:NnNTF #1
    } { #2 }
  } { match } \prg_return_true: \prg_return_false:
}

\BNVS_new_conditional:cpnn { match_if_once:nn } #1 #2 { T, F, TF } {
  \BNVS_seq_use:nc {
    \regex_extract_once:nnNTF { #1 } { #2 }
  } { match } \prg_return_true: \prg_return_false:
}

\BNVS_new_conditional:cpnn { match_if_pop_left:c } #1 { T, F, TF } {
  \BNVS_tl_use:nc {
    \BNVS_seq_use:Nc \seq_pop_left:NNTF { match }
  } { #1 } {
    \prg_return_true:
  } {
    \prg_return_false:
  }
}
```

`\__bnvs_match_pop:`	`\__bnvs_match_pop:`
`\__bnvs_match_pop:c`	`\__bnvs_match_pop:c` {⟨var$_1$⟩}
`\__bnvs_match_pop:cc`	`\__bnvs_match_pop:cc` {⟨var$_1$⟩} {⟨var$_2$⟩}
`\__bnvs_match_pop:ccc`	`\__bnvs_match_pop:ccc` {⟨var$_1$⟩} {⟨var$_2$⟩} {⟨var$_3$⟩}
`\__bnvs_match_pop:cccc`	`\__bnvs_match_pop:cccc` {⟨var$_1$⟩} {⟨var$_2$⟩} {⟨var$_3$⟩} {⟨var$_4$⟩}
`\__bnvs_match_pop:ccccc`	`\__bnvs_match_pop:ccccc` {⟨var$_1$⟩} {⟨var$_2$⟩} {⟨var$_3$⟩} {⟨var$_4$⟩} {⟨var$_5$⟩}

```
\tl_new:N \l__bnvs_match_pop_tl
\BNVS_new:cpn { match_pop: } {
  \seq_pop_left:NN \l__bnvs_match_seq \l__bnvs_match_pop_tl
}

\BNVS_new:cpn { match_pop:c } #1 {
  \BNVS_tl_use:nc {
    \seq_pop_left:NN \l__bnvs_match_seq
  } { #1 }
}

\BNVS_new:cpn { match_pop:cc } #1 {
  \__bnvs_match_pop:c { #1 }
  \__bnvs_match_pop:c
}

\BNVS_new:cpn { match_pop:ccc } #1 {
  \__bnvs_match_pop:c { #1 }
  \__bnvs_match_pop:cc
}
```

```
\BNVS_new:cpn { match_pop:cccc } #1 {
  \__bnvs_match_pop:c { #1 }
  \__bnvs_match_pop:ccc
}

\BNVS_new:cpn { match_pop:ccccc } #1 {
  \__bnvs_match_pop:c { #1 }
  \__bnvs_match_pop:cccc
}
```

1.12 Main scanner

There are different policies to store the intermediate results of a scanning operation. One possibility is to feed a global variable as scanning occurs. As you cannot use unbalanced braces in function arguments, we can use other characters than usual "\", "{" and "}" before we rescan the final result with an appropriate catcode regime. Another possibility is to heavily use TEX groups to store intermediate results in local variables and only consider properly balanced material. At the end of the group, the scanned material must be forwarded to the group owner. The latter is used here.

The scanning process reads ⟨*content*⟩ from the ⟨*input stream*⟩ and store a corresponding *abstract syntax tree* (AST) denoted by ⟨*content*⟩*. The AST will be executed either to define named overlay sets or to resolve named overlay queries. The workflow consists of

1. begin with `\BNVS_begin_scan:`
2. define the AST exportation policy
3. define the scanning policy
4. define the branch functions named `\BNVS_scanned_`⟨*branch*⟩, where ⟨*branch*⟩ is one of `Block:w`, `One_colon:w`, `Colons:w`, `comMa:w`, `L:w`, `Stop:`,
5. call `\BNVS_scaN:w` or some `\BNVS_scaN_...:...w` function to peek characters from the ⟨*input stream*⟩ and store corresponding AST
6. end with `\BNVS_end_scan:`, which exports the AST created according to the exportation policy.

We use the same scanner to parse definitions and queries. We do not use any key–value facility built into LATEX.

1.12.1 Base grammar

Hereafter is a pragmatic grammar used for parsing, not all the expressions covered do have a real meaning. The characters all have category other. We start with basic entities.

```
⟨spc⟩          ::= Horizontal white space characters
⟨lower⟩        ::= a lowercase letter
⟨upper⟩        ::= an uppercase letter
⟨digit⟩        ::= a decimal digit
⟨sign⟩         ::= '-' | '+'
⟨comma⟩        ::= ⟨spc⟩ ',' ⟨spc⟩
⟨alpha⟩        ::= ⟨lower⟩ | ⟨upper⟩ | '_'
⟨alnum⟩        ::= ⟨alpha⟩ | ⟨digit⟩
⟨index⟩        ::= ⟨sign⟩* ⟨digit⟩+
⟨number⟩       ::= ⟨sign⟩? ⟨significand⟩ ⟨exponent⟩?
⟨significand⟩  ::= ( '.'? ⟨digit⟩+ ) | ( ⟨digit⟩+ '.' ⟨digit⟩* )
⟨exponent⟩     ::= ( 'e' | 'E' ) ⟨sign⟩? ⟨digit⟩+
⟨other⟩        ::= ⟨number⟩ | '+' | '-' | '*' | '/'
```

`\c__bnvs_N_regex` Signed integer.

(*End of definition for* `\c__bnvs_N_regex`.)

```
\regex_const:Nn \c__bnvs_N_regex {
  [-+]* \d+
}
```

`\c__bnvs_dot_N_regex` Signed integer with a dot `.` just before.

(*End of definition for* `\c__bnvs_dot_N_regex`.)

```
\regex_const:Nn \c__bnvs_dot_N_regex {
  \. \ur{c__bnvs_N_regex}
}
```

`\c__bnvs_A_dot_N_Z_regex` Full signed integer with a dot `.` jus before.

(*End of definition for* `\c__bnvs_A_dot_N_Z_regex`.)

```
\regex_const:Nn \c__bnvs_A_dot_N_Z_regex {
  \A \ur { c__bnvs_dot_N_regex } \Z
}
```

`\c__bnvs_A_i_Z_regex` Signed integer with no capture groups.

(*End of definition for* `\c__bnvs_A_i_Z_regex`.)

```
\regex_const:Nn \c__bnvs_A_i_Z_regex { \A \ur{c__bnvs_N_regex} \Z }
```

`\c__bnvs_dot_R_regex` A possibly empty list of `.`⟨*short name*⟩ items representing a path. Integers are allowed as well.

```
\regex_const:Nn \c__bnvs_R_regex {
  \ur{c__bnvs_N_regex} | [[:alnum:]_]+
}
```

(*End of definition for* `\c__bnvs_dot_R_regex`.)

`\c__bnvs_dot_R_regex` A possibly empty list of `.`⟨*short name*⟩ items representing a path. Integers are allowed as well.

```
\regex_const:Nn \c__bnvs_dot_R_regex {
  \. \ur{c__bnvs_R_regex}
}
```

(*End of definition for* `\c__bnvs_dot_R_regex`.)

`\c__bnvs_S_regex` This regular expression is used for short names. The short name of an overlay set consists of a non void list of alphanumerical characters and underscore, but with no leading digit.

```
\regex_const:Nn \c__bnvs_S_regex {
  [[:alpha:]_][[:alnum:]_]*
}
```

(*End of definition for* `\c__bnvs_S_regex`.)

`\c__bnvs_P_regex` This regular expression is used for short alphanumerical path components with at least one letter.

```
\regex_const:Nn \c__bnvs_P_regex {
  \d*[[:alpha:]_][[:alnum:]_]*
}
```

(*End of definition for* `\c__bnvs_P_regex`.)

`\c__bnvs_dot_P_regex` This regular expression is used for short alphanumerical path components with at least one letter.

```
\regex_const:Nn \c__bnvs_dot_P_regex {
  \. \ur{c__bnvs_P_regex}
}
```

(*End of definition for* `\c__bnvs_dot_P_regex`.)

1.12.2 Storage

`\BNVS_xprt_ast:n` `\BNVS_xprt_ast:n {`⟨*content*⟩`*}`
`\BNVS_xprt_clear:` `\BNVS_xprt_clear:`

`\BNVS_xprt_ast:n` is a utility function to store its argument in the `xprted` variable. `\BNVS_xprt_clear:` resets the scan storage, called when starting a new TEX group dedicated to scanning.

```
\cs_new:Npn \BNVS_xprt_clear: {
  \__bnvs_tl_clear:c { xprted }
}
\cs_new:Npn \BNVS_xprt_ast:n #1 {
  \BNVS_xprt_ast_Raw:
  \__bnvs_tl_put_right:cn { xprted } { #1 }
}
```

There is a special situation for raw material that is stored separately and exported at once before other material is exported or when a scan group ends.

\BNVS_xprt_Raw:n
\BNVS_xprt_ast_Raw:

\BNVS_xprt_Raw:n {⟨*something*⟩}
\BNVS_xprt_ast_Raw:

\BNVS_xprt_Raw:n synchronously cumulates its argument in the xprted_Raw variable. The function \BNVS_xprt_ast_Raw: is executed both at the beginning of the function \BNVS_xprt_ast:n and when a scan group ends. It exports the contents of the stored ⟨*material*⟩, when non empty, to the xprted variable as \:n{⟨*material*⟩} instruction and cleans the xprted_Raw variable.

```
\cs_new:Npn \BNVS_xprt_Raw:n #1 {
  \tl_if_empty:nF { #1 } {
    \__bnvs_tl_put_right:cn { xprted_Raw } { #1 }
  }
}

\cs_new:Npn \BNVS_xprt_ast_Raw: {
  \__bnvs_tl_if_empty:cF { xprted_Raw } {
    \BNVS_tl_use:nv {
      \__bnvs_tl_clear:c { xprted_Raw }
      \BNVS_xprt_ast:n { \:n }
      \BNVS_xprt_grp:n
    } { xprted_Raw }
  }
}
```

\BNVS_xprt_grp:n

\BNVS_xprt_grp:n {⟨*scanned*⟩}

Feeds the scan storage with ⟨*scanned*⟩, enclosed betwenn one pair of curly braces.

```
\cs_new:Npn \BNVS_xprt_grp:n #1 {
  \BNVS_xprt_ast:n { { #1 } }
}
```

\BNVS_xprted_brace:

\BNVS_xprted_brace:

Wraps what was exported so far between a pair of braces.

```
\cs_new:Npn \BNVS_xprted_brace: {
  \exp_args:NNx \BNVS_xprt_clear: \BNVS_xprt_ast:n {
     { \BNVS_xprted_use:n { \exp_not:n } }
  }
}
```

\BNVS_xprt_space:
\BNVS_xprt_space_set:n

\BNVS_xprt_space:
\BNVS_xprt_space_set:n {⟨*code:*⟩}

\BNVS_xprt_space: exports the space character. \BNVS_xprt_space_set:n sets the meaning of \BNVS_xprt_space: to ⟨*code:*⟩.

```
\cs_new:Npn \BNVS_xprt_space_set:n #1 {
  \cs_set:Npn \BNVS_xprt_space: {
    #1
  }
}

\BNVS_xprt_space_set:n {\BNVS_xprt_Raw:n { ~ } }
```

\BNVS_xprted_use:n `\BNVS_xprted_use:n {⟨code:n⟩}`

Puts back to the input stream the instructions ⟨*code:n*⟩ followed by the exported material so far, enclosed between one pair of braces.

```
\cs_new:Npn \BNVS_xprted_use:n #1 {
  \BNVS_xprt_ast_Raw:
  \BNVS_tl_use:nv { #1 } { xprted }
}
```

\BNVS_if_xprted_empty:TF `\BNVS_if_xprted_empty:TF {⟨yes:n⟩} {⟨no:n⟩}`

Execute ⟨*yes:n*⟩ instructions if something has been exported, ⟨*no:n*⟩ instructions otherwise.

```
\cs_new:Npn \BNVS_if_xprted_empty:TF {
  \BNVS_xprted_use:n { \tl_if_empty:nTF }
}
```

\BNVS_next_set:n `\BNVS_next_set:n {⟨next:⟩}`
\BNVS_next_use:n `\BNVS_next_use:n {⟨code:⟩}`

\BNVS_next_set:n stores ⟨*next:*⟩ to be executed later by \BNVS_next_use:n, just after ⟨*code:*⟩. Typically, \BNVS_next_set:n is used in a scan group closed by ⟨*code:*⟩. \BNVS_end_scan_next: ends the scan and executes ⟨*next:*⟩ code.

```
\BNVS_tl_new:c { next }
\cs_new:Npn \BNVS_next_set:n #1 {
  \__bnvs_tl_set:cn { next } {
    \exp_not:n {
      #1
    }
  }
}

\cs_new:Npn \BNVS_next_use:n #1 {
  \BNVS_tl_use_unbraced:nv { #1 } { next }
}
```

1.12.3 AST exportation policy

This is used by \BNVS_end_scan: to wrap the AST created during the scanning process before it is exported. The case of an empty created AST is sometimes special.

\BNVS_ast_pfX_set:n
\BNVS_ast_pfX_will_set:n

\BNVS_ast_pfX_set:n {⟨pfX:w⟩}
\BNVS_ast_pfX_will_set:n {⟨pfX:w⟩}

Set the function \BNVS_ast_pfX:w to expand to \exp_not:n { ⟨pfX:w⟩}. This is used to define \BNVS_ast_node_make:n, when the arguments must be scanned prior to knowing which function ⟨pfX:w⟩ fits. The "will" variant defers the execution just before \BNVS_end:.

```
\cs_new:Npn \BNVS_ast_pfX:w {}

\cs_new:Npn \BNVS_ast_pfX_set:n #1 {
  \cs_set:Npn \BNVS_ast_pfX:w { \exp_not:n { #1 } }
}

\cs_new:Npn \BNVS_ast_pfX_will_set:n #1 {
  \BNVS_will_end:n {
    \BNVS_ast_pfX_set:n { #1 }
  }
}
```

\BNVS_ast_plc_init:n
\BNVS_ast_plc_init:X
\BNVS_ast_plc_init:X:n
\BNVS_ast_plc_init_pfx:n
\BNVS_ast_plc_init_mpt:n
\BNVS_ast_plc_init_mpt:X
\BNVS_ast_plc_init_mpt:X:n
\BNVS_ast_plc_init_grp:n
\BNVS_ast_plc_init:XXn
\BNVS_ast_plc_brc:
\BNVS_ast_plc_Raw:
\BNVS_ast_plc_none:

\BNVS_ast_plc_init:n {⟨template:n⟩}
\BNVS_ast_plc_init:X {⟨pfX⟩}
\BNVS_ast_plc_init:X:n {⟨pfX⟩} {⟨template:n⟩}
\BNVS_ast_plc_init_pfx:n {⟨template:n⟩}
\BNVS_ast_plc_init_mpt:n {⟨template:n⟩}
\BNVS_ast_plc_init_mpt:X {⟨pfX⟩}
\BNVS_ast_plc_init_mpt:X:n {⟨pfX⟩} {⟨template:n⟩}
\BNVS_ast_plc_init_grp:n {⟨template:n⟩}
\BNVS_ast_plc_Raw:
\BNVS_ast_plc_none:

Intitialize the AST exportation policy through the meaning of the function \BNVS_ast_node_make:n. The ⟨template:n⟩ is by default ##1. With the first variant, nothing is exported if the input AST is blank, this is the default exportation policy. With the pfx variant, only the ⟨pfX:w⟩ is exported when the input AST is blank. The mpt and grp variants never export an empty AST. The first one exports the input AST as is whereas the second one exports the input AST between a pair of braces. \BNVS_ast_plc_Raw: exports as is, with no brace nor prefix. \BNVS_ast_plc_ignore: exports nothing at all. The ...:X variant is equivalent to the ...:X:n variant with {##1} as last argument.

```
\cs_new:Npn \BNVS_ast_plc_init:n #1 {
  \cs_set:Npn \BNVS_ast_node_make:n ##1 {
    \tl_if_blank:nF { ##1 } {
      \BNVS_ast_pfX:w
```

and \exp_not:n{⟨xprt:n⟩}.

```
      \exp_not:n { #1 }
    }
  }
```

```
}
\BNVS_ast_plc_init:n { #1 }

\cs_new:Npn \BNVS_ast_plc_init:X #1 {
  \BNVS_ast_pfX_set:n { #1 }
  \BNVS_ast_plc_init:n { { ##1 } }
}

\cs_new:Npn \BNVS_ast_plc_init:X:n #1 #2 {
  \BNVS_ast_pfX_set:n { #1 }
  \BNVS_ast_plc_init:n { #2 }
}

\cs_new:Npn \BNVS_ast_plc_init_pfx:n #1 {
  \cs_set:Npn \BNVS_ast_node_make:n ##1 {
    \BNVS_ast_pfX:w
    \tl_if_blank:nF { ##1 } {
```

and \exp_not:n{⟨xprt:n⟩}.

```
      \exp_not:n { #1 }
    }
  }
}

\cs_new:Npn \BNVS_ast_plc_init_mpt:n #1 {
s
  \cs_set:Npn \BNVS_ast_node_make:n ##1 {
    \BNVS_ast_pfX:w
```

and \exp_not:n{⟨xprt:n⟩}.

```
    \exp_not:n { #1 }
  }
}

\cs_new:Npn \BNVS_ast_plc_init_mpt:X #1 {
  \BNVS_ast_pfX_set:n { #1 }
  \BNVS_ast_plc_init_mpt:n { { ##1 } }
}

\cs_new:Npn \BNVS_ast_plc_init_mpt:X:n #1 #2 {
  \BNVS_ast_pfX_set:n { #1 }
  \BNVS_ast_plc_init_mpt:n { #2 }
}

\cs_new:Npn \BNVS_ast_plc_init_grp:n #1 {
s
  \cs_set:Npn \BNVS_ast_node_make:n ##1 {
    \BNVS_ast_pfX:w
```

and \exp_not:n{⟨xprt:n⟩}.

```
    \exp_not:n { { #1 } }
  }
}

\cs_new:Npn \BNVS_ast_plc_init:XX:n #1 #2 #3 {
```

```
  \cs_set:Npn \BNVS_ast_node_make:n ##1 {
    \tl_if_blank:nTF { ##1 } {
      \exp_not:n { #1 }
    } {
      \exp_not:n { #2 }
```

and \exp_not:n{⟨xprt:n⟩}.

```
      \exp_not:n { #3 }
    }
  }
}

\cs_new:Npn \BNVS_ast_plc_brc: {
  \cs_set:Npn \BNVS_ast_node_make:n ##1 {
    \exp_not:n { { ##1 } }
  }
}

\cs_new:Npn \BNVS_ast_plc_Raw: {
s
  \cs_set:Npn \BNVS_ast_node_make:n ##1 { \exp_not:n { ##1 } }
}

\cs_new:Npn \BNVS_ast_plc_none: {
s
  \cs_set:Npn \BNVS_ast_node_make:n ##1 { }
}
```

1.12.4 Branch functions

The scanning process only applies to characters of category codes space and other. It definitely stops at the first token that is not of that category. It takes one of 6 branches depending on what is scanned:

- ⟨*Block:w*⟩ for unit blocks like names,
- ⟨*One_colon:w*⟩ after a single colon,
- ⟨*Colons:w*⟩ after multiple colons,
- ⟨*comMa:w*⟩ after a comma,
- ⟨*L:w*⟩ after a closing delimiter,
- ⟨*S:*⟩ when scanning is done.

Block branch The Block branch is used when the last character scanned from the ⟨*input stream*⟩ fits the current requirement as defined by the last scan function called, but the next one does not.

\BNVS_scaN_Block:Fw
\BNVS_scanned_B:w
\BNVS_scanned_set:B
\BNVS_scaN_Block:B:Fw
\BNVS_scaN_Block_def::BF

\BNVS_scaN_Block:Fw {⟨*else:w*⟩} ⟨*input stream*⟩
\BNVS_scanned_B:w ⟨*input stream*⟩
\BNVS_scanned_set:B {⟨*on_B:w*⟩}
\BNVS_scaN_Block:B:Fw {⟨*scaN:B:w*⟩} {⟨*else:w*⟩} ⟨*input stream*⟩
\BNVS_scaN_Block_def::BF {⟨*block:B:Fw*⟩}

The \BNVS_scanned_B:w branch function is called by \BNVS_scaN_Block:Fw once a ⟨*block*⟩ has been scanned from the head of the ⟨*input stream*⟩ (greedy match). The function \BNVS_scanned_set:B sets the meaning of \BNVS_scanned_B:w to ⟨*Block:w*⟩. The exact definition of a block is driven by \BNVS_scaN_Block:B:Fw which meaning is set by \BNVS_scaN_Block_def::BF.

```
\cs_new:Npn \BNVS_scaN_Block:Fw #1 {
  \BNVS_scaN_Block:B:Fw {
    \BNVS_scanned_B:w
  } {
    #1
  }
```

There must be absolutely no code here. No hooking allowed either.

```
}

\cs_new:Npn \BNVS_scaN_Block_def::BF #1 {
  \cs_set:Npn \BNVS_scaN_Block:B:Fw ##1 ##2 {
    #1
  }
}

\cs_new:Npn \BNVS_scanned_B_wrap_do:n #1 {
  \BNVS_scanned_set:B {
    \BNVS_scanned_B_wrapper:B { #1 }
  }
}

\BNVS_scaN_Block_def::BF {
```

At the top level, the ⟨*else:w*⟩ instructions are always executed.

```
  #2
}
```

Colon branch

\c__bnvs_Colons_regex For ranges defined by a colon syntax. One catching group for more than one colon.

```
\regex_const:Nn \c__bnvs_Colons_regex { \s* :(:*) \s* }
```

(*End of definition for* `\c__bnvs_Colons_regex`.)

\BNVS_scaN_colon:Fw `\BNVS_scaN_colon:Fw {⟨else:w⟩} ⟨input stream⟩`
\BNVS_scanned_0:w `\BNVS_scanned_0:w ⟨input stream⟩`
\BNVS_scanned_set:0 `\BNVS_scanned_set:0 {⟨on_0:w⟩}`
\BNVS_scanned_C:w `\BNVS_scanned_C:w ⟨input stream⟩`
\BNVS_scanned_set:C `\BNVS_scanned_set:C {⟨on_C:w⟩}`

On scanning a standalone colon from the ⟨*input stream*⟩, calls the branch function `\BNVS_scanned_0:w`. On scanning multiple colons from the ⟨*input stream*⟩, calls the `\BNVS_scanned_C:w` branch function. In other cases, execute the ⟨*else:w*⟩ instructions.

One shot utility function only used by `\BNVS_scaN_colon:Fw`

```
\cs_new:Npn \BNVS_scanned_C_regex:n #1 {
  \tl_if_empty:nTF { #1 } {
    \BNVS_scanned_0:w
  } {
    \BNVS_scanned_C:w
  }
}

\cs_new:Npn \BNVS_scaN_colon:Fw #1 {
  \peek_regex_replace_once:NnTF \c__bnvs_Colons_regex { \cB\{ \1 \cE\} } {
    \BNVS_scanned_C_regex:n
  } {
    #1
  }
}
```

Comma branch

\c__bnvs_M_regex Comma as a separator.

```
\regex_const:Nn \c__bnvs_M_regex { \s* , \s* }
```

(*End of definition for* `\c__bnvs_M_regex`.)

`\peek_regex_remove_once:NTF` is buggy in L3 programming layer <2025-01-18>.

```
\cs_new:Npn \BNVS_peek_regex_remove_once:NTF #1 {
  \peek_regex_replace_once:NnTF #1 { }
}
```

\BNVS_scaN_M:Fw `\BNVS_scaN_M:Fw {⟨else:w⟩} ⟨input stream⟩`
\BNVS_scanned_M:w `\BNVS_scaN_M:w ⟨input stream⟩`
\BNVS_scanned_set:M `\BNVS_scaN_M_set:n {⟨on_M:w⟩}`

Once a comma has just been scanned from the head of the ⟨*input stream*⟩, calls `\BNVS_scanned_M:w` branch function, otherwise execute ⟨*else:w*⟩ instructions. `\BNVS_scaN_M_set:n` sets the meaning of `\BNVS_scaN_M:w` to ⟨*comMa:w*⟩.

```
\cs_new:Npn \BNVS_scaN_M:Fw #1 {
```

\peek_regex_remove:NnTF is buggy in L3 programming layer <2025-01-18>.

```
  \BNVS_peek_regex_remove_once:NTF \c__bnvs_M_regex {
    \BNVS_scanned_M:w
  } {
    #1
  }
```

There must be absolutely no code here. No hooking allowed either.

```
}
```

Close branch

\c__bnvs_cLose_regex Closing delimiters.

```
\regex_const:Nn \c__bnvs_cLose_regex { \s* (%([{
  \}|\]|\)
  ) \s*
}
```

(*End of definition for* \c__bnvs_cLose_regex.)

\BNVS_scaN_L:Fw \BNVS_scaN_L:Fw {⟨*else:w*⟩} ⟨*input stream*⟩
\BNVS_scanned_L:nw \BNVS_scanned_L:nw ⟨*clositer*⟩ ⟨*input stream*⟩
\BNVS_scanned_L:w \BNVS_scanned_L:w ⟨*input stream*⟩
\BNVS_scanned_L_set: \BNVS_scanned_set:L ⟨*on_L:w*⟩

When the \BNVS_scaN_L:Fw function scans one of '`)`', '`]`' or '`}`' closing delimiter, it calls the branch function \BNVS_scanned_L:nw with that delimiter as argument, which in turn calls \BNVS_scanned_L:w, after it manages eventual errors in balancing delimiters. Otherwise execute ⟨*else:w*⟩ code.

```
\cs_new:Npn \BNVS_scaN_L:Fw #1 {
  \peek_regex_replace_once:NnTF \c__bnvs_cLose_regex { \cB\{ \1 \cE\} } {
    \BNVS_scanned_L:nw
  } {
    #1
  }
}

\cs_new:Npn \BNVS_scanned_L:nw #1 {
  \BNVS_error:n { Unexpected~delimiter~`#1' }
  \BNVS_scanned_L:w
}

\tl_map_inline:nn {{Block}{One_colon}{Colons}{comMa}{cLose}} {
  \cs_new:cpn { BNVS_scanned_#1_set:n } ##1 {
    \cs_set:cpn { BNVS_scanned_#1:w } {
      ##1
    }
  }
}
```

Do nothing operation at the top level.

```
  \cs_new:cpn { BNVS_scanned_#1:w } {
  }
}
```

```
\tl_map_inline:nn {BOCML} {
  \cs_new:cpn { BNVS_scanned_set:#1 } ##1 {
    \cs_set:cpn { BNVS_scanned_#1:w } {
      ##1
    }
  }
```

Do nothing operation at the top level.

```
  \cs_new:cpn { BNVS_scanned_#1:w } {
  }
}
```

Stop branch The Stop branch is executed in the Stop situation, *id est* when the head of the ⟨*input stream*⟩ cannot be scanned. In practice, when the head of the ⟨*input stream*⟩ is not a character of category code space or other.

`\BNVS_scaN_S:F` `\BNVS_scaN_S:F {⟨else:n⟩} ⟨input stream⟩`
`\BNVS_scanned_S:` `\BNVS_scanned_S:`
`\BNVS_scanned_set:S` `\BNVS_scanned_set:S {⟨on_S:⟩}`

When the head of the ⟨*input stream*⟩ is not a space nor a character of category code other, `\BNVS_scaN_S:F` calls the `\BNVS_scanned_S:` branch function, otherwise execute ⟨*else:n*⟩ instructions. The function `\BNVS_scanned_set:S` sets the meaning of `\BNVS_scanned_S:` to ⟨*S:*⟩.

```
\cs_new:Npn \BNVS_scanned_set:S #1 {
  \cs_set:Npn \BNVS_scanned_S: {
    #1
  }
}
```

Do nothing operation at the top level.

```
\cs_new:Npn \BNVS_scanned_S: {
}

\cs_new:Npn \BNVS_scaN_S:F #1 {
  \peek_catcode:NTF \c_catcode_other_token { #1 } {
    \peek_catcode_remove:NTF \c_space_token {
      \BNVS_xprt_space:
      #1
    } {
      \BNVS_scanned_S:
    }
  }
```

There must be absolutely no code here. No hooking allowed either.

```
}
```

\BNVS_scanned_set:BOCMLS
\BNVS_scanned_set:OCMLS
\BNVS_scanned_set:OC
\BNVS_scanned_set:LS
\BNVS_scanned_set:MLS

Branch setter

```
\BNVS_scanned_set:BOCMLS {⟨on_B:w⟩} {⟨on_O:w⟩} {⟨on_C:w⟩} {⟨on_M:w⟩} {⟨on_L:w⟩}
  {⟨on_S:⟩}
\BNVS_scanned_set:OCMLS {⟨on_O:w⟩} {⟨on_C:w⟩} {⟨on_M:w⟩} {⟨on_L:w⟩} {⟨on_S:⟩}
\BNVS_scanned_set:OC {⟨on_O:w⟩} {⟨on_C:w⟩}
\BNVS_scanned_set:LS {⟨on_L:w⟩} {⟨on_S:⟩}
\BNVS_scanned_set:MLS {⟨on_M:w⟩} {⟨on_L:w⟩} {⟨on_S:⟩}
```

Combination of branch setters. Better for reading experience.

```
\cs_new:Npn \BNVS_scanned_set:BOCMLS #1 {
  \BNVS_scanned_set:B { #1 }
  \BNVS_scanned_set:OCMLS
}

\cs_new:Npn \BNVS_scanned_set:OC #1 #2 {
  \BNVS_scanned_set:O { #1 }
  \BNVS_scanned_set:C { #2 }
}

\cs_new:Npn \BNVS_scanned_set:OCMLS #1 #2 {
  \BNVS_scanned_set:OC { #1 } { #2 }
  \BNVS_scanned_set:MLS
}

\cs_new:Npn \BNVS_scanned_set:LS #1 #2 {
  \BNVS_scanned_set:L { #1 }
  \BNVS_scanned_set:S { #2 }
}

\cs_new:Npn \BNVS_scanned_set:MLS #1 {
  \BNVS_scanned_set:M { #1 }
  \BNVS_scanned_set:LS
}
```

1.12.5 Begin and end of scanning processes

\BNVS_begin_scan:

```
\BNVS_begin_scan:
```

Start a scanning group. *Implementation detail:* the function is split into 3 parts for debugging and testing purposes.

```
\cs_new:Npn \BNVS_begin_scan: {
  \BNVS_scaN_will_begin:
  \BNVS_begin:
  \BNVS_scaN_did_begin:
}

\cs_new:Npn \BNVS_scaN_will_begin: {
  \BNVS_xprt_ast_Raw:
}
```

```
\cs_new:Npn \BNVS_scaN_did_begin: {
  \BNVS_xprt_clear:
  \BNVS_ast_plc_init:n { ##1 }
  \BNVS_ast_pfX_set:n {}
  \BNVS_next_set:n {}
}
```

`\BNVS_end_scan:n`		`\BNVS_end_scan:n {`⟨*content*⟩`*}`
`\BNVS_end_scan:`		`\BNVS_end_scan:`
`\BNVS_end_scan_next:`	`\BNVS_end_scan_ast:n`	`\BNVS_end_scan_next:`
		`\BNVS_end_scan_ast:n {`⟨*content*⟩`*}`

Exactly one of these functions must be called to close the TEX group opened while beginning the scanning process. `\BNVS_end_scan_next:` calls `\BNVS_end_scan:` and what was stored as next instructions before closing the group. `\BNVS_end_scan:` call `\BNVS_end_scan:n` on what was exported before closing the group (at this level), once prepared by `\BNVS_ast_node_make:n`... `\BNVS_end_scaN_no_I_change:` does not export `I` and `S(`⟨*I*⟩`)` variables.

```
\cs_new:Npn \BNVS_end_scan:n #1 {
  \BNVS_will_end:n {
    \__bnvs_tl_if_empty:cF { I } {
      \BNVS_tl_use:nv { \__bnvs_tl_set:cn I } I
      \BNVS_tl_use:nv { \__bnvs_tl_set:cn { S ( \l__bnvs_I_tl ) } } {
        S ( \l__bnvs_I_tl )
      }
    }
  }
```

`\BNVS_ast_node_make:n` must be called as lately as possible before ending the group.

```
  \BNVS_will_end:n {
    \BNVS_did_end:n { \BNVS_xprt_ast:n }
    \exp_args:Nx \BNVS_did_end_braced:n {
      \BNVS_ast_node_make:n { #1 }
    }
  }
  \BNVS_end:
}

\cs_new:Npn \BNVS_end_scan_ast:n #1 {
  \BNVS_ast_plc_none:
  \BNVS_end_scan:
  \BNVS_xprt_ast:n { #1 }
}

\cs_new:Npn \BNVS_end_scan: {
  \BNVS_xprted_use:n {
    \BNVS_end_scan:n
  }
}

\cs_new:Npn \BNVS_end_scan_next: {
  \BNVS_next_use:n { \BNVS_end_scan: }
}
```

\BNVS_scanned_B_wrap:n:B `\BNVS_scanned_B_wrap:n:B {⟨before⟩} {⟨scaN:B:w⟩}`
\BNVS_scanned_B_wrap:B `\BNVS_scanned_B_wrap:B {⟨scaN:B:w⟩}`

Build the `\BNVS_scanned_B:w` branch function. ⟨*before*⟩ instructions are executed before ending the current scan group whereas ⟨*scaN:B:w*⟩ instructions are executed after. They must terminate by a branch function. `\BNVS_scanned_B_wrap:n:B` calls `\BNVS_will_end:n`[1] and `\BNVS_scanned_B_wrap:B.` with an empty ⟨*before*⟩.

```
\cs_new:Npn \BNVS_scanned_B_wrap:n:B #1 {
  \BNVS_will_end:n { #1 }
  \BNVS_scanned_B_wrap:B
}

\cs_new:Npn \BNVS_scanned_B_wrap:B #1 {
  \BNVS_scanned_set:B {
    \BNVS_end_scan: #1
  }
}
```

\BNVS_no_I_change: `\BNVS_no_I_change:`
\BNVS_end_scaN_no_I_change: `\BNVSS_end_scaN_no_I_change:`

Used as ⟨*before*⟩ instruction in order not to export the frame identifier..

```
\cs_new:Npn \BNVS_no_I_change: {
  \BNVS_will_end:n { \__bnvs_tl_clear:c I }
}

\cs_new:Npn \BNVS_end_scaN_no_I_change: {
  \BNVS_no_I_change:
  \BNVS_end_scan:
}
```

1.12.6 Main scan functions

\BNVS_scaN:w `\BNVS_scaN:w ⟨input stream⟩`
\BNVS_scan:nw `\BNVS_scan:nw {⟨raw⟩} ⟨input stream⟩`

Medley of the above `\BNVS_scaN_...:Fw` scan functions. Mutability is allowed by a prior call to `\BNVS_scaN_Block_def:BF`.

```
\cs_new:Npn \BNVS_scaN:w {
  \BNVS_scaN_L:Fw {
    \BNVS_scaN_colon:Fw {
      \BNVS_scaN_S:F {
        \BNVS_scaN_Block:Fw {
          \BNVS_scaN_M:Fw {
            \BNVS_scan:nw
          }
        }
      }
    }
  }
}
```

[1] Not yet documented

```
\cs_new:Npn \BNVS_scan:nw #1 {
  \BNVS_xprt_Raw:n { #1 }
  \BNVS_scaN:w
}
```

`\BNVS_scan:RBOCMLSTN:w` `\BNVS_scan:RBOCMLSTN:w {⟨R:⟩} {⟨on_B:w⟩} {⟨on_O:w⟩} {⟨on_C:w⟩} {⟨on_M:w⟩} {⟨on_L:w⟩} {⟨on_S:⟩} {⟨T:⟩} {⟨scaN:w⟩} ⟨input stream⟩`

Start a scanning process by executing the ⟨*R:*⟩ instructions. The six next arguments are function bodies used to define the corresponding branch functions. The ⟨*T:*⟩ and ⟨*scaN:w*⟩ instructions are finally executed. They are expected to scan the input stream and terminate by calling one of the branch functions just defined. They consist of some `\BNVS_scaN_...:...w` function defined below.

The scanning functions have instructions as argument. Instead of simply using `n` in the signature of such functions, we use letters `RBOCMLSTN` as defined here, with a supplemental colon separator. Using `b` in the signature means that the instructions will be followed by a call to the `\BNVS_scanned_B:w` function.

```
\cs_new:Npn \BNVS_scan:RBOCMLSTN:w #1 #2 #3 #4 #5 #6 #7 #8 #9 {
  #1
  \BNVS_scanned_set:BOCMLS { #2 } { #3 } { #4 } { #5 } { #6 } { #7 }
  #8
  #9
```

There must be absolutely no code here. No hooking allowed either.

```
}
```

`\BNVS_begin_scan:nn` `\BNVS_begin_scan:nn {⟨xprt:n⟩} {⟨next:⟩}`

OBSOLRTE. Call `\BNVS_begin_scan:` with default arguments that end the scan group and call the eponym branch function.

```
\cs_new:Npn \BNVS_begin_scan:nn #1 #2 {
  \BNVS_begin_scan:
  \BNVS_ast_plc_init:n { #1 }
  \BNVS_next_set:n { #2 }
}
```

`\BNVS_subscan:RTN:w` `\BNVS_subscan:N:w` `\BNVS_subscan:MLSN:w` `\BNVS_subscan:OCMLSN:w`

`\BNVS_subscan:RTN:w {⟨R:⟩} {⟨T:⟩} \BNVS_subscan:N:w {⟨:w⟩} {⟨scaN_BOCMLS:w⟩} ⟨input stream⟩`
`\BNVS_subscan:OCMLSN:w {⟨O:w⟩} {⟨C:w⟩} {⟨M:w⟩} {⟨L:w⟩} {⟨S:⟩} {⟨scaN_OCMLS:w⟩} ⟨input stream⟩`

Start a scanning branch section by executing the ⟨*R:*⟩ instructions and setting the branch functions to default values. The ⟨*T:*⟩ instructions are then executed to eventually override these defaults. The process is executed within a group. Finally, the ⟨*scaN_BOCMLS:w*⟩ instructions are executed. They must terminate with a call to one of the branch functions.

```
\cs_new:Npn \BNVS_subscan:RTN:w #1 #2 #3 {
  \BNVS_scan:RBOCMLSTN:w {
    \BNVS_begin_scan:
```

```
    #1
  } {
    \BNVS_end_scan_next: \BNVS_scanned_B:w
  } {
    \BNVS_end_scan_next: \BNVS_scanned_O:w
  } {
    \BNVS_end_scan_next: \BNVS_scanned_C:w
  } {
    \BNVS_end_scan_next: \BNVS_scanned_M:w
  } {
    \BNVS_end_scan_next: \BNVS_scanned_L:w
  } {
    \BNVS_end_scan_next: \BNVS_scanned_S:
  } {
    #2
  } {
    #3
  }
}

\cs_new:Npn \BNVS_subscan:N:w {
  \BNVS_subscan:RTN:w { } { }
}

\cs_new:Npn \BNVS_subscan:OCMLSN:w #1 #2 #3 #4 #5 {
  \BNVS_subscan:RTN:w { } {
    \BNVS_scanned_set:OCMLS { #1 } { #2 } { #3 } { #4 } { #5 }
  }
```

There must be absolutely no code here. No hook is allowed either.

```
}

\cs_new:Npn \BNVS_subscan:MLSN:w #1 #2 #3 {
  \BNVS_subscan:RTN:w { } {
    \BNVS_scanned_set:MLS { #1 } { #2 } { #3 }
  }
```

There must be absolutely no code here. No hook is allowed either.

```
}
```

1.12.7 ⟨*Rnd*⟩, ⟨*Sqr*⟩ and ⟨*Crl*⟩ units

We assume that delimiters are paired and balanced. Nevertheless, some errors are catched and properly managed. Here is a grammar for material enclosed between delimiters

```
⟨Rnd(⟨blnc⟩)⟩ ::= ⟨Rnd_open⟩ ⟨blnc⟩ ⟨Rnd_close⟩
⟨Rnd_open⟩    ::= ⟨spc⟩ '(' ⟨spc⟩
⟨Rnd_close⟩   ::= ⟨spc⟩ ')' ⟨spc⟩
⟨Sqr(⟨blnc⟩)⟩ ::= ⟨Sqr_open⟩ ⟨blnc⟩ ⟨Sqr_close⟩
⟨Sqr_open⟩    ::= ⟨spc⟩ '[' ⟨spc⟩
⟨Sqr_close⟩   ::= ⟨spc⟩ ']' ⟨spc⟩
⟨Crl(⟨blnc⟩)⟩ ::= ⟨Crl_open⟩ ⟨blnc⟩ ⟨Crl_close⟩
⟨Crl_open⟩    ::= ⟨spc⟩ '{' ⟨spc⟩
⟨Crl_close⟩   ::= ⟨spc⟩ '}' ⟨spc⟩
```

with different variations of associate AST's. (missing description for ⟨*Sqr*⟩ and ⟨*Crl*⟩ are similar)

```
⟨Rnd(⟨blnc⟩)⟩⋆ ::= ⟨pfX:w⟩ { ⟨blnc⟩⋆ }
                 | ⟨pfX:w⟩   ⟨blnc⟩⋆
                 |            { ⟨blnc⟩⋆ }
                 |              ⟨blnc⟩⋆
```

The exported AST also depends on the exportation policy, whether ⟨*blnc*⟩ is empty or not.

\BNVS_scaN_L:N:EbsTN:w
\BNVS_scaN_L:N:PXbsTN:w

```
\BNVS_scaN_L:N:RBSTN:w ⟨cLose⟩
  {⟨R:⟩}          {⟨scaN:B:w⟩} {⟨scaN:S:⟩} {⟨T:⟩} {⟨scaN:w⟩}
\BNVS_scaN_L:N:PXBSTN:w ⟨cLose⟩
  {⟨xprt:n⟩} {⟨pfX:w⟩} {⟨scaN:B:w⟩} {⟨scaN:S:⟩} {⟨T:⟩} {⟨scaN:w⟩}
```

An opening delimiter has been scanned, scan the material up to the balancing closing delimiter using the ⟨*scaN:w*⟩ instructions. Implementation detail: we begin a scanning group that will be closed on scanning the ⟨*close*⟩ delimiter, or a different closing delimiter, allways at the same level. The ⟨*T:*⟩ allows some fine tuning. The ⟨*scaN:w*⟩ function scans the ⟨*input stream*⟩, and is expected to end on a closing delimiter at the current level by sending \BNVS_scanned_L:w, which meaning is ⟨*cLose*⟩ followed by \BNVS_scanned_L:w. If the ⟨*scaN:w*⟩ will call itself after a block, a comma or a colon if the corresponding branch function is called. This is the normal instruction flow.

```
\cs_new:Npn \BNVS_scaN_L:N:RBSTN:w #1 #2 #3 #4 #5 #6 {
  \BNVS_scan:RBOCMLSTN:w {
```

One group level for the contents.

```
    \BNVS_begin_scan:
    \cs_set:Npn \BNVS_scanned_L:nw ##1 {
```

Anyways, branch here.

```
      \BNVS_scanned_L:w
    }
    #2
  } {
```

Any branch function, but the two last ones, executes the ⟨*scaN:w*⟩ instructions.

```
    #6
  } {
    #6
  } {
    #6
  } {
    #6
  } {
```

Close branch. End the group, execute ⟨*block:*⟩ instructions and calls the function \BNVS_scanned_B:w. Intermediate groups should be closed appropriately with the right definition of branch functions. The value of `I` is not exported to the containing group.

```
    \BNVS_end_scaN_no_I_change:
    #3
```

```
  } {
```

Stop branch. Execute the ⟨*stop:*⟩ instructions after the group ends.

```
    \BNVS_error:n { Missing~`#1'. }
    \BNVS_end_scaN_no_I_change:
    #4
  } {
```

Execute the ⟨*T:*⟩ instructions.

```
    #5
  } {
```

Execute the ⟨*scaN:w*⟩ instructions.

```
    #6
  }
}

\cs_new:Npn \BNVS_scaN_L:N:PXBSTN:w #1 #2 #3 {
  \BNVS_scaN_L:N:RBSTN:w #1 {
    \BNVS_ast_plc_init_pfx:n { #2 }
    \BNVS_ast_pfX_will_set:n { #3 }
  }
}
```

`\c__bnvs_Rnd_open_regex` Round opening delimiter.

```
\regex_const:Nn \c__bnvs_Rnd_open_regex { \s* \( %)
  \s* }
```

(*End of definition for* `\c__bnvs_Rnd_open_regex`.)

`\BNVS_scaN_Rnd:RBSTN:Fw` `\BNVS_scaN_Rnd:RBSTN:Fw {⟨R:⟩} {⟨scaN:B:w⟩} {⟨scaN:S:⟩} {⟨T:⟩} {⟨scaN:w⟩} {⟨F:w⟩} ⟨input stream⟩`

Scan a ⟨*Rnd*⟩ unit if any and execute one of ⟨*block:*⟩ or ⟨*S:*⟩ instructions, otherwise execute ⟨*F:w*⟩ instructions.

```
\cs_new:Npn \BNVS_scaN_Rnd:RBSTN:Fw #1 #2 #3 #4 #5 #6 {
  \BNVS_peek_regex_remove_once:NTF \c__bnvs_Rnd_open_regex {
    \BNVS_scaN_L:N:RBSTN:w %(
    ) {
      \BNVS_ast_plc_init_grp:n { ##1 }
      #1
    } { #2 } { #3 } { #4 } { #5 }
  } {
    #6
  }
}
```

`\c__bnvs_Sqr_open_regex` Square opening delimiter.

```
\regex_const:Nn \c__bnvs_Sqr_open_regex { \s* \[ %]
  \s* }
```

(*End of definition for* `\c__bnvs_Sqr_open_regex`.)

`\BNVS_scaN_Sqr:RBSTN:Fw` `\BNVS_scaN_Sqr:RBSTN:Fw {⟨R:⟩} {⟨scaN:B:w⟩} {⟨scaN:S:⟩} {⟨T:⟩} {⟨scaN:w⟩} {⟨F:w⟩} ⟨input stream⟩`

Scan ⟨*Sqr*⟩ unit with the help of ⟨*scaN:w*⟩, then execute one of `\BNVS_scanned_block:` or `\BNVS_scanned_S:` instructions. ⟨*scaN:w*⟩ must end with one of the branch functions `\BNVS_scanned_L:w` or `\BNVS_scanned_S:`. Otherwise execute ⟨*F:w*⟩ instructions.

```
\cs_new:Npn \BNVS_scaN_Sqr:RBSTN:Fw #1 #2 #3 #4 #5 #6 {
  \BNVS_peek_regex_remove_once:NTF \c__bnvs_Sqr_open_regex {
    \BNVS_scaN_L:N:RBSTN:w %[
    ] { #1 } { #2 } { #3 } { #4 } { #5 }
  } {
    #6
  }
}
```

`\c__bnvs_Crl_open_regex` Curly opening delimiter.

(*End of definition for* `\c__bnvs_Crl_open_regex`.)

```
\regex_const:Nn \c__bnvs_Crl_open_regex { \s* \{ %}
  \s* }
```

`\BNVS_scaN_Crl:RBSTN:Fw` `\BNVS_scaN_Crl:RBSTN:Fw {⟨R:⟩} {⟨block:⟩} {⟨stop:⟩} {⟨scaN:w⟩} {⟨F:w⟩} ⟨input stream⟩`

Scan ⟨*Crl*⟩ unit with the help of ⟨*scaN:w*⟩, then execute one of ⟨*block:*⟩ or ⟨*S:*⟩ instructions. ⟨*scaN:w*⟩ must end with one of the branch functions `\BNVS_scanned_L:w` or `\BNVS_scanned_S:`. Otherwise execute ⟨*F:w*⟩ instructions.

```
\group_begin:
\char_set_catcode_group_begin:N (
\char_set_catcode_group_end:N )
\char_set_catcode_other:N \{
\char_set_catcode_other:N \}
\cs_new:Npn \BNVS_scaN_Crl:RBSTN:Fw #1 #2 #3 #4 #5 #6 (
  \BNVS_peek_regex_remove_once:NTF \c__bnvs_Crl_open_regex (
    \BNVS_scaN_L:N:RBSTN:w %{
    } ( #1 ) ( #2 ) ( #3 ) ( #4 ) ( #5 )
  ) (
    #6
  )
)
\group_end:
```

1.12.8 Raw units

```
⟨Raw⟩      ::= ⟨Raw_open⟩ ⟨call⟩ ⟨Rnd_close⟩
⟨Raw_open⟩ ::= ⟨spc⟩ '"(' ⟨spc⟩
⟨RawLst⟩   ::= ( ⟨RawRnd⟩ | ⟨RawSqr⟩ | ⟨RawCrl⟩ | ⟨RawOtr⟩ )*
⟨RawRnd⟩   ::= ⟨Rnd_open⟩ ⟨RawLst⟩ ⟨Rnd_close⟩
⟨RawSqr⟩   ::= ⟨Sqr_open⟩ ⟨RawLst⟩ ⟨Sqr_close⟩
⟨RawCrl⟩   ::= ⟨Crl_open⟩ ⟨RawLst⟩ ⟨Crl_close⟩
⟨RawOtr⟩   ::= ⟨characters of category other except delimiters⟩
```

\c__bnvs_open_Raw_regex Raw opening delimiter.

```
\regex_const:Nn \c__bnvs_open_Raw_regex { \s* "\( %)
  \s* }
```

(*End of definition for* `\c__bnvs_open_Raw_regex`.)

\BNVS_scaN_Raw_LS:w `\BNVS_scaN_Raw_LS:w` ⟨*input stream*⟩
\BNVS_scaN_Raw:nw `\BNVS_scaN_Raw:nw` {⟨*input stream*⟩} ⟨*input stream*⟩

Scan balanced material taking only delimiters into account. It stops on an unbalanced closing delimiter or in a stop situation. Everything in between is exported as is.

```
\cs_new:Npn \BNVS_scaN_Raw:nw #1 {
  \BNVS_xprt_ast:n { #1 }
  \BNVS_scaN_Raw_LS:w
}

\group_begin:
\char_set_catcode_group_begin:N <
\char_set_catcode_group_end:N >
\char_set_catcode_other:N \{
\char_set_catcode_other:N \}
\cs_new:Npn \BNVS_scaN_Raw_LS:w  <
```

If we find an opening delimiter.

```
  \BNVS_scaN_Rnd:RBSTN:Fw <
    \BNVS_xprt_space_set:n < \BNVS_xprt_ast:n < ~ > >%(
    \BNVS_ast_plc_init_mpt:n < ##1 ) >
  > <
    \BNVS_scanned_set:L <
      \BNVS_end_scaN_no_I_change:
      \BNVS_scaN_Raw_LS:w
    >
    \BNVS_scanned_B:w
  > <%(
    \BNVS_error:n < Missing~`)' > %(
    \BNVS_xprt_ast:n < ) >
    \BNVS_scanned_S:
  > <
    \BNVS_xprt_ast:n < ( > %)
  > <
    \BNVS_scaN_Raw_LS:w
  > <
```

```
    \BNVS_scaN_Sqr:RBSTN:Fw <
      \BNVS_xprt_space_set:n < \BNVS_xprt_ast:n < ~ > >%[
      \BNVS_ast_plc_init_mpt:n < ##1 ] >
    > <
      \BNVS_scanned_set:L <
        \BNVS_end_scaN_no_I_change:
        \BNVS_scaN_Raw_LS:w
      >
      \BNVS_scanned_B:w
    > <%[
      \BNVS_error:n < Missing~`]' > %[
      \BNVS_xprt_ast:n < ] >
      \BNVS_scanned_S:
    > <
      \BNVS_xprt_ast:n < [ > %]
    > <
      \BNVS_scaN_Raw_LS:w
    > <
      \BNVS_scaN_Crl:RBSTN:Fw <
        \BNVS_xprt_space_set:n < \BNVS_xprt_ast:n < ~ > >%{
        \BNVS_ast_plc_init_mpt:n < ##1 } >
      > <
        \BNVS_scanned_set:L <
          \BNVS_end_scaN_no_I_change:
          \BNVS_scaN_Raw_LS:w
        >
        \BNVS_scanned_B:w
      > <%{
        \BNVS_error:n < Missing~`}' > %{
        \BNVS_xprt_ast:n < } >
        \BNVS_scanned_S:
      > <
        \BNVS_xprt_ast:n < { > %}
      > <
        \BNVS_scaN_Raw_LS:w
      > <
```

The head of the ⟨*input stream*⟩ is not an opening delimiter, we look for a closing delimiter, or a stop situation or finally export one character.

```
        \BNVS_scaN_L:Fw < \BNVS_scaN_S:F < \BNVS_scaN_Raw:nw > >
> > > >
\group_end:
```

\BNVS_scaN_Raw:BS:Fw `\BNVS_scaN_Raw:BS:Fw {⟨scaN:B:w⟩} {⟨scaN:S:⟩} {⟨F:w⟩} ⟨input stream⟩`

Scan balanced material taking only delimiters into account. Starts on a `"(`.

```
\cs_new:Npn \BNVS_scaN_Raw:BS:Fw #1 #2 #3 {
  \BNVS_peek_regex_remove_once:NTF \c__bnvs_open_Raw_regex {
    \BNVS_scaN_L:N:RBSTN:w %(
    ) {
      \BNVS_ast_plc_init_pfx:n { \:n { ##1 } }
      \BNVS_xprt_space_set:n { \BNVS_xprt_ast:n { ~ } }
    } { #1 } { #2 } { } {
      \BNVS_scaN_Raw_LS:w
```

```
    }
  } {
    #3
  }
}
```

1.12.9 ⟨*csl(⟨item⟩)*⟩ units

Here is a partial grammar for a comma separated list template.

⟨*csl(⟨item⟩)*⟩ ::= ⟨*item*⟩? (⟨*comma*⟩ ⟨*item*⟩?)*

The corresponding AST template is below, where ⟨*item*⟩* is the AST for ⟨*item*⟩:

⟨*csl(⟨item⟩)*⟩* ::= { ⟨*item*⟩* } { ⟨*item*⟩* } ⟨*...*⟩ { ⟨*item*⟩* }

\BNVS_scaN_csl:RLSTN:w

\BNVS_scaN_csl:RLSTN:w {⟨*R:*⟩} {⟨*on_L:w*⟩} {⟨*on_S:*⟩} {⟨*T:*⟩} {⟨*scaN_LS:w*⟩} ⟨*input stream*⟩

This function scans the ⟨*input stream*⟩ for a comma separated list of items, regardless enclosing delimiters, according to the ⟨*scaN_LS:w*⟩ instructions. These instruction must end with a branch function, of which only \BNVS_scanned_L:w and \BNVS_scanned_S: are catched. The scanned material is then exported according to the instructions given in the ⟨*R:*⟩. On scanning a comma, a new group is created. The exported material is empty for an empty list, for input `a` it is {a*}, for input `a,b` it is {a*}{b*}, and so on.

```
\cs_new:Npn \BNVS_scaN_csl:RLSTN:w #1 #2 #3 #4 #5 {
  \BNVS_scan:RBOCMLSTN:w {
```

One group level for each item of the list.

```
    \BNVS_begin_scan:
```

Execute the ⟨*R:*⟩ instructions.

```
    \BNVS_ast_plc_init:n { { ##1 } }
    #1
  }
```

For the first branch functions (BOC), execute the ⟨*scaN:w*⟩ instructions.

```
    { #5 }
    { #5 }
    { #5 }
    {
      \BNVS_if_xprted_empty:TF {
```

Comma branch, the first item is empty, skip it.

```
        #5
      } {
```

Comma branch, export the first item when not empty. Forwards `I`, to the outer group.

```
      \BNVS_end_scan:
      \BNVS_scaN_csl:RLSTN:w { #1 } { } { } { #4 } { #5 }
    }
  }
```

Close branch.

```
  { \BNVS_end_scan: #2 \BNVS_scanned_L:w }
  { \BNVS_end_scan: #3 \BNVS_scanned_S: }
```

By default, the list contains only one item.

```
  { #4 }
  { #5 }
}
```

`\BNVS_scaN_csl:RT:w` `\BNVS_scaN_csl:RT:w {⟨R:⟩} {⟨T:⟩} ⟨input stream⟩`

These functions scan the comma separated list of items, regardless the enclosing delimiters, based on `\BNVS_scaN:w`.

```
\cs_new:Npn \BNVS_scaN_csl:RT:w #1 #2 {
  \BNVS_scaN_csl:RLSTN:w
    { #1 }
    { }
    { }
    { #2 }
    { \BNVS_scaN:w }
}
```

1.12.10 ISR unit

Here is a partial grammar for identifiers,

```
⟨ISR⟩     ::= ( ⟨IS⟩ | ⟨dot_P⟩ ) ⟨R⟩*
⟨IS⟩      ::= ( ⟨I⟩? '!' )? ⟨S⟩
⟨I⟩       ::= ⟨S⟩
⟨S⟩       ::= ⟨alpha⟩ ⟨alnum⟩*
⟨dot_P⟩   ::= '.' ⟨P⟩
⟨P⟩       ::= ⟨digit⟩* ⟨S⟩
⟨R⟩       ::= ⟨dot_P⟩ | ⟨dot_N⟩ | ⟨n⟩
⟨dot_N⟩   ::= '.' ⟨N⟩
⟨n⟩       ::= ⟨Sqr_open⟩ ⟨n_expr⟩ ⟨Sqr_close⟩
⟨n_expr⟩  ::= ⟨blnc⟩
```

and the associate AST :

```
⟨IS⟩⋆  ::= \IS:w  { ⟨I⟩ } { ⟨S⟩ }
⟨ISR⟩⋆ ::= \ISR:w { ⟨I⟩ } { ⟨S⟩ } { ⟨R⟩⋆ }
⟨R⟩⋆   ::= ( ⟨P⟩⋆ | ⟨N⟩⋆ | ⟨n⟩⋆ )*
⟨P⟩⋆   ::= { \P:n { ⟨P⟩ } }
⟨N⟩⋆   ::= { \N:n { ⟨N⟩ } }
⟨n⟩⋆   ::= { \n:n { ⟨n_expr⟩⋆ } }
```

\c__bnvs_IKS_regex The start of a qualified dotted name, with three capture groups.

(*End of definition for* `\c__bnvs_IKS_regex`.)

```
\regex_const:Nn \c__bnvs_IKS_regex {
```

1: the optional frame ⟨*id*⟩ with

2: the optional exclamation mark

```
  (?: ( \ur{c__bnvs_S_regex} )? ( ! ) )?
```

3: followed by a non empty short name.

```
  ( \ur{c__bnvs_S_regex} )
}
```

\BNVS_scanned:B:IKS:w `\BNVS_scanned:B:IKS:w {⟨scaN:B:w⟩} {⟨I⟩} {⟨K⟩} {⟨S⟩} ⟨input stream⟩`

Only called by `\BNVS_scaN_IS:B:Fw`. Execute ⟨*scaN:B:w*⟩ instructions, which terminate by `\BNVS_scanned_B:w`.

```
\cs_new:Npn \BNVS_scanned:B:IKS:w #1 #2 #3 #4 {
  \tl_if_empty:nTF { #3 } {
```

No frame identifier given, use the current value. Store the short name, store in the appropriate variable.

```
    \__bnvs_tl_clear_new:c { S( \BNVS_tl_use:c I ) }
    \__bnvs_tl_set:cn { S( \BNVS_tl_use:c I ) } { #4 }
  } {
```

Frame identifier given. Proceed as before mutatis mutandis.

```
    \__bnvs_tl_set:cn I { #2 }
    \__bnvs_tl_clear_new:c { S(#2) }
    \__bnvs_tl_set:cn { S(#2) } { #4 }
  }
  \__bnvs_tl_set:cn S { #4 }
```

Create a scan group to manage the ⟨*pfX:w*⟩.

```
  \BNVS_subscan:RTN:w {
```

By default we start on a standalone identifier,

```
    \BNVS_ast_plc_init:n { ##1 }
    \BNVS_ast_pfX_set:n { \IS:w }
  } {
```

Export ⟨*I*⟩ and ⟨*S*⟩.

```
    \BNVS_tl_use:Nv \BNVS_xprt_grp:n I
    \BNVS_xprt_grp:n { #4 }
  } {
```

Execute the ⟨*scaN:B:w*⟩ instructions including the terminating branch function `\BNVS_scanned_B:w`. The ⟨*pfX:w*⟩ will possibly change.

```
    \cs_set:Npn \BNVS_scaN:B:w ##1 { #1 }
    \BNVS_scaN:B:w { \BNVS_scanned_B:w }
  }
}
```

\BNVS_scaN_IS:B:Fw `\BNVS_scaN_IS:B:Fw {⟨scaN:B:w⟩} {⟨F:w⟩} ⟨input stream⟩`

Scan an ⟨IS⟩ unit if any, stores the ⟨I⟩, ⟨S⟩ between braces, and calls `\BNVS_scanned:B:IKS:w` that will execute ⟨scaN:B:w⟩ instructions (see above). The ⟨F:w⟩ instructions are executed when no ⟨IS⟩ unit is found.

```
\cs_new:Npn \BNVS_scaN_IS:B:Fw #1 #2 {
  \peek_regex_replace_once:NnTF \c__bnvs_IKS_regex {
    \cB\{ \1 \cE\} \cB\{ \2 \cE\} \cB\{ \3 \cE\}
  } {
    \BNVS_scanned:B:IKS:w { #1 }
  } {
```

Choose the false branch.

```
    #2
  }
}
```

\BNVS_scanning_PNn:P:w `\BNVS_scanning_PNn:P:w {⟨P⟩} ⟨input stream⟩`

Export the ⟨P⟩ component then scan another ⟨PNn⟩ component.

```
\cs_new:Npn \BNVS_scanning_PNn:P:w #1 {
  \BNVS_xprt_grp:n { \P:n { #1 } }
  \BNVS_scaN_PNn:w
}
```

\BNVS_scanning_PNn:N:w `\BNVS_scanning_PNn:P:w {⟨N⟩} ⟨input stream⟩`

Export the ⟨N⟩ component then scan another ⟨PNn⟩ component.

```
\cs_new:Npn \BNVS_scanning_PNn:N:w #1 {
```

Normalize ⟨N⟩.

```
  \exp_args:Ne \BNVS_xprt_grp:n { \exp_not:N \N:n { \int_eval:n { #1 } } }
  \BNVS_scaN_PNn:w
}
```

\BNVS_scanning_PNn_n:w `\BNVS_scanning_PNn_n:w ⟨input stream⟩`

Only called by `\BNVS_scaN_PNn:w`.

```
\cs_new:Npn \BNVS_scanning_PNn_n:w {
  \BNVS_scaN_L:N:RBSTN:w %[
  ] {
    \BNVS_ast_plc_init:n { { \n:n { ##1 } } }
  } {
    \BNVS_scanned_B:w
  } {
    \BNVS_scanned_S:
  } {

```

On cLose branch, scan ⟨PNn⟩ after ⟨n⟩.

```
      \BNVS_scanned_set:L { \BNVS_end_scaN_no_I_change: \BNVS_scaN_PNn:w }
    } {
      \BNVS_scaN_n_expr_LS:w
    }
}
```

`\BNVS_scaN_PNn:w` `\BNVS_scaN_PNn:w` ⟨*input stream*⟩

Scan all the individual ⟨*R*⟩ components, exporting them one after the other, then execute `\BNVS_scanned_B:w` branch function. Calls `\BNVS_scanning_PNn:P:w`, `\BNVS_scanning_PNn:N:w` or `\BNVS_scanning_PNn_n:w`.

```
\cs_new:Npn \BNVS_scaN_PNn:w {
  \peek_charcode_remove:NTF . {
    \peek_regex_replace_once:NnTF \c__bnvs_P_regex { \cB\{ \0 \cE\} } {
      \BNVS_scanning_PNn:P:w
    } {
      \peek_regex_replace_once:NnTF \c__bnvs_N_regex { \cB\{ \0 \cE\} } {
        \BNVS_scanning_PNn:N:w
      } {
```

Terminates the block and reinject the dot '.'.

```
        \BNVS_scanned_B:w .
      }
    }
  } {
    \BNVS_peek_regex_remove_once:NTF \c__bnvs_Sqr_open_regex {
      \BNVS_scanning_PNn_n:w
    } {
```

Also terminates the block.

```
      \BNVS_scanned_B:w
    }
  }
}
```

`\BNVS_scanning_R:B:P:w` `\BNVS_scanning_R:B:P:w` {⟨*scaN_B:w*⟩} {⟨*P*⟩} ⟨*input stream*⟩

Only called by `\BNVS_scaN_R:B:w` when a heading ⟨*P*⟩ has just been scanned. Calls `\BNVS_scaN_PNn:w`.

```
\cs_new:Npn \BNVS_scanning_R:B:P:w #1 #2 {
```

New scan group to gather the ⟨*R*⟩ relative path.

```
  \BNVS_subscan:RTN:w {
    \BNVS_ast_plc_init_grp:n { ##1 }
    \BNVS_xprt_grp:n { \P:n { #2 } }
  } {
    \BNVS_scanned_set:B {
```

```
      \BNVS_end_scaN_no_I_change:
      #1
    }
  } {
    \BNVS_scaN_PNn:w
  }
}
```

`\BNVS_scanning_R:B:N:w` `\BNVS_scanning_R:B:N:w {⟨scaN_B:w⟩} {⟨N⟩} ⟨input stream⟩`

Only called by `\BNVS_scaN_R:B:w` when a heading ⟨*N*⟩ has just been scanned. Calls `\BNVS_scaN_PNn:w`.

```
\cs_new:Npn \BNVS_scanning_R:B:N:w #1 #2 {
```

New scan group to gather the ⟨*R*⟩ relative path.

```
  \BNVS_subscan:RTN:w {
    \BNVS_ast_plc_init_grp:n { ##1 }
    \exp_args:Ne \BNVS_xprt_grp:n {
```

Normalize ⟨*N*⟩.

```
      \exp_not:N \N:n { \int_eval:n { #2 } }
    }
  } {
    \BNVS_scanned_set:B {
      \BNVS_end_scaN_no_I_change:
      #1
    }
  } {
    \BNVS_scaN_PNn:w
  }
}
```

`\BNVS_scanning_R_n:B:w` `\BNVS_scanning_R_n:B:w {⟨scaN_B:w⟩} ⟨input stream⟩`

Only called by `\BNVS_scaN_R:B:w` when a heading '[' has just been scanned. Terminates by `\BNVS_scaN_PNn:w` to scan another component. See above for ⟨*scaN:B:w*⟩.

```
\cs_new:Npn \BNVS_scanning_R_n:B:w #1 {
```

New scan group to gather the ⟨*R*⟩ relative path.

```
  \BNVS_subscan:RTN:w {
    \BNVS_ast_plc_init_grp:n { ##1 }
  } {
```

On block branch, close the scan group and execute the ⟨*scaN_B:w*⟩ instructions.

```
    \BNVS_scanned_set:B {
      \BNVS_end_scaN_no_I_change:
      #1
    }
  } {
```

Export material up to the closing square bracket as one single argument group.

```
      \BNVS_scaN_L:N:RBSTN:w %[
      ] {
        \BNVS_ast_plc_init:n { { \n:n { ##1 } } }
      } {
        \BNVS_scanned_B:w
      } {
        \BNVS_scanned_S:
      } {
```

On close branch, scan ⟨*PNn*⟩ after ⟨*n*⟩.

```
        \BNVS_scanned_set:L { \BNVS_end_scaN_no_I_change: \BNVS_scaN_PNn:w }
      } {
        \BNVS_scaN_n_expr_LS:w
      }
    }
}
```

`\BNVS_scaN_R:B:w` `\BNVS_scaN_R:B:w {⟨scaN:B:w⟩} ⟨input stream⟩`

Scan a ⟨*R*⟩ component and execute ⟨*scaN_B:w*⟩ instructions. The scanning stops exactly when some regular expression matches, or when squared material is scanned. The function `\BNVS_scaN_R:B:w` calls

- `\BNVS_scanning_R:B:P:w`,
- `\BNVS_scanning_R:B:N:w` or
- `\BNVS_scanning_R_n:B:w`

with ⟨*scaN:B:w*⟩ as argument.

```
\cs_new:Npn \BNVS_scaN_R:B:w #1 {
  \peek_charcode_remove:NTF . {
    \peek_regex_replace_once:NnTF \c__bnvs_P_regex { \cB\{ \0 \cE\} } {
      \BNVS_scanning_R:B:P:w { #1 }
    } {
      \peek_regex_replace_once:NnTF \c__bnvs_N_regex { \cB\{ \0 \cE\} } {
        \BNVS_scanning_R:B:N:w { #1 }
      } {
```

Terminates the block and reinject the dot.

```
        \BNVS_xprt_grp:n { }
        #1
        .
      }
    }
  } {
    \BNVS_peek_regex_remove_once:NTF \c__bnvs_Sqr_open_regex {
      \BNVS_scanning_R_n:B:w { #1 }
    } {
```

Terminates the block too.

```
      \BNVS_xprt_grp:n { }
      #1
```

```
    }
  }
}

\tl_new:N \l__bnvs_other_X_tl
\tl_set:Ne \l__bnvs_other_X_tl { \tl_to_str:n { X } }
```

`\BNVS_scanned_dot:B:P:w` `\BNVS_scanned_dot:B:P:w {⟨scaN:B:w⟩} {⟨P⟩} ⟨input stream⟩`
`\BNVS_scanned_dot:n:B:P:w` `\BNVS_scanned_dot:B:P:w {⟨before⟩} {⟨scaN:B:w⟩} {⟨P⟩} ⟨input stream⟩`

Only called by `\BNVS_scaN_dot_P:B:Fw`. The second functions calls rhe first one with an empty ⟨*before*⟩.

```
\cs_new:Npn \BNVS_scanned_dot:n:B:P:w #1 #2 #3 {
```

No frame identifier given, no short name given.

```
  \__bnvs_tl_if_exist:cF { S( \BNVS_tl_use:c I ) } {
```

It defaults to `X`.

```
    \BNVS_tl_new:c { S( \BNVS_tl_use:c I ) }
    \__bnvs_tl_set:cv { S( \BNVS_tl_use:c I ) } { other_X }
  }
  \__bnvs_tl_set:cv S { S( \BNVS_tl_use:c I) }
```

Create a scan group to manage the ⟨*pfX:w*⟩. It will be closed by a call to `\BNVS_scanned_B:w`.

```
  \BNVS_subscan:RTN:w {
```

By default we start on a standalone identifier,

```
    \BNVS_ast_pfX_set:n { \ISR:w }
    \exp_args:Ne \BNVS_ast_plc_init_mpt:n {
      { \__bnvs_tl_use:c I }
      { \__bnvs_tl_use:c S }
      { \exp_not:n { ##1 } }
    }
    \BNVS_xprt_ast:n { { \P:n { #3 } } }
  } {
    \BNVS_scanned_B_wrap:n:B { #1 } { #2 }
```

Create a scan group to manage the ⟨*pfX:w*⟩.

```
  } {
```

Find the next components.

```
    \BNVS_scaN_PNn:w
  }
}

\cs_new:Npn \BNVS_scanned_dot:B:P:w {
  \BNVS_scanned_dot:n:B:P:w {}
}
```

`\BNVS_scaN_dot_P:B:Fw` `\BNVS_scaN_dot_P:B:Fw {⟨scaN:B:w⟩} {⟨F:w⟩} ⟨input stream⟩`

Scan a ⟨*dot_P*⟩ unit and execute ⟨*scaN:B:w*⟩ instructions that must end with `\BNVS_scanned_B:w`. The ⟨*F:w*⟩ instructions are executed when no ⟨*dot_P*⟩ unit is found.

```
\cs_new:Npn \BNVS_scaN_dot_P:B:Fw #1 #2 {
  \peek_charcode_remove:NTF . {
    \peek_regex_replace_once:NnTF \c__bnvs_P_regex { \cB\{ \0 \cE\} } {
      \BNVS_scanned_dot:B:P:w { #1 }
    } {
```

Choose the false branch and reinject the dot.

```
      #2
      .
    }
  } {
```

Choose also the false branch.

```
    #2
  }
}
```

`\BNVS_scanned_dot:n:B:N:w` `\BNVS_scanned_dot:n:B:N:w {⟨before⟩} {⟨scaN:B:w⟩} {⟨N⟩} ⟨input stream⟩`
`\BNVS_scanned_dot:B:N:w` `\BNVS_scanned_dot:B:N:w {⟨scaN:B:w⟩} {⟨N⟩} ⟨input stream⟩`

Only called by `\BNVS_scaN_dot_N:B:Fw`.

```
\cs_new:Npn \BNVS_scanned_dot:n:B:N:w #1 #2 #3 {
```

No frame identifier given, no short name given.

```
  \__bnvs_tl_if_exist:cF { S( \BNVS_tl_use:c I ) } {
```

It defaults to `X`.

```
    \BNVS_tl_new:c { S( \BNVS_tl_use:c I ) }
    \__bnvs_tl_set:cv { S( \BNVS_tl_use:c I ) } { other_X }
  }
  \__bnvs_tl_set:cv S { S( \BNVS_tl_use:c I) }
```

Create a scan group to manage the ⟨*pfX:w*⟩. It will be closed by a call to `\BNVS_scanned_B:w`.

```
  \BNVS_subscan:RTN:w {
```

By default we start on a standalone identifier,

```
    \BNVS_ast_pfX_set:n { \ISR:w }
    \exp_args:Ne \BNVS_ast_plc_init_mpt:n {
      { \__bnvs_tl_use:c I }
      { \__bnvs_tl_use:c S }
      { \exp_not:n { ##1 } }
    }
    \exp_args:Ne \BNVS_xprt_ast:n {
      { \exp_not:N \N:n { \int_eval:n { #3 } } }
    }
  } {
    \BNVS_scanned_B_wrap:n:B { #1 } { #2 }
```

Create a scan group to manage the ⟨*pfX:w*⟩.

```
  } {
```

Find the next components.

```
    \BNVS_scaN_PNn:w
  }
}

\cs_new:Npn \BNVS_scanned_dot:B:N:w {
  \BNVS_scanned_dot:n:B:N:w { }
}
```

Create a scan group to manage the ⟨*pfX:w*⟩.

\BNVS_scaN_dot_N:B:Fw `\BNVS_scaN_dot_N:B:Fw {⟨scaN:B:w⟩} {⟨F:w⟩} ⟨input stream⟩`

Scan a ⟨*dot_N*⟩ unit and execute ⟨*scaN:B:w*⟩ instructions that must end with \BNVS_scanned_B:w. The ⟨*F:w*⟩ instructions are executed when no ⟨*dot_N*⟩ unit is found.

```
\cs_new:Npn \BNVS_scaN_dot_N:B:Fw #1 #2 {
  \peek_charcode_remove:NTF . {
    \peek_regex_replace_once:NnTF \c__bnvs_N_regex { \cB\{ \0 \cE\} } {
      \BNVS_scanned_dot:B:N:w { #1 }
    } {
```

Choose the false branch and reinject the dot.

```
      #2
      .
    }
  } {
```

Choose also the false branch.

```
    #2
  }
}
```

\BNVS_scaN_ISR:B:Fw `\BNVS_scaN_ISR:B:Fw {⟨scaN:b:⟩} {⟨F:w⟩} ⟨input stream⟩`

Scan an ⟨*ISR*⟩ unit if any, ⟨*scaN:b:*⟩ instructions terminated by a call to the branch function \BNVS_scanned_B:w. The ⟨*F:w*⟩ instructions are executed when no ⟨*ISR*⟩ unit is found.

```
\cs_new:Npn \BNVS_scaN_ISR:B:Fw #1 #2 {
  \BNVS_scaN_IS:B:Fw {
    \BNVS_scaN_R:B:w { #1 }
  } {
    #2
  }
}
```

1.12.11 Assignment units

Here is the grammar

```
⟨ISRMOR⟩   ::= ⟨ISR⟩ ⟨MOR⟩
⟨MOR⟩      ::= ⟨ModOpEq⟩ ⟨Rhs⟩
⟨ModOpEq⟩  ::= ⟨spc⟩ ( ⟨Mod⟩ ⟨Op⟩ '=' ) ⟨spc⟩
⟨Mod⟩      ::= '' | '?'
⟨Op⟩       ::= '' | '+' | '-' | '/' | '*'
⟨Rhs⟩      ::= ⟨see below⟩
```

with the associate AST

```
⟨ISRMOR⟩* ::= \ISRMOR:w { ⟨I⟩ } { ⟨S⟩ } { ⟨R⟩* } { ⟨Mod⟩ } { ⟨Op⟩ } {
↪  ⟨Rhs⟩* }
```

`\c__bnvs_ModOpEq_regex` Assignment operators, two capture groups.

(*End of definition for* `\c__bnvs_ModOpEq_regex`.)

```
\regex_const:Nn \c__bnvs_ModOpEq_regex {
  \s*
```

1: an optional modifier.

```
  ( \? )?
```

2: an optional operator.

```
  ( [-+/*] )?

  =\s*
}
```

`\BNVS_scanned_ModOpEq:MLS:nnw` `\BNVS_scanned_ModOpEq:MLS:nnw {⟨on_M:w⟩} {⟨on_L:w⟩} {⟨on_S:⟩} {⟨Mod⟩} {⟨Op⟩} ⟨input stream⟩`

Only called by `\BNVS_scaN_MOR:MLS:Fw`. Calls `\BNVS_scaN_MLS:w`.

```
\cs_new:Npn \BNVS_scanned_ModOpEq:MLS:nnw #1 #2 #3 #4 #5 {
  \BNVS_xprt_grp:n { #4 }
  \BNVS_xprt_grp:n { #5 }
  \BNVS_scan:RBOCMLSTN:w {
    \BNVS_begin_scan:
    \BNVS_ast_plc_init_grp:n { ##1 }
  } { \BNVS_error:n { Unexpected } \BNVS_scanned_L:w }
    { \BNVS_error:n { Unexpected } \BNVS_scanned_L:w }
    { \BNVS_error:n { Unexpected } \BNVS_scanned_L:w }
    { \BNVS_end_scaN_no_I_change: #1 }
    { \BNVS_end_scaN_no_I_change: #2 }
    { \BNVS_end_scaN_no_I_change: #3 }
    { }
    { \BNVS_scaN_MLS:w }
}
```

\BNVS_scaN_MOR:MLS:Fw `\BNVS_scaN_MOR:MLS:Fw {⟨on_M:w⟩} {⟨on_L:w⟩} {⟨on_S:⟩} {⟨F:w⟩} ⟨input stream⟩`

Scan a ⟨*ModOpEq*⟩ unit, if any, and execute one of ⟨*comMa:w*⟩, ⟨*L:w*⟩ or ⟨*S:*⟩ instructions, otherwise execute ⟨*F:w*⟩ instructions.

```
\cs_new:Npn \BNVS_scaN_MOR:MLS:Fw #1 #2 #3 #4 {
  \peek_regex_replace_once:NnTF \c__bnvs_ModOpEq_regex {
    \cB\{ \1 \cE\} \cB\{ \2 \cE\}
  } {
    \BNVS_ast_pfX_will_set:n { \ISRMOR:w }
    \BNVS_scanned_ModOpEq:MLS:nnw { #1 } { #2 } { #3 }
  } {
    #4
  }
}
```

\BNVS_scaN_ISRMOR:MLS:Fw `\BNVS_scaN_ISRMOR:MLS:Fw {⟨on_M:w⟩} {⟨on_L:w⟩} {⟨on_S:⟩} {⟨F:w⟩} ⟨input stream⟩`

Scan a ⟨*ISRMOR*⟩ unit, if any, and execute one of ⟨*comMa:w*⟩, ⟨*L:w*⟩ or ⟨*S:*⟩ instructions, otherwise execute ⟨*F:w*⟩ instructions.

```
\cs_new:Npn \BNVS_scaN_ISRMOR:MLS:Fw #1 #2 #3 #4 {
  \BNVS_scaN_ISR:B:Fw {
    \BNVS_scaN_MOR:MLS:Fw {
      \BNVS_ast_pfX_will_set:n { \ISRMOR }
      #1
    } {
      \BNVS_ast_pfX_will_set:n { \ISRMOR }
      #2
    } {
      \BNVS_ast_pfX_will_set:n { \ISRMOR }
      #3
    } { #4 }
  } {
    #4
  }
}
```

1.12.12 Call units

Here is the grammar

⟨*ISRC*⟩ ::= ⟨*ISR*⟩ ⟨*call*⟩
⟨*call*⟩ ::= ⟨*Rnd_open*⟩ ⟨*blnc*⟩ ⟨*Rnd_close*⟩

with the associate AST

⟨*ISRC*⟩* ::= `\ISRC:w` { ⟨*I*⟩ } { ⟨*S*⟩ } { ⟨*R*⟩* } { ⟨*Raw*⟩* }

\BNVS_scaN_call:B:w `\BNVS_scaN_call:B:Fw {⟨scaN:B:w⟩} ⟨input stream⟩`

Scan a ⟨call⟩ unit, if any, and execute ⟨scaN:B:w⟩ instructions.

```
\cs_new:Npn \BNVS_scaN_call:B:w #1 {
  \BNVS_scaN_Rnd:RBSTN:Fw { } { #1 } {
    \BNVS_scanned_S:
  } { } {
    \BNVS_scaN_Raw_LS:w
  } { #1 }
}
```

\BNVS_scaN_ISRC:B:Fw `\BNVS_scaN_ISRC:B:Fw {⟨scaN:B:w⟩} {⟨F:w⟩} ⟨input stream⟩`

Scan a ⟨ISRC⟩ unit, if any, and execute ⟨scaN:B:w⟩ instructions, otherwise execute ⟨F:w⟩ instructions.

```
\cs_new:Npn \BNVS_scaN_ISRC:B:Fw #1 {
  \BNVS_scaN_ISR:B:Fw {
    \BNVS_ast_pfX_will_set:n { \ISRC:w }
    \BNVS_scaN_call:B:w { #1 }
  }
```

There must be absolutely no code here. No hook is allowed either.

```
}
```

1.12.13 VAZL units

Here is a preliminary grammar for ranges or values:

```
⟨V⟩        ::= ⟨MLS⟩
⟨A⟩        ::= ⟨MLS⟩
⟨Z⟩        ::= ⟨MLS⟩
⟨L⟩        ::= ⟨MLS⟩
⟨VAZL⟩     ::= ⟨V⟩ | ⟨AZL⟩ | ⟨ALZ⟩
⟨AZL⟩      ::= ⟨A⟩? ':'  ⟨Z⟩ ( "::" ⟨L⟩? )?
⟨ALZ⟩      ::= ⟨A⟩? "::" ⟨L⟩ ( ':'  ⟨Z⟩? )?
⟨MLS⟩      ::= ( ⟨spc⟩ | ⟨ISR⟩ | ⟨ISRC⟩ | ⟨ISRMOR⟩ | ⟨Rnd⟩ | ⟨Sqr⟩ | ⟨Crl⟩
  ↪  | ⟨Rslv⟩ | ⟨Raw⟩ | ⟨other⟩ )*
```

and the associate AST :

```
⟨V⟩⋆    ::= \V:n { ⟨MLS⟩⋆ }
⟨A⟩⋆    ::= \A:n { ⟨MLS⟩⋆ }
⟨V⟩⋆    ::= \Z:n { ⟨MLS⟩⋆ }
⟨V⟩⋆    ::= \L:n { ⟨MLS⟩⋆ }
⟨AZ⟩⋆   ::= \AZ:w { ⟨MLS⟩⋆ } { ⟨MLS⟩⋆ }
⟨AL⟩⋆   ::= \AL:w { ⟨MLS⟩⋆ } { ⟨MLS⟩⋆ }
⟨ZL⟩⋆   ::= \ZL:w { ⟨MLS⟩⋆ } { ⟨MLS⟩⋆ }
⟨LZ⟩⋆   ::= \LZ:w { ⟨MLS⟩⋆ } { ⟨blnc⟩⋆ }
⟨AZL⟩⋆  ::= \AZL:w { ⟨MLS⟩⋆ } { ⟨blnc⟩⋆ } { ⟨blnc⟩⋆ }
⟨ALZ⟩⋆  ::= \ALZ:w { ⟨MLS⟩⋆ } { ⟨blnc⟩⋆ } { ⟨blnc⟩⋆ }
⟨star⟩⋆ ::= \star:
```

\BNVS_skip_OCMLS:oc:w `\BNVS_skip_OCMLS:oc:w {⟨O error⟩} {⟨C error⟩}`
\BNVS_skip_OCMLS:w `\BNVS_skip_OCML:w`

Utilities. Skip range components, to recover from errors.

```
\cs_set:Npn \BNVS_skip_OCMLS:oc:w #1 #2 {
  \BNVS_subscan:RTN:w { } {
    \BNVS_scanned_set:OCMLS {
      #1
      \BNVS_ast_plc_none: \BNVS_end_scan: \BNVS_skip_OCMLS:oc:w { } { }
    } {
      #2
      \BNVS_ast_plc_none: \BNVS_end_scan: \BNVS_skip_OCMLS:oc:w { } { }
    } {
      \BNVS_ast_plc_none: \BNVS_end_scan: \BNVS_scanned_M:w
    } {
      \BNVS_ast_plc_none: \BNVS_end_scan: \BNVS_scanned_L:w
    } {
      \BNVS_ast_plc_none: \BNVS_end_scan: \BNVS_scanned_S:
    }
  } {
    \BNVS_scaN_OCMLS:w
  }
}

\cs_set:Npn \BNVS_skip_OCMLS:w {
  \BNVS_skip_OCMLS:oc:w { } { }
}
```

\BNVS_scaN_VAZL_MLS:w `\BNVS_scaN_VAZL_MLS:w ⟨input stream⟩`

Scan the input stream for a ⟨*VAZL*⟩ unit. The scanning process will end on the first comma, the first unbalanced closing delimiter (at the same level) or in stop situation, calling one of `\BNVS_scanned_L:w`, `\BNVS_scanned_M:w` or `\BNVS_scanned_S:`. Used by `\BNVS_scaN_VAZL:MLS:w`.

```
}
\cs_new:Npn \BNVS_scaN_VAZL_MLS:w {
```

Create a subscan session to manage the ast policy. The ⟨*pfX*⟩ will depend on the material scanned. The first component is scanned in this session, the other ones are scanned in sub sessions.

```
  \BNVS_subscan:OCMLSN:w {
```

A single colon has been scanned, next operations depend on the emptiness of ⟨*A*⟩.

```
    \BNVS_if_xprted_empty:TF {
```

Scan a `:`⟨*Z*⟩ or a `:`⟨*Z*⟩`::`⟨*L*⟩ range. Keep the same scanning session for ⟨*Z*⟩.

```
      \BNVS_ast_plc_init:XX:n { \star: } { \:Z } { { ####1 } }
      \BNVS_scanned_set:OC {
        \BNVS_error:n { Unexpected~`:',~replaced~by~`::'. }
        \BNVS_scanned_C:w
      } {
        \BNVS_if_xprted_empty:TF {
```

Bad `:` `::...`⟨*L*⟩ range.

```
          \BNVS_error:n { Unexpected~`::...',~ignored. }
          \BNVS_ast_plc_init_mpt:X:n { \star: } { }
          \BNVS_skip_OCMLS:oc:w { } { }
        } {
```

: $\langle Z \rangle$ range.

```
          \BNVS_ast_plc_init_mpt:X:n { } { ########1 }
          \BNVS_xprted_brace:
          \BNVS_scanning_ZL:w
        }
      }
      \BNVS_scaN_OCMLS:w
    } {
      \BNVS_xprted_brace:
      \BNVS_scanning_AZ:w
    }
  } {
```

A double colon has been scanned, next operations depend on the emptiness of $\langle A \rangle$.

```
    \BNVS_if_xprted_empty:TF {
```

Scan a `::`$\langle L \rangle$ or a `::`$\langle L \rangle$`:`$\langle Z \rangle$ range. Keep the same scanning session for $\langle L \rangle$.

```
      \BNVS_ast_plc_init:XX:n { \star: } { \:L } { { ####1 } }
      \BNVS_scanned_set:OC {
        \BNVS_if_xprted_empty:TF {
```

Bad `::` `:...`$\langle L \rangle$ range.

```
          \BNVS_error:n { Unexpected~`:...',~ignored. }
          \BNVS_ast_plc_init_mpt:X:n { \star: } { }
          \BNVS_skip_OCMLS:oc:w { } { }
        } {
```

`::` $\langle L \rangle$ range.

```
          \BNVS_ast_plc_init_mpt:X:n { } { ########1 }
          \BNVS_xprted_brace:
          \BNVS_scanning_LZ:w
        }
      } {
        \BNVS_error:n { Unexpected~`::',~replaced~by~`:'. }
        \BNVS_scanned_O:w
      }
      \BNVS_scaN_OCMLS:w
    } {
      \BNVS_xprted_brace:
      \BNVS_scanning_AL:w
    }
  } {
    \BNVS_end_scan: \BNVS_scanned_M:w
  } {
    \BNVS_end_scan: \BNVS_scanned_L:w
  } {
    \BNVS_end_scan: \BNVS_scanned_S:
  } {
```

By default, we assume a $\langle V \rangle$.

```
    \BNVS_ast_plc_init:X { \:V }
    \BNVS_scaN_OCMLS:w
  }
}
```

\BNVS_scanning_ZL:w

\BNVS_scanning_ZL:w

Utility. Scanning a $:\langle Z\rangle$ or a $:\langle Z\rangle::\langle L\rangle$ range, the $\langle Z\rangle$ is already exported between braces.

```
\cs_new:Npn \BNVS_scanning_ZL:w {
```

Create a subscan process to scan the $\langle L\rangle$ component. Only the \BNVS_scanned_M:w, \BNVS_scanned_L:w or \BNVS_scanned_S: branch functions are expected, the other ones raise.

```
  \BNVS_subscan:RTN:w { } {
    \BNVS_will_end:n {
      \BNVS_if_xprted_empty:TF {
        \BNVS_did_end:n {
          \BNVS_ast_pfX_set:n { \:Z }
        }
      } {
        \BNVS_ast_plc_init_grp:n { ##1 }
        \BNVS_did_end:n {
          \BNVS_ast_pfX_set:n { \:ZL }
        }
      }
    }
    \BNVS_scanned_set:OC {
      \BNVS_error:n { Unexpected~`:...',~ignored. }
      \BNVS_end_scan:
      \BNVS_skip_OCMLS:oc:w { } { }
    } {
      \BNVS_error:n { Unexpected~`::...',~ignored. }
      \BNVS_end_scan:
      \BNVS_skip_OCMLS:oc:w { } { }
    }
  } {
    \BNVS_scaN_OCMLS:w
  }
}
```

\BNVS_scanning_LZ:w

\BNVS_scanning_LZ:w

Utility. Scanning a $::\langle L\rangle$ or a $::\langle L\rangle:\langle Z\rangle$ range, the $\langle L\rangle$ is already exported between braces.

```
\cs_new:Npn \BNVS_scanning_LZ:w {
```

Create a subscan process to scan the $\langle Z\rangle$ component. Only the \BNVS_scanned_M:w, \BNVS_scanned_L:w or \BNVS_scanned_S: branch functions are expected, the other ones raise.

```
  \BNVS_subscan:RTN:w { } {
```

```
    \BNVS_will_end:n {
      \BNVS_if_xprted_empty:TF {
        \BNVS_did_end:n {
          \BNVS_ast_pfX_set:n { \:L }
        }
      } {
        \BNVS_ast_plc_init_grp:n { ##1 }
        \BNVS_did_end:n {
          \BNVS_ast_pfX_set:n { \:LZ }
        }
      }
    }
    \BNVS_scanned_set:OC {
      \BNVS_error:n { Unexpected~`:...',~ignored. }
      \BNVS_end_scan:
      \BNVS_skip_OCMLS:oc:w { } { }
    } {
      \BNVS_error:n { Unexpected~`::...',~ignored. }
      \BNVS_end_scan:
      \BNVS_skip_OCMLS:oc:w { } { }
    }
  } {
    \BNVS_scaN_OCMLS:w
  }
}
```

\BNVS_scanning_AZ:w

\BNVS_scanning_AZ:w

Utility. Scanning a $\langle A \rangle : \langle Z \rangle$ or a $\langle A \rangle : \langle Z \rangle :: \langle L \rangle$ range, only the $\langle A \rangle$ is already exported, between braces.

```
\cs_new:Npn \BNVS_scanning_AZ:w {
```

Create a subscan process to catch the emptiness of $\langle Z \rangle$.

```
  \BNVS_subscan:RTN:w {
    \BNVS_ast_plc_brc:
    \BNVS_scanned_set:OCMLS {
      \BNVS_error:n { Unexpected~`:',~replaced~by~`::'. }
      \BNVS_scanned_C:w
    } {
      \BNVS_end_scan:
      \BNVS_scanning_AZL:w
    } {
      \BNVS_end_scan:
      \BNVS_ast_plc_init:X:n { \:AZ } { ####1 }
      \BNVS_scanned_M:w
    } {
      \BNVS_end_scan:
      \BNVS_ast_plc_init:X:n { \:AZ } { ####1 }
      \BNVS_scanned_L:w
    } {
      \BNVS_end_scan:
      \BNVS_ast_plc_init:X:n { \:AZ } { ####1 }
      \BNVS_scanned_S:
```

```
    }
    \BNVS_scaN_OCMLS:w
  }
}
```

\BNVS_scanning_AZL:w \BNVS_scanning_AZL:w

Utility. Scanning a $\langle A \rangle : \langle Z \rangle$ range or a $\langle A \rangle : \langle Z \rangle : : \langle L \rangle$ range, the $\langle A \rangle$ and $\langle Z \rangle$ are already exported between braces. Only the \BNVS_scanned_M:w, \BNVS_scanned_L:w or \BNVS_scanned_S: are allowed, the other ones raise.

```
\cs_new:Npn \BNVS_scanning_AZL:w {
```

Create a subscan process to catch the emptiness of $\langle L \rangle$.

```
  \BNVS_subscan:RTN:w {
    \BNVS_will_end:n {
      \BNVS_if_xprted_empty:TF {
        \BNVS_ast_plc_none:
        \BNVS_did_end:n {
          \BNVS_ast_plc_init_mpt:X:n { \:AZ } { ##1 }
        }
      } {
        \BNVS_ast_plc_brc:
        \BNVS_did_end:n {
          \BNVS_ast_plc_init_mpt:X:n { \:AZL } { ##1 }
        }
      }
    }
    \BNVS_scanned_set:OC {
      \BNVS_error:n { Unexpected~`:...',~ignored. }
      \BNVS_end_scan:
      \BNVS_skip_OCMLS:w
    } {
      \BNVS_error:n { Unexpected~`::...',~ignored.. }
      \BNVS_end_scan:
      \BNVS_skip_OCMLS:w
    }
    \BNVS_scaN_OCMLS:w
  }
}
```

\BNVS_scanning_AL:w \BNVS_scanning_AL:w

Utility. Scanning a $\langle A \rangle : : \langle L \rangle$ or a $\langle A \rangle : : \langle L \rangle : \langle Z \rangle$ range, only the $\langle A \rangle$ is already exported, between braces.

```
\cs_new:Npn \BNVS_scanning_AL:w {
```

Create a subscan process to catch the emptiness of $\langle L \rangle$.

```
  \BNVS_subscan:N:w {
```

```
    \BNVS_ast_plc_brc:
    \BNVS_scanned_set:OCMLS {
      \BNVS_end_scan:
      \BNVS_scanning_ALZ:w
    } {
      \BNVS_error:n { Unexpected~`::',~replaced~by~`:'. }
      \BNVS_scanned_O:w
    } {
      \BNVS_end_scan:
      \BNVS_ast_plc_init:X:n { \:AL } { ####1 }
      \BNVS_scanned_M:w
    } {
      \BNVS_end_scan:
      \BNVS_ast_plc_init:X:n { \:AL } { ####1 }
      \BNVS_scanned_L:w
    } {
      \BNVS_end_scan:
      \BNVS_ast_plc_init:X:n { \:AL } { ####1 }
      \BNVS_scanned_S:
    }
    \BNVS_scaN_OCMLS:w
  }
}
```

`\BNVS_scanning_ALZ:w` `\BNVS_scanning_ALZ:w`

Utility. Scanning a $\langle A \rangle :: \langle L \rangle$ or $\langle A \rangle :: \langle L \rangle : \langle Z \rangle$ range, the $\langle A \rangle$ and $\langle AL \rangle$ are already exported between braces. Only the `\BNVS_scanned_M:w`, `\BNVS_scanned_L:w` or `\BNVS_scanned_S:` are allowed, the other ones raise.

```
\cs_new:Npn \BNVS_scanning_ALZ:w {
```

Create a subscan process to catch the emptiness of $\langle Z \rangle$.

```
  \BNVS_subscan:N:w {
    \BNVS_will_end:n {
      \BNVS_if_xprted_empty:TF {
        \BNVS_ast_plc_none:
        \BNVS_did_end:n {
          \BNVS_ast_plc_init_mpt:X:n { \:AL } { ##1 }
        }
      } {
        \BNVS_ast_plc_brc:
        \BNVS_did_end:n {
          \BNVS_ast_plc_init_mpt:X:n { \:ALZ } { ##1 }
        }
      }
    }
    \BNVS_scanned_set:OC {
      \BNVS_error:n { Unexpected~`:...',~ignored. }
      \BNVS_end_scan:
      \BNVS_skip_OCMLS:w
    } {
      \BNVS_error:n { Unexpected~`::...',~ignored.. }
      \BNVS_end_scan:
      \BNVS_skip_OCMLS:w
```

```
    }
    \BNVS_scaN_OCMLS:w
  }
}
```

\BNVS_scaN_VAZL:MLS:w `\BNVS_scaN_VAZL:MLS:w {⟨on_M:w⟩} {⟨on_L:w⟩} {⟨on_S:⟩} ⟨input stream⟩`

Scan the input stream for a ⟨*VAZL*⟩ unit. The scanning process will end on the first comma, the first unbalanced closing delimiter (at the same level) or in stop situation. The \BNVS_scaN_OCMLS:w instructions are used in the background to scan the ⟨*input stream*⟩, they end with one of \BNVS_scanned_M:w, \BNVS_scanned_L:w or \BNVS_scanned_S:.

```
\cs_new:Npn \BNVS_scaN_VAZL:MLS:w #1 #2 #3 {
```

Create a subscan process to manage the branch parameters.

```
  \BNVS_subscan:MLSN:w {
    \BNVS_end_scan: #1
  } {
    \BNVS_end_scan: #2
  } {
    \BNVS_end_scan: #3
  } {
    \BNVS_ast_plc_init:n { ##1 }
    \BNVS_scaN_VAZL_MLS:w
  }
}
```

1.12.14 VAZL lists

Here is a preliminary grammar for ranges or values:

```
⟨VAZLRnd⟩ ::= ⟨Rnd_open⟩ ⟨VAZLLst⟩ ⟨Rnd_close⟩
⟨VAZLLst⟩ ::= ⟨spc⟩ ( ⟨VAZL⟩ ( ⟨comma⟩ ⟨VAZL⟩ )* )? ⟨spc⟩
```

and the associate AST :

```
⟨VAZLRnd⟩*::= { ⟨VAZLLst⟩* }
⟨VAZLLst⟩*::= ( { ⟨VAZL⟩* } ) *
```

\BNVS_scaN_VAZLLst_LS:w `\BNVS_scaN_VAZLLst_LS:w ⟨input stream⟩`

Scan the ⟨*input stream*⟩ for a comma separated list of ⟨*VAZL*⟩ entries. Terminates branching to \BNVS_scanned_S: in stop situation, branching to \BNVS_scanned_B:w otherwise.

```
\cs_new:Npn \BNVS_scaN_VAZLLst_LS:w {
```

```
  \BNVS_scaN_csl:RLSTN:w { } {
    \BNVS_scanned_L:w
  } {
    \BNVS_scanned_S:
  } {
  } {
    \BNVS_scaN_VAZL_MLS:w
  }
}
```

\BNVS_scaN_VAZLRnd_BS:X:Fw

\BNVS_scaN_VAZLRnd_BS:X:Fw {⟨*pfX*⟩}
{⟨*:w*⟩}
⟨*input stream*⟩

Scan ⟨*VAZLRnd*⟩ if any, otherwise execute ⟨*:w*⟩ instructions.

```
\cs_new:Npn \BNVS_scaN_VAZLRnd_BS:X:Fw #1 #2 {
  \BNVS_scaN_Rnd:RBSTN:Fw { } {
    \BNVS_scanned_B:w
  } {
    \BNVS_scanned_S:
  } {
    \BNVS_ast_plc_init:X:n { #1 } { { ##1 } }
  } {
    \BNVS_scaN_VAZLLst_LS:w
  } { #2 }
}
```

1.12.15 Resolution units

Here is the grammar

```
⟨Rslv⟩ ::= ⟨spc⟩ '?' ⟨Rnd⟩
```

with the associate AST

```
⟨Rslv⟩* ::= \Rslv:n { ⟨Rnd_csl⟩* }
```

\BNVS_scaN_Rslv:RBSTN:Fw
\BNVS_scaN_Block_Rslv:B:Fw

\BNVS_scaN_Rslv:RBSTN:Fw {⟨*R:*⟩} {⟨*scaN:B:w*⟩} {⟨*scaN:S:*⟩} {⟨*T:*⟩} {⟨*scaN:w*⟩}
{⟨*F:w*⟩} ⟨*input stream*⟩
\BNVS_scaN_Block_Rslv:B:Fw {⟨*yes:w*⟩} {⟨*F:w*⟩} ⟨*input stream*⟩

Scan a ⟨*Rslv*⟩ unit, if any, and execute one of ⟨*block:*⟩ or ⟨*stop:*⟩ instructions and the eponym branch functions, otherwise execute ⟨*F:w*⟩ instructions.

```
\cs_new:Npn \BNVS_scaN_Rslv:RBSTN:Fw #1 #2 #3 #4 #5 #6 {
  \peek_charcode_remove:NTF ? {
    \BNVS_scaN_Rnd:RBSTN:Fw {
      \BNVS_ast_plc_init:n { \Rslv:n { ##1 } }
      \BNVS_scaN_Block_def::BF {
        \BNVS_scaN_Block_Rslv:B:Fw { ##1 } { ##2 }
```